M&A CYBERSECURITY DUE DILIGENCE

RedFlag

Documentation

The complete documentation set for the RedFlag platform, comprising the handover, technical, operational, user, testing, legal and process material, together with the architecture decision records. Generated from the Markdown sources under docs/.

REPOSITORY	github.com/adityaa206/redflag
STATUS	Complete — documented for handover
COMPILED	27 July 2026
SOURCE	Markdown in docs/ — this PDF is generated from it

Contents

FRONT MATTER

Front matter

Index and documentation conventions

Documentation Index

Documentation Index

The complete documentation set for RedFlag, the M&A cybersecurity due-diligence platform.

The whole set is also available as a single bookmarked PDF — **RedFlag_Documentation.pdf** (201 pages) — regenerated from these Markdown sources with `python tools/build_docs_pdf.py . <output-dir>`.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

What RedFlag is

RedFlag is an end-to-end **M&A cybersecurity due-diligence platform**. Given a target company, it maps the internet-facing attack surface, scores every finding with an SSVC/EPSS-aligned risk model, checks DNS/email security, TLS health and breach exposure, assesses internal security maturity, plans Day-1 connectivity, estimates remediation and integration cost, and reasons about attack paths — all inside a single Reflex web application. Its core is deliberately **offline, free, and LLM-free**: same input, same output, no paid API, no GPU.

Handover

DOCUMENT	WHAT IT COVERS
HANDOVER.md	Project status, what shipped, where everything lives, sign-off checklist
PROJECT_REPORT.md	Problem, objectives, scope, solution, outcomes
ACCESS_AND_CREDENTIALS.md	Account/key inventory and transfer checklist (no secrets)
KNOWN_ISSUES_AND_BACKLOG.md	Known bugs, incomplete features, tech debt, next steps
KNOWLEDGE_TRANSFER.md	Tacit knowledge, gotchas, "why it's like this"

Technical

DOCUMENT	WHAT IT COVERS
ARCHITECTURE.md	Layering, component and data-flow diagrams, module responsibilities
DATA_MODEL.md	The `Finding` model and every enum, verified from `analysis/schema.py`
MODULE_REFERENCE.md	Public function signatures per package, with side effects
CONFIGURATION.md	Every `.env` variable, constant and YAML knob, with retuning recipes
INTEGRATIONS.md	Each external tool/API: endpoints, keys, limits, degradation

DOCUMENT	WHAT IT COVERS
BRAIN_KNOWLEDGE_BASE.md	The self-improving offline knowledge base

Operations

DOCUMENT	WHAT IT COVERS
INSTALLATION.md	Prerequisites and step-by-step install (Windows + macOS)
DEVELOPER_ONBOARDING.md	Get productive: layout tour, tests, conventions, extension points
DEPLOYMENT_RUNBOOK.md	Running and operating the app; ports, outputs, cautions
TROUBLESHOOTING.md	Symptom → cause → fix reference

User

DOCUMENT	WHAT IT COVERS
USER_GUIDE.md	Running a scan and a walkthrough of all seven tabs
RESULTS_INTERPRETATION.md	How to read and trust every number RedFlag prints

Testing

DOCUMENT	WHAT IT COVERS
TEST_PLAN.md	How to run the suite, per-file coverage map, what is *not* covered
LIMITATIONS.md	Honest accuracy caveats and scope boundaries

Legal & security

DOCUMENT	WHAT IT COVERS
AUTHORIZED_USE.md	Legal and ethical boundaries of running an active scanner
LICENSES_AND_ATTRIBUTION.md	Project licence, dependency SBOM, external-service terms
SECURITY_AND_PRIVACY.md	Data handling, the egress table, secrets management

Process

DOCUMENT	WHAT IT COVERS
CHANGELOG.md	Version and milestone history (canonical copy at repo root)
ROADMAP.md	Forward direction if the project continues
adr/	Eight Architecture Decision Records with rationale

Conventions

Each fact is recorded in exactly one canonical document; other documents cross-reference it rather than repeating it. Directories are organised by audience: `handover/` addresses the receiving supervisor, `technical/` the maintaining engineer, `operations/` the operator, and `user/` the analyst. `testing/`, `legal/` and `process/` hold the assurance, compliance and history material.

Every figure, signature, threshold and configuration key in this set was read from the source rather than from the README. Where the two disagreed, the code was treated as the truth and the README was corrected to match.

Related documents

- Repository front page: ../README.md
- Licence: ../LICENSE
- Contribution guide: ../CONTRIBUTING.md
- Vulnerability reporting: ../SECURITY.md

PART I

Handover

Project status, transition and outstanding items

Handover

The master transition document: what I built, where it lives, and how to pick it up.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Project status

Status: complete and handed over. RedFlag runs end to end as a single Reflex web application with a 143-test engine suite passing. Every capability listed in section 2 is implemented, wired into the UI, and exercised by tests or by manual run-through.

"Done" here means:

- The full pipeline (scan → enrich → merge → score → assess → plan → cost → narrate → export) executes from a single **Run scan** click.
- All thresholds, weights, pricing, questions and narrative text are config-driven; no behaviour requires a code change to retune.
- The tool degrades gracefully with no API keys, no Nuclei binary, and no network for the optional feeds.
- Documentation for handover is in place (this set).

It does **not** mean the roadmap is exhausted — see KNOWN_ISSUES_AND_BACKLOG.md.

2. What was delivered

Scanning and evidence fusion

- Active Nmap scanning with a full mode and a top-200-port **Fast Scan Mode**.
- Passive Shodan OSINT (live API lookup, or an uploaded Shodan host JSON that takes priority).
- Nuclei template DAST — runs the local binary if present, or ingests uploaded JSONL.
- OpenVAS/GVM XML and OWASP ZAP XML ingestion that **correlate into** the Nmap layer by `host:port`, upgrading matched findings rather than replacing them.
- Vulners NSE parsing plus optional Vulners API exploit confirmation.
- DNS/email security audit (SPF, DMARC, DKIM, DNSSEC).
- TLS certificate health plus crt.sh certificate-transparency subdomain discovery.
- LeakIX breach and exposure lookup.
- Excel asset-inventory ingestion that stamps data sensitivity onto matching hosts.

Intelligence enrichment

- CISA KEV cross-reference for active-exploitation status.
- NVD CVSS v3.1 lookup with in-process caching and parallel batch fetch.
- FIRST.org EPSS exploitation probability, including **promotion** of a high-probability CVE's exploit status.

Analysis

- A weighted 0–100 risk score with an evidence-strength multiplier and four deal-killer override rules.
- A 23-question / 7-domain maturity assessment compared against a configurable corporate standard.
- A Day-1 Safe Harbor Blueprint: connectivity ladder, tier gates, review pillars, P0–P3 roadmap.
- An offline MITRE ATT&CK attacker-brain that chains findings into a kill-chain and renders a radial mind-map as SVG.
- A networkx attack-graph that quantifies chokepoints, blast radius, and shortest paths to crown-jewel data.
- A persistent, self-improving knowledge base ("the brain") that accumulates across scans and writes an Obsidian-compatible vault.

Costing and reporting

- A YAML-driven remediation cost engine with deduplication, low/base/high scenarios, CapEx/OpEx split, and a human-review gate.
- A Day-1 **integration budget** that prices the recommended connectivity model — and every rung of the ladder — with sourced 2026 pricing, vendor-quote overrides, and a variance-based 80% confidence interval.
- A deterministic, YAML-backed narrative engine (no LLM).
- CSV export plus three PDF reports (full, Day-1, cost).

3. What is not done

Nothing in the shipped feature set is known to be broken. What remains is unbuilt roadmap work — compliance framework mapping, secret scanning, multi-target comparison, containerised startup — plus one piece of superseded code that is no longer imported.

Full detail, including the tech-debt register: KNOWN_ISSUES_AND_BACKLOG.md.

4. Where everything lives

THING	LOCATION
Repository	`github.com/adityaa206/redflag` (remote `origin`)
Working checkout	`C:\Users\Adityaa\Redflag`
Documentation	`docs/` (this set)
Reflex app entry point	`rxconfig.py` → `redflag_ui/redflag_ui.py`
UI state and pipeline	`redflag_ui/state.py` `RedFlagState.run_scan`)
Engines	`analysis/`, `scanners/`, `cost/`, `narrative/`, `reports/`
Configuration	`config/` — `__init__.py` constants + seven YAML files
Tests and fixtures	`tests/` (143 tests), `tests/fixtures/`

THING	LOCATION
Licence	`LICENSE` — MIT
Secrets	`.env` in the repo root — **git-ignored**, never committed
Scan output (runtime)	`%TEMP%/redflag_scans` (Windows) / `$TMPDIR/redflag_scans`
Knowledge base (runtime)	`~/RedFlag-Brain` — override with `REDFLAG_BRAIN_DIR`
Shipped brain seed	`analysis/brain_seed/brain.json` (sanitised; identities stripped)

> NOTE
>
> The live scan output and the brain deliberately live **outside** the repository tree. Writing inside it trips Reflex's dev file-watcher, which hot-reloads the backend mid-scan and loses the findings. See KNOWLEDGE_TRANSFER.md.

5. How to run it in one minute

```
cd C:\Users\Adityaa\Redflag
.\venv\Scripts\Activate.ps1
python -m reflex run
```

Frontend on <http://localhost:3000>, backend on <http://localhost:8000>. The first launch compiles a Next.js frontend and needs Node.js 18+ (Reflex offers to install it).

Full instructions, including a clean install from scratch: INSTALLATION.md.

6. Access and ownership

RedFlag is a **local-only** tool. There is no hosted deployment, no database, and no cloud account to transfer. What does need transferring is the repository and, if they are to be reused, the two optional API keys.

- Full inventory and transfer steps: ACCESS_AND_CREDENTIALS.md
- Data-handling posture: SECURITY_AND_PRIVACY.md

The repository is `github.com/adityaa206/redflag`. Access is granted through GitHub's collaborator settings; both procedures — adding a maintainer and transferring ownership outright — are written out in ACCESS_AND_CREDENTIALS.md §1.

7. Picking it up

Everything needed to run, understand, extend and operate RedFlag is in this documentation set. The shortest useful path through it:

TO DO THIS	READ
Get it running on a new machine	INSTALLATION.md
Understand how it is put together	ARCHITECTURE.md
Run an assessment and read the output	USER_GUIDE.md, RESULTS_INTERPRETATION.md
Change a threshold, weight or price	CONFIGURATION.md
Add a scanner or extend an engine	DEVELOPER_ONBOARDING.md, MODULE_REFERENCE.md
Know what is deliberately *not* guaranteed	LIMITATIONS.md
Run a scan lawfully	AUTHORIZED_USE.md
Understand the non-obvious parts before changing them	KNOWLEDGE_TRANSFER.md

The two things worth doing first on a new machine, because they surface most setup problems immediately, are a clean install and a test run:

```
pytest tests/ -v
```

143 tests, no network, no target, no external binary required. If they pass, the engines are intact and only the environment-specific parts — the Nmap binary and the Node.js frontend build — remain to be verified, both of which the installation guide covers.

Related documents

- PROJECT_REPORT.md — the narrative account of the work
- KNOWN_ISSUES_AND_BACKLOG.md — what remains
- KNOWLEDGE_TRANSFER.md — the non-obvious parts
- ARCHITECTURE.md — how the system is put together
- INSTALLATION.md — full setup

Project Report

My account of the problem, the approach I took, and what came out of it.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Problem statement

In a merger or acquisition, the acquirer inherits the target's security posture in full — its unpatched systems, its exposed services, its prior breaches, and its regulatory liabilities — at the moment the deal closes. Financial and legal due diligence have mature, standardised processes. Technical security due diligence largely does not.

The practical difficulties are three:

1. **Evidence is fragmented.** An Nmap sweep, a Shodan record, an OpenVAS report and a ZAP scan each describe a slice of the same estate in a different vocabulary. Read separately, they produce four unranked lists rather than one decision.
2. **Severity is not risk.** A CVSS 9.8 on an internal test box is not the same commercial risk as a CVSS 7.5 on an internet-facing system holding regulated data. Deal decisions need the second framing, not the first.
3. **Risk is hard to price.** "There are 47 findings" does not tell a deal team what to negotiate. "Remediation is $180K–$420K, and connecting the two networks safely on Day 1 costs another $310K" does.

RedFlag exists to close that gap: to turn heterogeneous scanner output into one ranked, priced, sequenced risk picture that a deal team can act on.

2. Objectives

The objectives I set out to meet, and which the implementation is built against:

- Aggregate multiple scanner sources into a single, deduplicated finding set.
- Score findings on commercial risk, not raw severity.
- Produce an inside-out maturity view to complement the outside-in scan.
- Answer the Day-1 question: *how do we connect the two organisations safely?*
- Attach a defensible cost to the answer.
- Do all of it at **zero running cost** — no paid API, no LLM, no GPU.

3. Scope

In scope

- Single-target assessment (one hostname or IP per run), plus uploaded scanner artefacts.

- Outside-in technical scanning and OSINT enrichment.
- Inside-out maturity questionnaire across seven security domains.
- Risk scoring, Day-1 connectivity planning, cost modelling, attack-path reasoning.
- Deterministic narrative generation and CSV/PDF export.
- Local, single-user operation.

Out of scope

- Authenticated or credentialed scanning of the target (RedFlag ingests OpenVAS/ZAP output but does not perform authenticated scans itself).
- Multi-target or portfolio comparison.
- Subnet-wide Shodan enumeration (single host only).
- Continuous monitoring — RedFlag is a point-in-time assessment.
- Any form of exploitation. RedFlag observes and reasons; it never attacks.
- Multi-user access control, hosting, or persistence beyond local files.

4. Solution overview

RedFlag is a thirteen-step pipeline behind a single button. Full detail lives in ARCHITECTURE.md; the shape of it:

1. **Collect** — Nmap scans the target; staged uploads (Shodan JSON, OpenVAS XML, ZAP XML, Nuclei JSONL, asset Excel) are read alongside it.
2. **Correlate** — uploaded scanner results are merged into the Nmap layer by `host:port`. A match upgrades the existing finding's evidence strength, CVSS and exploit status; it does not create a duplicate.
3. **Enrich** — CISA KEV, NVD CVSS, Vulners and FIRST.org EPSS attach exploit intelligence. DNS, TLS and breach scanners contribute findings of their own.
4. **Score** — every finding gets a weighted 0–100 risk score, an evidence-strength multiplier, and a deal tier; three override rules force a deal-killer verdict outright.
5. **Assess** — the maturity questionnaire produces per-domain scores against a corporate standard, generating gaps that feed both the Day-1 roadmap and the cost engine.
6. **Plan** — the Day-1 engine recommends the highest connectivity tier whose entry gate passes, and sequences every finding and gap into P0–P3.
7. **Reason** — the attacker-brain chains findings into a MITRE ATT&CK kill-chain and mind-map; the attack graph measures chokepoints, blast radius, and paths to crown-jewel data.
8. **Price** — the cost engine converts findings, gaps and the recommended connectivity model into low/base/high line items with a CapEx/OpEx split and a confidence interval.
9. **Learn** — the run is folded into a persistent knowledge base so the next scan starts smarter.
10. **Report** — deterministic narrative text plus CSV and three PDF reports.

The subsystems, and why they are separated:

SUBSYSTEM	RESPONSIBILITY
`scanners/`	Talk to the outside world. Every function degrades to empty rather than raising.
`analysis/`	Pure, deterministic reasoning over findings. No network, no UI.
`cost/`	Convert findings and plans into money. YAML-priced, catalogue-driven.
`narrative/`	Template-driven prose. Same context in, same sentence out.
`reports/`	Serialisation to CSV and PDF.
`config/`	Every tunable number and string, in YAML.
`redflag_ui/`	Presentation only. Calls engines, flattens results into view-models.

The strict separation is the point: the UI was migrated from Streamlit to Reflex without a single change to the scoring, costing or planning logic.

5. Key design decisions

Eight decisions shaped the system. Each has a full record in docs/process/adr/.

#	DECISION	ONE-LINE RATIONALE
0001	No LLM anywhere in the core	Deal documents must be reproducible and auditable; a stochastic generator is neither
0002	Deterministic YAML narrative engine	Analysts can read and edit the exact sentences the tool will print
0003	Entirely free, no paid API	Removes a per-scan cost that would otherwise discourage use
0004	Reflex as the UI framework	Multi-page routed app in pure Python; Streamlit's rerun model could not carry it
0005	Brain accumulates, never trains	Retrieval over a growing corpus gives compounding value with no GPU and no drift
0006	Evidence-strength multiplier	A confirmed finding should outrank a banner-only inference at equal severity
0007	EPSS can promote exploit status	Makes the model forward-looking rather than purely retrospective
0008	Uploads correlate into Nmap	Preserves the port-level ground truth while raising confidence

6. Results and outcomes

Delivered capabilities — twelve integrated scanners and feeds, a four-factor weighted scoring model, a seven-domain maturity assessment, a four-tier Day-1 connectivity planner, two complementary attack-path engines (narrative and quantitative), a self-improving knowledge base, a two-bucket cost model with statistical confidence intervals, and four export formats.

Engineering outcomes

- 143 automated tests across eleven test modules, covering scoring, maturity, cost, narrative, Day-1, EPSS, Nuclei, attack-graph, simulation and end-to-end integration.
- A UI framework migration (Streamlit → Reflex) completed with **zero changes** to the engines — a direct validation of the layering.
- Complete configuration externalisation: seven YAML files plus one constants module drive every threshold, weight, price, question and sentence.

Operational status. RedFlag has been exercised end to end against authorised test targets and against the committed mock fixtures, which drive the full pipeline — merge, score, assess, plan, cost, narrate, export — without touching a live host. It has not been used to assess a real acquisition target, and no client report has been produced from it. It is a complete, working tool that has not yet been put in front of a live deal.

7. Objectives met vs. outstanding

OBJECTIVE	STATUS	NOTE
Fuse multiple scanners into one finding set	**Met**	Correlation merge for OpenVAS, ZAP, Nuclei; Shodan enrichment; five upload slots
Score commercial risk, not severity	**Met**	Four-factor weighting + evidence multiplier + four override rules
Inside-out maturity view	**Met**	23 questions, 7 domains, configurable corporate standard
Answer the Day-1 connectivity question	**Met**	Four-tier ladder with pass/blocked gates and a P0–P3 roadmap
Price the risk	**Met**	Remediation + integration buckets, low/base/high, 80% CI
Zero running cost	**Met**	Every core path works with no API key; optional keys only add enrichment
Attack-path reasoning	**Met**	MITRE ATT&CK narrative brain + networkx quantitative graph
Compliance framework mapping	**Outstanding**	ISO 27001 / NIST CSF / SOC 2 / GDPR / PCI-DSS mapping not built
Multi-target comparison	**Outstanding**	Single target per run
Secret scanning	**Outstanding**	trufflehog integration not built
Containerised startup	**Outstanding**	No Docker Compose

Outstanding items are tracked in ROADMAP.md and KNOWN_ISSUES_AND_BACKLOG.md.

8. Lessons learned

Architectural boundaries pay for themselves at the worst possible moment. Replacing the entire user interface — Streamlit for Reflex — took zero changes to `analysis/`, `scanners/`, `cost/`, `narrative/` and `reports/`. That was not luck. It was the result of a rule I held to from the first commit: the UI may call an engine and format its output, but it may never contain a decision. Had scoring logic leaked into the presentation layer, the migration would have meant rewriting the risk model under time pressure, with the tests no longer describing

the thing being rewritten. The discipline felt pedantic while I was writing it and paid for itself in a single afternoon.

Determinism is a feature when the output is evidence. Choosing a template engine over a language model constrained what RedFlag can say. What it bought is that the same findings always produce the same report, every sentence traces to a YAML block someone can read and edit, and a number in a deal document can be defended line by line. For a tool whose output may inform a purchase price, being reproducible mattered more than being fluent.

Constraints shaped the design rather than limiting it. The requirement to run at zero cost ruled out commercial vulnerability feeds and forced me toward CISA KEV, FIRST.org EPSS, NVD, crt.sh and LeakIX. Those turned out to be the right sources anyway: KEV is the authoritative record of what is actually being exploited, and EPSS is a better predictor of near-term exploitation than CVSS severity. The free path was also the more defensible one.

A pipeline with many integrations survives only if every part is allowed to fail. Fourteen integrations means fourteen chances for a scan to die on a timeout, a rate limit, a missing binary or an expired key. Every scanner returns `[]` rather than raising, and the orchestration catches broadly on purpose. The cost is that a genuine bug can be swallowed silently — a trade-off I made deliberately and recorded in the tech-debt register rather than leaving implicit.

Externalising configuration is what makes a scoring model arguable. Weights, thresholds, prices, questions and narrative text all live in YAML and one constants module. That means a disagreement about whether exposure should outweigh CVSS is settled by editing a file and re-running, not by reading Python. A risk model nobody can adjust is a risk model nobody trusts.

9. Timeline

Reconstructed from the commit history (dates are commit dates on `origin/main`):

DATE	MILESTONE
2026-05-25	Initial commit; scanner pipeline (Vulners NSE, ZAP, OpenVAS), PDF report, config centralisation
2026-05-28	Comprehensive README
2026-05-29	Maturity Assessment, Cost Engine and Narrative Template Engine (Build Spec v2)
2026-06-05	DNS/email security, TLS health, breach check, What-If simulator
2026-06-08	Attack Path tab
2026-06-09	`requirements.txt`; Windows + macOS installation guides
2026-06-23	First Reflex UI variant (Executive Editorial / emerald)
2026-06-24	Day-1 Safe Harbor Blueprint
2026-06-29	**Migration to Reflex complete; Streamlit `app.py` retired.** Sanitised brain seed shipped
2026-06-30	Nuclei scanning, EPSS scoring, attack-graph analytics
2026-07-02	Day-1 integration budget; variance-based confidence interval; vendor-quote overrides
2026-07-27	Documentation set for handover

The work ran continuously from the initial commit on 2026-05-25 to the documentation set on 2026-07-27 —

roughly nine weeks. The five natural phases visible above are the scanner pipeline, the analysis and costing engines, the Phase-1 outside-in scanners, the Reflex migration, and the Day-1 integration budget.

Related documents

- HANDOVER.md — the transition cover document
- ARCHITECTURE.md — how the solution is structured
- adr/ — the decision records behind section 5
- ROADMAP.md — where it could go next
- TEST_PLAN.md — evidence for the quality claims

Access & Credentials

An inventory of every account and key needed to own RedFlag, and how to transfer each one.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

> NOTE
>
> **This document contains no secrets and must never contain any.** It is an inventory. Actual key values live only in a local, git-ignored `.env` file, or in a password manager. If a real key is ever found in this file or anywhere else in the repository, treat it as compromised and rotate it immediately.

1. Repository ownership

ITEM	VALUE
Repository	`github.com/adityaa206/redflag`
Remote name	`origin`
Default branch	`master` on GitHub; `main` and `master` are kept in sync
Secondary remote	`upstream` → `quadindy/marisk` (a separate account; RedFlag does not publish there)

To add a maintainer (recommended for a supervisor who needs read/write but not ownership): GitHub → repository → **Settings → Collaborators and teams → Add people** → grant `Write` or `Maintain`.

To transfer ownership outright: GitHub → repository → **Settings → General → Danger Zone → Transfer ownership**. Note that this moves issues, stars and the URL; the old URL will redirect.

Adding a maintainer is the lighter option and is sufficient for anyone who needs to read, clone and contribute. Ownership transfer is only necessary if the repository is to leave the `adityaa206` account entirely.

The `upstream` remote points at a different account's repository and exists only as a fetch reference. RedFlag is published to `origin`; nothing in this project is pushed to `upstream`.

2. API key inventory

Both keys are **optional**. RedFlag runs a complete assessment with neither — see section 5.

SERVICE	ENV VARIABLE	USED BY	FREE TIER	SIGN-UP	REQUIRED?
Shodan	`SHODAN_API_KEY`	`scanners/shodan_scan.py` → `lookup_host()`	Yes (limited credits)	account.shodan.io	No

SERVICE	ENV VARIABLE	USED BY	FREE TIER	SIGN-UP	REQUIRED?
Vulners	`VULNERS_API_KEY`	`scanners/vulners_enrich.py`; also passed to the Nmap `vulners` NSE script	Yes	vulners.com/userinfo	No

There is also one **non-secret** environment variable:

VARIABLE	PURPOSE	DEFAULT
`REDFLAG_BRAIN_DIR`	Overrides where the knowledge base is stored	`~/RedFlag-Brain`

Cost note: a live Shodan lookup consumes **1 credit per IP queried**. Uploading a Shodan host JSON instead costs nothing and takes priority over the live call.

Full behavioural detail per integration: INTERGRATIONS.md.

3. Secrets handling policy

1. Keys live **only** in `.env` in the repository root.
2. `.env` is listed in `.gitignore` and must never be committed. `.env.example` is the committed template and contains placeholders only.
3. To provision a fresh environment:

```
# Windows
copy .env.example .env
notepad .env
```

```
# macOS / Linux
cp .env.example .env
nano .env
```

4. Never paste a key into a screenshot, a chat message, an issue, a commit message, or a documentation file.
5. If a key is exposed anywhere, rotate it first and investigate second.

4. Key rotation

Both keys are personal to whoever holds the account, so the cleanest position for anyone taking the project on is to issue their own rather than inherit these. RedFlag needs no key to function, so a new holder can also simply run without them. The procedures below are recorded for either case.

How to rotate a Shodan key

1. Sign in at account.shodan.io.
2. Open the **API Key** section — the key shown there is your active key.

3. Use the **Reset / regenerate** control to issue a new key. The previous key stops working immediately.
4. Update `SHODAN_API_KEY` in your local `.env`.
5. Confirm the old value appears nowhere in the repository history, in screenshots, or in shared documents.

How to rotate a Vulners key

1. Sign in at vulners.com/userinfo.
2. Open **API Keys**, revoke the existing key, and create a replacement scoped to `api`.
3. Update `VULNERS_API_KEY` in `.env`.

If the key must not be reused by the receiver: revoke it and let them create their own free account. RedFlag needs no key to function.

5. Running with no keys at all

WHAT IS LOST	WORKAROUND
Live Shodan host lookup	Upload a Shodan host JSON in the **Shodan JSON** slot — it takes priority over the live call anyway
Vulners exploit confirmation	Findings still score; `exploit_status` simply stays `UNKNOWN` unless KEV or EPSS supplies it
Nothing else	NVD, CISA KEV, EPSS, DNS, TLS, crt.sh and LeakIX all work with no key

6. Other accounts and infrastructure

ITEM	STATUS
Hosting / cloud deployment	**None.** RedFlag is a local desktop application.
Database	**None.** All state is in-memory or in local files.
CI/CD	**None configured.**
Container registry	**None.**
Monitoring / error tracking	**None.**
External binaries required	Nmap (required), Nuclei (optional), Node.js 18+ (installed by Reflex on first run)

This inventory is complete. The repository contains no Dockerfile, no `.github/` workflows, no infrastructure-as-code, and no deployment configuration of any kind; `rxconfig.py` declares no remote API or deploy URL. RedFlag has only ever run on a local machine.

7. Setting up on a new machine

#	STEP
1	Clone the repository, or accept the collaborator invitation and then clone
2	Copy `.env.example` to `.env` — RedFlag runs fully without editing it
3	Add a Shodan or Vulners key to `.env` only if live enrichment from those services is wanted
4	Confirm `.env` is git-ignored: `git check-ignore -v .env`
5	Read AUTHORIZED_USE.md before running any scan against a host

No secret appears in any tracked file in this repository. `.env` is git-ignored and `.env.example` contains placeholders only.

Related documents

- HANDOVER.md — the transition cover document
- SECURITY_AND_PRIVACY.md — what data leaves the machine
- INTEGRATIONS.md — per-service auth and degradation behaviour
- CONFIGURATION.md — every environment variable and config knob

Known Issues & Backlog

Outstanding defects, unbuilt features, technical debt, and a suggested order of work.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Known defects

Each is verified against the code. None prevents a scan from running.

1.1 LICENSE file was missing — resolved

`README.md` stated "*MIT — see LICENSE for details*" and linked to `LICENSE`, but no such file was present, which left the repository strictly **all rights reserved** — a licence claim with no licence text is ambiguous rather than permissive. A standard MIT `LICENSE` is now committed at the repository root and matches the README's declaration. Recorded in LICENSES_AND_ATTRIBUTION.md §1.

1.2 The README's test count was stale — resolved

`README.md` stated **128 passing tests** in three places: the badge, the architecture tree and the "Running Tests" section. The suite contains **143 tests**, all passing. All three references now state 143. Recorded in TEST_PLAN.md.

1.3 .gitignore excluded the entire documentation set — resolved

`.gitignore` contained a blanket `*.md` rule. Because `README.md` was committed before that rule landed, git kept tracking it, which masked the problem — but **any new Markdown file was silently ignored**, including this entire `docs/` tree, `CONTRIBUTING.md`, `SECURITY.md` and `CHANGELOG.md`.

Explicit negations now version the published documentation and the root community files, while agent and scratch notes remain local. The whole `docs/` tree, the community files and `LICENSE` are tracked; anything generated stays out, with one deliberate exception — `docs/RedFlag_Documentation.pdf`, the rendered book, is versioned so it can be read without building it.

1.4 tests/fixtures/mocknuclei.jsonl was untracked — resolved

The Nuclei mock fixture (4.2 KB) was present on disk but absent from version control, so a fresh clone could not reproduce the documented Nuclei upload workflow. It is now committed alongside the OpenVAS and ZAP mocks.

1.5 The README fixture table omitted the Nuclei mock — resolved

`README.md` → *Using the Mock Data Files* listed `mock_openvas.xml`, `mock_zap.xml` and `sample_assessment.json`, but not `mock_nuclei.jsonl`. The table now lists all four. Also documented in USER_GUIDE.md.

1.6 The README documented three deal-killer override rules — resolved

`analysis/triage.py` → `check_override_rules()` implements **four**; the README's Scoring Reference listed

three, omitting the manual analyst flag that fires when `override_reason` contains `"active compromise"`. The README table now lists all four. The full set is documented in RESULTS_INTERPRETATION.md §3.

2. Incomplete features

From the README roadmap, unchecked items — none are started.

FEATURE	NOTES
Free-tier LLM narrative layer over the brain	Explicitly optional and additive. Must not replace the deterministic engine — see ADR-0001 and ADR-0002.
Compliance gap mapping (ISO 27001 / NIST CSF / SOC 2 / GDPR / PCI-DSS)	The maturity domains map cleanly onto control families; this is largely a new YAML plus a view. Highest business value of the unbuilt items.
Public-repo secret scanning (trufflehog)	Would slot in as a new module under `scanners/` producing `Finding` objects, exactly like the existing scanners.
Multi-target comparison view	Requires persistence beyond a single in-memory run; the largest architectural change on the list.
Multi-host Shodan / subnet support	`lookup_host()` is single-IP today. Shodan's search API supports `net:` queries; credit cost scales per IP.
Docker Compose one-command startup	Complicated by the Nmap binary requirement and by Reflex's Node.js frontend build.

3. Technical debt

ITEM	LOCATION	ASSESSMENT
Legacy attack-graph builder	`analysis/graph_builder.py` (182 lines)	Superseded by `analysis/attack_brain.py` and `analysis/attack_graph.py`. Still carries an old "Design System v4" colour palette. A repository-wide search for `graph_builder` returns no importers, so it is dead code and safe to delete.
Shadowed legacy constants module	`config.py` at the repository root (43 lines)	Python resolves the `config/` **package** before the `config.py` **module**, so `from config import WEIGHT_CVSS` always reads `config/__init__.py`. The root file is dead. It was the only place the *rationale* for the scoring weights was written down; those comments now live at the top of `config/__init__.py`, so the file can be deleted without losing anything.
Streamlit-era docstring	`analysis/maturity.py` → `get_all_question_ids()`	Docstring says "*used to build the Streamlit form*". Streamlit is gone; the Reflex form is built by `_build_maturity_form()` in `redflag_ui/state.py`. Cosmetic.
Broad `except Exception` in the scan pipeline	`redflag_ui/state.py` → `run_scan`	Deliberate: it is what makes a twelve-integration pipeline survive one feed being down. The trade-off is that a genuine bug in a scanner is swallowed silently. Consider logging the exception rather than passing.

ITEM	LOCATION	ASSESSMENT
Duplicated `_higher_exploit` / `_is_ip` helpers	`scanners/{openvas_parse,zap_scan,nuclei_scan,vulners_parse}.py`; `scanners/{dns_scan,tls_scan,breach_scan}.py`	Four and three near-identical copies respectively. Candidates for a shared `scanners/_common.py`. Low risk, low urgency.
`reports/pdf_report.generate_cost_section` is remediation-only	`reports/pdf_report.py`	The Day-1 **integration budget** (the `integration` bucket, ladder costs, accuracy readout) is displayed in the UI but is not represented in the cost PDF. The rollup already carries `integration_total`, `accuracy_pct` and `ci_low/ci_p50/ci_high`.
No Risk Scorecard view-model	`redflag_ui/state.py`	A simple High/Medium/Low scorecard summary was considered but never built.
`data/results/` is vestigial	Repository	Default `output_dir` for `run_nmap_scan()` and `export_findings_csv()`, but the UI overrides both to a temp directory. Git-ignored. Harmless.

4. Inline debt markers in code

The source carries **none**. A repository-wide search for `TODO`, `FIXME`, `XXX` and `HACK` across all `.py` files returns two matches, both false positives — literal `CVE-XXXX-XXXX` placeholder strings inside comments in `scanners/shodan_scan.py` and `scanners/openvas_parse.py`.

5. Suggested order of work

Every defect in §1 is closed. What follows is unbuilt work, ordered by value against effort.

1. **Cover `redflag_ui/` with tests.** The engines carry 143 tests; the interface layer carries none, because the Streamlit suite was retired in the migration and not replaced. The view-model flattening in `state.py` is real logic and is currently verified only by running the app.
2. **Extend the cost PDF to cover the integration budget** (§3). The figures are already computed; this is a reporting gap, and the one area where the interface is ahead of the exports.
3. **Compliance gap mapping** (§2). The highest-value unbuilt feature for the M&A audience, and it reuses the existing maturity domains and YAML pattern.
4. **Remove the dead code** — `analysis/graph_builder.py` and the root `config.py` (§3). Both are confirmed unimported.
5. **Log rather than discard scanner exceptions** (§3).
6. **Secret scanning via trufflehog** (§2), which fits the existing scanner contract.

7. **Multi-target comparison** (§2), last, as the only item requiring an architectural change.

Related documents

- ROADMAP.md — direction, as opposed to the concrete tasks here
- LIMITATIONS.md — accuracy caveats that are *by design*, not bugs
- TEST_PLAN.md — what is and is not covered by tests

Knowledge Transfer

Design rationale and non-obvious behaviour that are not apparent from the source alone.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Design principles

Three principles govern the structure of the codebase.

1.1 There is exactly one finding set

Every scanner, feed and uploaded file exists to produce or improve `Finding` objects (`analysis/schema.py`). Nothing renders its own results. A Nuclei hit does not appear in a "Nuclei section" — it merges into the finding that Nmap already created for that `host:port`, raising its evidence strength from `CONFIRMED`-by-banner to a verified vulnerability, its CVSS, and its exploit status. The output is one ranked list, not twelve lists.

This is why the merge functions are named `merge_*_with_nmap` rather than `add_*_findings`.

1.2 Scanners perform network I/O; analysis does not

`scanners/` is the only layer permitted to make a network call. Everything in `analysis/`, `cost/` and `narrative/` is a pure function of its inputs: identical findings produce an identical score, sentence and price, offline, on every run. This property is what makes the reports defensible in a deal room, and what makes the engines testable without network mocking.

`analysis/brain_memory.py` is the sole exception, and only for its optional `ingest_kev()` call.

1.3 The interface is a projection

`redflag_ui/state.py` implements no business logic. It calls the engines, then flattens their rich objects into flat dataclass "view-models" (`FindingRow`, `LadderStep`, `GateRow`, `CostLadderRow` …) whose fields are already strings, integers and CSS class names. The Reflex components bind to those fields directly.

This boundary was validated in practice: the interface was migrated from Streamlit to Reflex in June 2026 with **no changes to any engine**.

1.4 The pipeline, end to end

`RedFlagState.run_scan` (`redflag_ui/state.py:958`) is the spine. In order:

1. Nmap → XML → `analyze_nmap_file()` → base findings
2. Vulners NSE block parsed from the same XML → merged in
3. Shodan: a **staged upload beats the live API call**; enrich matched ports, plus standalone CVE/port findings
4. OpenVAS → ZAP → Nuclei, each correlation-merged by `host:port`
5. DNS, TLS and breach scans (live target only) append their own findings
6. Asset-inventory Excel stamps `data_sensitivity` onto matching hosts

7. EPSS attaches exploitation probability and may **promote** exploit status
8. `triage_all()` scores and sorts everything
9. `build_view()` runs the Day-1, attack-brain, attack-graph and narrative engines and flattens the lot into view-models
10. `_learn_and_recall()` reads the brain's prior knowledge, then folds this scan into it

2. Non-obvious behaviour

Each of the following is a constraint discovered in development and not evident from the source.

2.1 Nothing may be written inside the repository at runtime

Nmap XML goes to `%TEMP%/redflag_scans` (`_SCAN_DIR` in `state.py`); the brain goes to `~/RedFlag-Brain`. This is not tidiness — it is load-bearing. Reflex's development server watches the worktree, and a write inside it triggers a hot reload that **resets backend state mid-scan**. The scan completes and the findings vanish. The tell-tale is `Compiling…` appearing in the terminal during a scan.

2.2 The project must not reside in OneDrive

The working checkout was moved from `OneDrive\Desktop\Redflag` to `C:\Users\Adityaa\Redflag` because OneDrive's sync engine fights Reflex's Node/Vite build: `EBUSY` errors, npm prune churn, and a blank page with "cannot resolve react". `.web/` and `node_modules/` must not reside in a synchronised folder.

2.3 Hostnames must be resolved before a Shodan lookup

Shodan's host API takes an IP. `run_scan` calls `socket.gethostbyname(tgt)` first and passes `resolved_ip` onward. If resolution fails, the Shodan step is skipped silently rather than erroring.

2.4 The knowledge base bootstraps from a seed on first run

A fresh clone does **not** start with an empty brain. `BrainMemory._load()` sees no `~/RedFlag-Brain/brain.json` and copies `analysis/brain_seed/brain.json` in. `export_seed()` goes the other way and **strips the `targets` map** — the only field naming a scanned host — so committing a refreshed seed never reveals who was assessed. See BRAIN_KNOWLEDGE_BASE.md.

2.5 First launch compiles a frontend and requires Node.js

Reflex builds a Next.js bundle on first run. This takes approximately one minute and will offer to install Node.js 18+ if it is absent. The delay is expected and occurs only once.

2.6 Enum values, not enum objects

`Finding` sets `model_config = ConfigDict(use_enum_values=True)`, so a `Finding` field holds the **string** `"internet_facing"`, not `ExposureLevel.INTERNET_FACING`. But `triage_all()` assigns enum *objects* to `deal_tier`. The codebase therefore normalises everywhere with

```
def _v(x) -> str:
    return str(getattr(x, "value", x))
```

This helper must be used in preference to `str(x)`, which on an enum member yields `"DealTier.CRITICAL"` and silently fails every comparison and dictionary lookup.

2.7 Uploads take priority over live API calls

If a Shodan JSON is staged, `lookup_host()` is never called. This is deliberate: it saves an API credit and lets a target supply its own export.

2.8 Scanners degrade to an empty result, never to an exception

Missing Nuclei binary → `[]`. No Vulners key → status unchanged. Network down → EPSS enrichment skipped. This is why a scan with no keys and no optional binaries still produces a full report. The cost is that a genuine bug inside a scanner is swallowed by `except Exception: pass` in `run_scan`. A scanner that appears to produce nothing should therefore be invoked directly from a Python shell rather than diagnosed through the interface.

2.9 Reflex-specific constraints

- `rx.upload` cannot take a `Var` for `class_name` ("Cannot iterate over Var"). Keep it static and put conditional classes on a wrapping `rx.el.div`.
- Dynamic widths and gradients must be **precomputed string Vars** (`bar_w="60%"`, `donut_gradient="conic-gradient(...)"`), never f-strings built over a Var in a component.
- Backend-only state vars use a leading underscore (`_findings`, `_assessment`, `_gap_report`, `_staged`) so Reflex does not serialise the raw engine objects to the browser.
- A compile error on hot reload **terminates the development server**. Component edits should be validated by building the page component and calling `.render()` in the virtual environment, which surfaces binding errors without involving the running server.

---

3. Areas requiring particular care

AREA	CONSIDERATION
Scoring arithmetic (`analysis/triage.py`)	The weights must sum to 1.0 or the score ceases to be a 0–100 scale. The evidence multiplier is applied *after* the weighted sum. Deal-killer overrides are evaluated *before* any scoring and force the score to 100; altering their order alters verdicts. 24 tests constrain this file.
Knowledge-base write path (`analysis/brain_memory.py`)	Writes to the operator's home directory. `_save()` and `_write_vault()` are wrapped in bare `except: pass` so that a disk fault cannot interrupt a scan; consequently a failure produces no diagnostic. Where the knowledge base does not update, verify that the directory exists and is writable.
`export_seed()`	Determines what is safe to publish. It strips `targets` and nothing else. Any new field added to `brain.json` that could identify a target must be stripped here as well.

AREA	CONSIDERATION
Silent degradation without keys	A report produced without a Vulners key is indistinguishable from one produced with it, save for weaker exploit statuses. The recipient of a report should be told which enrichments were live.
`config/` YAML edits	The loader caches every file for the lifetime of the process. A YAML change requires an application restart, or `reload_all()` under test, before taking effect.
Correlation-merge functions	These mutate and upgrade existing findings in place. An upgrade that inadvertently *downgraded* CVSS or exploit status would understate a genuine risk; the `_higher_exploit()` helpers exist to prevent this and should be retained.
PDF generation	fpdf2 raises on characters outside the embedded font. PDF text passes through `_safe()` in `reports/pdf_report.py`. A new glyph introduced without verification raises at export time rather than at authoring time.

4. My assessment

What I would keep unchanged. The scoring model and the layering around it. Four weighted factors, an evidence multiplier and a small set of categorical overrides is enough to rank findings the way a deal team actually thinks, and it is simple enough to explain in a meeting without a slide. Keeping every one of those numbers in configuration rather than in code is what makes the model arguable instead of authoritative, which for a risk score is the more useful property. The scanner contract — every scanner returns `Finding` objects or an empty list, never an exception — is the other part I would not change; it is why fourteen integrations coexist without any one of them being able to end a scan.

Where the design is honestly weaker than it looks. The cost engine applies a genuinely careful uncertainty model — triangular distributions widened by confidence, aggregated with partial correlation, producing an 80% interval — to inputs that ultimately come from a catalogue lookup and a headcount. The statistics are sound; the precision they imply exceeds the quality of what feeds them. The confidence interval should be read as a statement about spread in the benchmark data, not as a claim to know the true cost to within a band. Anyone extending this should invest in the input catalogue before refining the mathematics further.

The knowledge base weights by prevalence only. A technique seen in twenty scans two months ago counts the same as one seen yesterday. For a corpus of this size that is the right simplification — recency weighting on sparse data mostly amplifies noise — but it will not stay right. Once the brain holds a few hundred scans, something in the finding will need to decay, or old estate patterns will quietly dominate the recall panel. That is the first change I would make as the corpus grows.

The largest structural gap is the interface layer's test coverage. The engines carry 143 tests; `redflag_ui/` carries none, because the Streamlit suite was retired in the migration and never replaced. The view-model flattening in `state.py` is real logic — it normalises enums, precomputes gradients and widths, and decides what each page can display — and it is currently verified only by running the application. That is the piece of this project I would fix first.

On the migration. Replacing the entire UI without touching a single engine confirmed the boundary was drawn in the right place. The lesson I take from it is narrower than "keep layers separate": it is that the boundary has to be *enforced while it is inconvenient*. Every time it would have been quicker to compute something in a page, doing it in the engine instead is what made the eventual rewrite a two-day job rather than a rebuild.

Concrete follow-on work derived from the code is recorded in KNOWN_ISSUES_AND_BACKLOG.md §5.

Related documents

- ARCHITECTURE.md — the structural view of section 1
- MODULE_REFERENCE.md — signatures and side effects
- TROUBLESHOOTING.md — diagnostics for section 2
- KNOWN_ISSUES_AND_BACKLOG.md — outstanding work

PART II

Technical

Architecture, data model, interfaces and configuration

Architecture

How RedFlag is structured, how data flows through it, and where to plug new things in.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. High-level overview

RedFlag is a single-process Python application with a Reflex (Next.js/React) front end. One user action — **Run scan** — drives a thirteen-step pipeline that collects evidence from up to twelve sources, correlates it into a single `Finding` set, scores it, and then fans that one set out to five independent analysis engines (maturity, Day-1 planning, cost, attack-path reasoning, narrative) before serialising the result to CSV, PDF and a persistent knowledge base.

The organising principle is **one unified risk picture**: no scanner renders its own output. Each source either creates `Finding` objects or improves existing ones.

2. Layered architecture

Four layers, with a strict dependency direction — each layer may import from the layers below it, never above.

```
┌──────────────────────────────────────────────────────────┐
  PRESENTATION            redflag_ui/
  Reflex components, routed pages, RedFlagState
  Contains NO business logic — calls engines, flattens output
  into flat view-model dataclasses
└──────────────────────────────────────────────────────────┘
           calls
┌──────────────────────────────────────────────────────────┐
  ENGINES     analysis/   cost/   narrative/   reports/
  Pure, deterministic, offline. Same input → same output.
└──────────────────────────────────────────────────────────┘
           consumes
┌──────────────────────────────────────────────────────────┐
  COLLECTION              scanners/
  The ONLY layer that performs network I/O.
  Every function degrades to [] rather than raising.
└──────────────────────────────────────────────────────────┘
           configured by
┌──────────────────────────────────────────────────────────┐
  CONFIGURATION           config/
  Constants module + 7 YAML files + a cached loader
└──────────────────────────────────────────────────────────┘
```

State management. All UI state lives in one class, `RedFlagState` in `redflag_ui/state.py`. Raw engine objects (`_findings`, `_assessment`, `_gap_report`, `_staged`, `_rollup`) are held in **backend-only vars** — the leading underscore stops Reflex from serialising them to the browser — so they can be re-fed to the Day-1, cost and export engines on demand without a rescan.

Why the separation is load-bearing. The UI was migrated from Streamlit to Reflex in June 2026 with zero changes to any engine. That migration is the practical proof the boundary holds.

3. Component diagram

DIAGRAM

Presentation — redflag_ui/

redflag_ui.py — rx.App · 9 routed pages · state.py — RedFlagState · view-models · pages/ — overview · findings · attack — maturity · day1 · cost · export · legal · components/ — shell · ui

Collection — scanners/

nmap_scan · shodan_scan · nuclei_scan · openvas_parse · zap_scan · vulners_parse — vulners_enrich · kev_lookup · epss_scan · dns_scan · tls_scan · breach_scan

Analysis — analysis/

schema.py — Finding + enums · parser.py · triage.py — scoring + tiers · maturity.py · standards_compare.py · day1.py · attack_brain.py · attack_graph.py · brain_memory.py · parsers/excel_assets.py

Output engines

cost/ — catalog · estimator · deduplicator — scenario_engine · day1_costing — simulation · rollup · exporters · narrative/ — blocks · engine · report_builder · reports/ — generator (CSV) · pdf_report

Also: config/ — loader + 7 YAMLs + constants · ~/RedFlag-Brain — brain.json + Obsidian vault

FLOW

```
redflag_ui.py -> pages/
pages/ -> state.py
components/ -> state.py
state.py -> Collection
state.py -> triage.py
state.py -> day1.py
state.py -> attack_brain.py
state.py -> attack_graph.py
state.py -> maturity.py
state.py -> standards_compare.py
state.py -> cost/
state.py -> narrative/
state.py -> reports/
state.py -> brain_memory.py
Collection -> schema.py
parser.py -> schema.py
triage.py -> schema.py
shodan_scan -> kev_lookup
shodan_scan -> vulners_parse
nuclei_scan -> kev_lookup
```

```
day1.py -> config/
maturity.py -> config/
standards_compare.py -> config/
triage.py -> config/
cost/ -> config/
narrative/ -> config/
brain_memory.py -> ~/RedFlag-Brain
attack_brain.py -> brain_memory.py
```

4. Data-flow diagram

DIAGRAM

Also: Target host / IP — (optional) · Staged uploads — Shodan JSON · OpenVAS XML — ZAP XML · Nuclei JSONL · Asset XLSX

FLOW

```
Target host / IP -> run_nmap_scan()
run_nmap_scan() -> analyze_nmap_file()
run_nmap_scan() -> parse_vulners_from_nmap_xml()
parse_vulners_from_nmap_xml() -> merge_vulners_with_nmap()
analyze_nmap_file() -> merge_vulners_with_nmap()
Staged uploads -> parse_shodan_json()
Target host / IP -> lookup_host()
parse_shodan_json() -> SE ("upload wins")
lookup_host() -> SE
merge_vulners_with_nmap() -> SE
SE -> merge_openvas_with_nmap()
merge_openvas_with_nmap() -> merge_zap_with_nmap()
merge_zap_with_nmap() -> merge_nuclei_with_nmap()
Staged uploads -> merge_openvas_with_nmap()
Staged uploads -> merge_zap_with_nmap()
Staged uploads -> merge_nuclei_with_nmap()
Target host / IP -> merge_nuclei_with_nmap()
merge_nuclei_with_nmap() -> + run_dns_scan()
Target host / IP -> + run_dns_scan()
+ run_dns_scan() -> apply_sensitivity_to_findings()
Staged uploads -> apply_sensitivity_to_findings()
apply_sensitivity_to_findings() -> enrich_findings_with_epss()
enrich_findings_with_epss() -> triage_all()
triage_all() -> run_assessment()
triage_all() -> build_day1_blueprint()
triage_all() -> run_cost_pipeline()
triage_all() -> analyze_attack_paths()
triage_all() -> build_*_narrative()
run_assessment() -> build_day1_blueprint()
run_assessment() -> run_cost_pipeline()
build_day1_blueprint() -> run_cost_pipeline()
run_cost_pipeline() -> CSV · PDF (full / Day-1 / cost)
build_*_narrative() -> CSV · PDF (full / Day-1 / cost)
build_day1_blueprint() -> CSV · PDF (full / Day-1 / cost)
analyze_attack_paths() -> recall() then learn_from_scan()
```

Ordering constraints that matter

- Vulners NSE data is parsed from the *same* Nmap XML, so it must run after the scan and before the merges.
- A **staged Shodan JSON takes priority over the live API call** — an upload skips the credit spend entirely.
- Correlation merges (OpenVAS → ZAP → Nuclei) all run against the Nmap layer and are order independent between themselves, but must precede triage.
- EPSS runs **last before triage**, because promoting `exploit_status` changes the score.
- `recall()` is always called **before** `learn_from_scan()`, so the insights shown reflect what the brain knew coming in, not what it just learned.

5. Module responsibility table

PATH	RESPONSIBILITY
`rxconfig.py`	Reflex config: `app_name="redflag_ui"`, sitemap plugin, Tailwind preflight deliberately omitted
`redflag_ui/redflag_ui.py`	`rx.App` and the nine routed pages
`redflag_ui/state.py`	`RedFlagState`: the `run_scan` pipeline, all view-models, uploads, brain learn/recall, exports
`redflag_ui/components/shell.py`	Top bar, navigation, scan bar, five upload slots, footer
`redflag_ui/components/ui.py`	`section()`, `empty_state()`, `placeholder()` helpers
`redflag_ui/pages/*.py`	One module per route; pure presentation bound to state fields
`scanners/nmap_scan.py`	Locate the Nmap binary, run full or fast scan, write XML, attach the Vulners NSE script if installed
`scanners/shodan_scan.py`	Live Shodan lookup, uploaded-JSON parsing, finding enrichment, standalone findings, NVD CVSS lookup
`scanners/nuclei_scan.py`	Locate and run the Nuclei binary, parse JSONL, correlation-merge
`scanners/openvas_parse.py`	OpenVAS/GVM XML → findings; correlation-merge
`scanners/zap_scan.py`	OWASP ZAP XML → findings; correlation-merge
`scanners/vulners_parse.py`	Parse the Vulners NSE block out of Nmap XML; merge
`scanners/vulners_enrich.py`	Vulners API exploit confirmation (upgrade only, never downgrade)
`scanners/kev_lookup.py`	CISA KEV catalogue fetch and lookup, cached per process
`scanners/epss_scan.py`	FIRST.org EPSS probability; promote `exploit_status` above threshold

PATH	RESPONSIBILITY
`scanners/dns_scan.py`	SPF, DMARC, DKIM and DNSSEC checks → findings
`scanners/tls_scan.py`	Certificate expiry, TLS version, crt.sh CT-log subdomain discovery
`scanners/breach_scan.py`	LeakIX domain and host exposure lookup
`analysis/schema.py`	The `Finding` Pydantic model and all six enums — the single source of truth
`analysis/parser.py`	Nmap XML → `Finding` objects, with service description/remediation tables
`analysis/triage.py`	Weighted risk scoring, evidence multiplier, deal-tier classification, override rules
`analysis/maturity.py`	Questionnaire → per-domain scores → `MaturityAssessment`
`analysis/standards_compare.py`	`MaturityAssessment` vs. corporate standard → `GapReport`
`analysis/day1.py`	Connectivity ladder, tier gates, review pillars, P0–P3 roadmap
`analysis/attack_brain.py`	MITRE ATT&CK technique mapping, kill-chain narration, radial mind-map SVG
`analysis/attack_graph.py`	networkx chokepoints, blast radius, crown-jewel shortest paths
`analysis/brain_memory.py`	Persistent knowledge base; recall, learn, KEV ingest, Obsidian vault writer, seed export
`analysis/parsers/excel_assets.py`	Asset-inventory Excel → `{host: DataSensitivity}`; stamp onto findings
`analysis/graph_builder.py`	**Legacy.** Superseded by `attack_brain` + `attack_graph`; not imported anywhere
`cost/schema.py`	Cost Pydantic models and five enums
`cost/catalog.py`	Map a finding or gap to a catalogue entry → `CostLineItem`
`cost/estimator.py`	Build raw line items from findings and maturity gaps
`cost/deduplicator.py`	Merge identical remediations; conservative cost selection
`cost/scenario_engine.py`	Low / base / high scenario totals with CapEx/OpEx split
`cost/day1_costing.py`	Price the recommended connectivity model and every ladder rung; vendor-quote overrides
`cost/simulation.py`	Variance-based 80% confidence interval and accuracy percentage
`cost/rollup.py`	Orchestrate estimate → dedupe → scenario → rollup; the cost entry point
`cost/exporters.py`	Cost rollup → CSV and XLSX bytes
`narrative/blocks.py`	Condition-matched, variable-substituted template block selection
`narrative/engine.py`	Build the context dicts and the six narrative sections

PATH	RESPONSIBILITY
`narrative/report_builder.py`	Assemble a full narrative report
`reports/generator.py`	Findings → pandas DataFrame → CSV
`reports/pdf_report.py`	Full, Day-1 and cost PDF sections via fpdf2
`config/loader.py`	Cached YAML loading and convenience accessors
`config/__init__.py`	Scoring weights, tier thresholds, Nmap arguments; re-exports loader helpers
`config/*.yaml`	Seven files: maturity questions, corporate standard, pricing benchmarks, remediation catalog, narrative blocks, Day-1 blueprint, Day-1 cost catalog
`assets/redflag.css`	The complete "Executive Editorial / emerald" stylesheet
`tests/`	143 engine tests plus fixtures

6. The key data object

Everything in RedFlag is an operation on `Finding` — created by a scanner, improved by a merge, scored by triage, sequenced by Day-1, priced by the cost engine, narrated by the narrative engine, and exported.

Its full field list, every enum member, and the finding lifecycle are documented in **DATA_MODEL.md**.

The second-most-important object is `CostLineItem` (`cost/schema.py`), which carries a `CostTriple` of low/base/high rather than a single number — a deliberate choice so no cost is ever shown without its uncertainty.

7. Extension points

Adding a new scanner

The contract is: *produce `Finding` objects, or improve existing ones; never raise.*

1. Create `scanners/my_scanner.py`.
2. Expose either `run_my_scan(target) -> list[Finding]` (a source that stands alone, like DNS or TLS) or `parse_my_xml(path) -> list[Finding]` plus `merge_my_with_nmap(nmap_findings, my_findings) -> list[Finding]` (a source that correlates, like OpenVAS or ZAP).
3. Add a member to `ScannerSource` in `analysis/schema.py`.
4. Set `evidence_strength` honestly — `CONFIRMED` only for a verified vulnerability, `EXTERNAL` for third-party intelligence. This directly scales the risk score; see ADR-0006.
5. Wrap all network I/O so a failure returns `[]`.

6. Call it from `RedFlagState.run_scan` at the right pipeline position (before `triage_all`).
7. If it needs an upload slot, add an entry to the `_SLOTS` list in `redflag_ui/components/shell.py` and a handler in `state.py`.
8. Add tests using a fixture in `tests/fixtures/` — never a live network call.

Adding a configuration knob

1. Add the key to the relevant YAML in `config/`.
2. Add an accessor to `config/loader.py` if it needs one (the file-level getters are already cached).
3. Read it in the engine. Never read a YAML directly from `redflag_ui/`.
4. Document it in CONFIGURATION.md.

Adding a cost line item

Add an entry to `config/remediation_catalog.yaml` under `cve_overrides`, `service_entries`, `tier_entries` or `maturity_entries` — no code change required. Day-1 integration items go in `config/day1_cost_catalog.yaml` under the relevant model.

Adding a narrative sentence

Add a block to the relevant section of `config/narrative_blocks.yaml` with a `when` condition, a `priority` and a `text` template. Lower priority is evaluated first; the first match wins.

Adding a page

Create `redflag_ui/pages/my_page.py` returning a component wrapped in `shell(...)`, then register it in the `_PAGES` list in `redflag_ui/redflag_ui.py`.

Related documents

- DATA_MODEL.md — the `Finding` object in full
- MODULE_REFERENCE.md — real signatures for every public function
- CONFIGURATION.md — every knob referenced above
- INTEGRATIONS.md — what each collection-layer module talks to
- BRAIN_KNOWLEDGE_BASE.md — the persistent store in the diagrams
- KNOWLEDGE_TRANSFER.md — the reasoning behind the layering

Data Model

The definitive reference for RedFlag's core data structures, verified field by field against analysis/schema.py and cost/schema.py.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. The Finding model

Defined in `analysis/schema.py`. A Pydantic v2 `BaseModel` with `model_config = ConfigDict(use_enum_values=True)`.

> NOTE
>
> **Consequence of `use_enum_values=True`:** enum-typed fields on a `Finding` instance hold the **string value** (`"internet_facing"`), not the enum member. But `analysis/triage.py` assigns enum *objects* to `deal_tier`. Always normalise with `str(getattr(x, "value", x))` — the `_v()` helper used throughout the codebase — and never with `str(x)`, which yields `"DealTier.CRITICAL"` on an enum member and silently breaks comparisons.

Identity

FIELD	TYPE	DEFAULT	MEANING
`id`	`str`	`uuid4()` hex string	Stable identifier; referenced by `CostLineItem.finding_ids`
`cve_id`	`str | None`	`None`	CVE identifier when the finding maps to one
`title`	`str`	**required**	Human-readable headline, e.g. `"RDP service exposed on port 3389"`

Location

FIELD	TYPE	DEFAULT	MEANING
`host`	`str | None`	`None`	IP address or hostname
`port`	`int | None`	`None`	TCP/UDP port. `None` for host-level findings (Shodan CVE intelligence, DNS records)
`service`	`str | None`	`None`	Service name as reported by the scanner (`"ssh"`, `"http"`, `"dns"`, `"breach"`)

Severity and likelihood

FIELD	TYPE	DEFAULT	MEANING
`cvss_score`	`float`	`0.0`	CVSS base score. **Constrained `0.0 ≤ x ≤ 10.0`** by Pydantic
`epss_score`	`float | None`	`None`	EPSS probability of exploitation within 30 days, `0.0`–`1.0`
`epss_percentile`	`float | None`	`None`	EPSS rank against all CVEs, `0.0`–`1.0`

Narrative

FIELD	TYPE	DEFAULT	MEANING
`description`	`str`	`""`	What the issue is and why it matters commercially
`remediation`	`str`	`""`	Concrete fix guidance; also feeds the Day-1 roadmap items

Risk dimensions — the four scoring inputs

FIELD	TYPE	DEFAULT	MEANING
`exposure`	`ExposureLevel`	`UNKNOWN`	Where the asset sits relative to the internet
`data_sensitivity`	`DataSensitivity`	`UNKNOWN`	Business/regulatory value of the data at risk
`exploit_status`	`ExploitStatus`	`UNKNOWN`	Whether a working exploit exists
(plus `cvss_score` above)			The fourth input

Provenance

FIELD	TYPE	DEFAULT	MEANING
`scanner_source`	`ScannerSource`	`MANUAL`	Which tool produced the finding
`evidence_strength`	`EvidenceStrength`	`UNKNOWN`	How well corroborated it is; multiplies the score
`raw_data`	`dict | None`	`None`	Untyped scanner payload. Also carries pipeline markers such as `epss_promoted`, `shodan_port_match`, `kev_hit`

Triage output — written by analysis/triage.py

FIELD	TYPE	DEFAULT	MEANING
`risk_score`	`float`	`0.0`	Final 0–100 score. Forced to exactly `100.0` by a deal-killer override
`deal_tier`	`DealTier`	`UNSCORED`	Classification derived from `risk_score`, or forced by an override

FIELD	TYPE	DEFAULT	MEANING
`override_reason`	`str | None`	`None`	**Dual-purpose.** Written by triage to explain a forced deal-killer verdict; *also read as an input* — if it contains the phrase `"active compromise"`, it triggers a deal-killer override

Timestamp

FIELD	TYPE	DEFAULT	MEANING
`discovered_at`	`datetime`	`datetime.now(timezone.utc)`	UTC creation time; exported in the CSV

2. Enums

All six live in `analysis/schema.py` and inherit from `str, Enum`, so they compare equal to their string values.

ExposureLevel — where the asset sits

MEMBER	VALUE	MEANING	SCORE
`INTERNET_FACING`	`internet_facing`	Reachable from the public internet	100
`PARTNER`	`partner`	Reachable from a partner or extranet network	60
`INTERNAL`	`internal`	Reachable only from inside the corporate network	30
`UNKNOWN`	`unknown`	Not established	50

Set by `analysis/parser.py` (`PARTNER` for commonly exposed services, else `INTERNAL`), upgraded to `INTERNET_FACING` by `enrich_findings_with_shodan()` when Shodan confirms the port, and set directly to `INTERNET_FACING` by the DNS, TLS, breach and Shodan-standalone paths.

DataSensitivity — what is at risk

MEMBER	VALUE	MEANING	SCORE
`CROWN_JEWEL`	`crown_jewel`	The organisation's most valuable data or systems	100
`REGULATED`	`regulated`	Subject to GDPR, HIPAA, PCI-DSS or equivalent	85
`SENSITIVE`	`sensitive`	Confidential but not regulated	55
`LOW`	`low`	Public or low-value	20
`UNKNOWN`	`unknown`	Not classified	50

Only ever set from the **asset-inventory Excel upload** (`apply_sensitivity_to_findings()`), which upgrades a finding **only if it is currently `UNKNOWN`** — it never downgrades. Without that upload, every finding stays `UNKNOWN` and scores the neutral 50.

ExploitStatus — can it actually be exploited

MEMBER	VALUE	MEANING	SCORE
`ACTIVE_EXPLOITATION`	`active_exploitation`	Confirmed exploited in the wild (CISA KEV)	100
`PUBLIC_EXPLOIT`	`public_exploit`	A public proof of concept exists	65
`NO_EXPLOIT`	`no_exploit`	No known exploit	10
`UNKNOWN`	`unknown`	Not established	30

Note that `UNKNOWN` (30) scores **higher** than `NO_EXPLOIT` (10) — absence of evidence is treated as more dangerous than evidence of absence.

Sources, in order of authority: CISA KEV → `ACTIVE_EXPLOITATION`; LeakIX credential/database exposure → `ACTIVE_EXPLOITATION`; Vulners API confirmation → `PUBLIC_EXPLOIT`; EPSS ≥ 0.50 promotion → `PUBLIC_EXPLOIT`. Upgrades only — `_higher_exploit()` helpers and `get_exploit_status_from_vulners()` never downgrade.

EvidenceStrength — how well corroborated

MEMBER	VALUE	MEANING	MULTIPLIER
`CONFIRMED`	`confirmed`	Directly verified by a scanner that tested it	**1.00**
`CORRELATED`	`correlated`	Two independent sources agree	**0.95**
`UNKNOWN`	`unknown`	Not established	**0.90**
`INFERRED`	`inferred`	Deduced, e.g. a missing DKIM selector across common names	**0.85**
`EXTERNAL`	`external`	Third-party intelligence not independently verified	**0.80**

The multiplier is applied to the weighted base score. Rationale: ADR-0006.

DealTier — the commercial verdict

MEMBER	VALUE	MEANING	RECOMMENDED ACTION
`DEAL_KILLER`	`deal_killer`	Blocks the close	Escalate before signing
`CRITICAL`	`critical`	Score ≥ 75	Remediate within 30 days of close
`MODERATE`	`moderate`	Score ≥ 50	90-day post-close roadmap
`MANAGEABLE`	`manageable`	Score < 50	Standard hygiene backlog
`UNSCORED`	`unscored`	Triage has not run	Skipped by the cost estimator

ScannerSource — provenance

MEMBER	VALUE	PRODUCED BY
`NMAP`	`nmap`	`analysis/parser.py`
`SHODAN`	`shodan`	`scanners/shodan_scan.py`
`OPENVAS`	`openvas`	`scanners/openvas_parse.py`
`ZAP`	`zap`	`scanners/zap_scan.py`
`NUCLEI`	`nuclei`	`scanners/nuclei_scan.py`
`VULNERS`	`vulners`	`scanners/vulners_parse.py`
`DNS`	`dns`	`scanners/dns_scan.py`
`TLS`	`tls`	`scanners/tls_scan.py`
`BREACH`	`breach`	`scanners/breach_scan.py`
`PDF`	`pdf_upload`	*Reserved — no producer today*
`EXCEL`	`excel_upload`	*Reserved — the Excel parser modifies findings rather than creating them*
`EMAIL`	`email_attachment`	*Reserved — no producer today*
`MANUAL`	`manual`	Default when nothing sets it

3. Derived and scoring fields

`risk_score` and `deal_tier` are written onto the `Finding` in place by `analysis/triage.py:triage()`.

```
# analysis/triage.py — the exact computation
cvss_normalized  = (finding.cvss_score / 10.0) * 100

base = (cvss_normalized   * WEIGHT_CVSS         # 0.25
      + exposure_score    * WEIGHT_EXPOSURE     # 0.25
      + sensitivity_score * WEIGHT_SENSITIVITY  # 0.20
      + exploit_score     * WEIGHT_EXPLOIT)     # 0.30

risk_score = round(base * EVIDENCE_MULTIPLIERS[evidence_strength], 2)
```

Override rules run first. `check_override_rules()` is evaluated *before* any scoring, and a match short-circuits: `risk_score = 100.0`, `deal_tier = DEAL_KILLER`, and `override_reason` is overwritten with the explanation. The four conditions are documented in RESULTS_INTERPRETATION.md.

`triage_all()` maps `triage()` over the list and returns it **sorted by `risk_score` descending** — the order the UI and every export rely on.

4. Lifecycle of a Finding

DIAGRAM

FLOW

```
Created -> Enriched
Enriched -> Correlated
Correlated -> Classified
Classified -> Predicted
Predicted -> Scored
Scored -> Consumed
```

1. **Created** — a scanner instantiates it. `risk_score` is `0.0`, `deal_tier` is `UNSCORED`.
2. **Enriched** — Shodan may upgrade `exposure` to `INTERNET_FACING` and `evidence_strength` to `CORRELATED`; KEV may set `ACTIVE_EXPLOITATION`; NVD may raise `cvss_score`.
3. **Correlated** — an OpenVAS, ZAP or Nuclei result matching the same `host:port` upgrades the existing finding in place rather than adding a duplicate. (ADR-0008)
4. **Classified** — the asset-inventory Excel stamps `data_sensitivity`, but only over `UNKNOWN`.
5. **Predicted** — EPSS attaches `epss_score`, `epss_percentile` and, at ≥ 0.50, promotes an `UNKNOWN` or `NO_EXPLOIT` status to `PUBLIC_EXPLOIT`, marking `raw_data["epss_promoted"]`. (ADR-0007)
6. **Scored** — `triage_all()` writes `risk_score` and `deal_tier`.
7. **Consumed** — by the Day-1 roadmap, cost estimator, attack brain and graph, narrative engine, CSV/PDF exporters, and the brain's `learn_from_scan()`.

5. Secondary models

Cost — cost/schema.py

MODEL	PURPOSE
`CostTriple`	Frozen `low / base / high` USD triple. Supports `+`, `.total(scenario)`, `.spread_ratio()`. A cost is **never** a single number
`CostLineItem`	One remediation or integration line. Carries `bucket` (`"remediation"` or `"integration"`), `category`, `capex_opex`, `confidence`, `review_flags`, `finding_ids`, and dedup metadata
`CostScenario`	Totals for one of low/base/high, split CapEx/OpEx
`CostRollup`	The aggregate: totals, bucket split, per-category breakdown, three scenarios, `accuracy_pct`, the P10/P50/P90 interval, and the review gate

Enums: `RemediationCategory` (11 members), `CapexOpex` (3), `CostConfidence` (3), `ScenarioType` (3), `ReviewFlag` (5).

Maturity — analysis/maturity.py and analysis/standardscompare.py

MODEL	PURPOSE
`DomainScore`	Frozen. One domain's weighted 0–5 score plus its thresholds and `gap_severity`
`MaturityAssessment`	All seven domain scores, overall score, deal-blocker flags, completion percentage
`MaturityGapSeverity`	`DEAL_BLOCKER` / `BELOW_MIN` / `ACCEPTABLE` / `AT_TARGET`
`GapItem`	One gap with its catalogue key for cost lookup
`GapReport`	Gaps split into deal-blocker, below-minimum and improvement buckets

Day-1 — analysis/day1.py

MODEL	PURPOSE
`Day1Phase`	`P0_PRE_CONNECT` / `P1_CONTAIN` / `P2_STABILISE` / `P3_INTEGRATE_READY`
`ConnectivityModel`	`ISOLATE` / `BROKER` / `FEDERATE` / `INTEGRATE`
`PillarStatus`	`GREEN` / `AMBER` / `RED` / `UNKNOWN`
`Pillar`	One review pillar with RAG status, evidence and a status-keyed recommendation
`CriterionResult` / `IntegrationGate`	A single gate criterion and the gate that aggregates them
`Day1ActionItem`	One roadmap item — a finding or a maturity gap — assigned to a phase
`Day1Blueprint`	The full result: recommended model, catalogue, pillars, gates, roadmap

Attack analysis — analysis/attackbrain.py and analysis/attackgraph.py

MODEL	PURPOSE
`Technique`	A MITRE ATT&CK technique with tactic, ID, name and a `.ref` URL
`AttackStep` / `AttackPlan`	The narrated kill-chain and its mind-map stages
`MindStage` / `MindChild`	Mind-map branch structure
`Chokepoint` / `CrownPath` / `GraphReport`	Quantitative graph output
`BrainStats` / `BrainInsight`	Knowledge-base read models

Related documents

- RESULTS_INTERPRETATION.md — what the numbers mean to a reader
- ARCHITECTURE.md — where these objects flow
- MODULE_REFERENCE.md — the functions that create and mutate them
- CONFIGURATION.md — the weights and thresholds referenced here
- ADR-0006, ADR-0007, ADR-0008

Module Reference

Public API per package. Every signature below is copied from the source, not paraphrased.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

Legend — [net] makes a network call · [disk] writes to disk · [cfg] reads YAML config (cached) · [key] behaviour changes with an API key · ! mutates its argument in place

scanners/ — the collection layer

The only package permitted to perform network I/O. **Every function here degrades to an empty result rather than raising**, so one dead feed cannot fail a scan.

scanners/nmapscan.py

```
def find_nmap() -> str | None
def vulners_nse_available() -> bool
def run_nmap_scan(target: str, output_dir: str = "data/results", fast_mode: bool = False) -> str
```

- `find_nmap()` — probes two fixed Windows paths: `C:\Program Files (x86)\Nmap\nmap.exe` then `C:\Program Files\Nmap\nmap.exe`. Returns `None` if neither exists.

Note: this is Windows-only path probing; it does not consult `PATH`.

- `vulners_nse_available()` — `True` if `vulners.nse` sits in Nmap's `scripts/` directory.
- `run_nmap_scan()` [net][disk][key] — the one function in the codebase that **does raise**: `FileNotFoundError` if Nmap is not installed. Writes `{output_dir}/nmap_{target}_{YYYYMMDD_HHMMSS}.xml` and returns its path. Uses `NMAP_SCAN_ARGS` or `NMAP_FAST_ARGS`, and appends `--script vulners --script-args vulners.mincvss=5.0` (plus `api_key=` if `VULNERS_API_KEY` is set) when the NSE script is present.

scanners/shodanscan.py

```
def fetch_cvss_from_nvd(cve_id: str) -> float
def fetch_cvss_batch(cve_ids: list[str]) -> dict[str, float]
def parse_shodan_json(data: dict) -> dict
def lookup_host(ip_address)
def enrich_findings_with_shodan(findings, shodan_result)
def create_shodan_findings(shodan_result, host, max_cves=SHODAN_MAX_CVES)
```

- `fetch_cvss_from_nvd()` [net] — `services.nvd.nist.gov`, 8 s timeout. Prefers CVSS v3.1, then v3.0, then v2; **falls back to 6.5** on any failure. Cached in-process in `_nvd_cache`.
- `fetch_cvss_batch()` [net] — parallel fetch capped at **5 workers** to respect NVD's unauthenticated 5-requests-per-30-seconds limit. Skips anything already cached.
- `parse_shodan_json()` — normalises an uploaded Shodan host record into the same dict shape

`lookup_host()` returns, and tags it `source: "external_json"`. Handles `vulns` as either a list or a dict. **Raises `ValueError`** with a readable message if the payload has no `ip_str`, `ip`.

- `lookup_host()` [net][key] — costs **1 Shodan credit per IP**. Returns `{"success": False, "error": …}` when `SHODAN_API_KEY` is unset or the API errors.
- `enrich_findings_with_shodan()` [net] — for findings whose port Shodan also observed: sets `exposure = INTERNET_FACING`, `evidence_strength = CORRELATED`, applies KEV/exploit status, and may raise `cvss_score` from NVD. Adds Shodan org/ASN/geo context to every finding's `raw_data`.
- `create_shodan_findings()` [net] — builds **standalone** findings (not merged): one per CVE up to `SHODAN_MAX_CVES` (10), plus one per risky observed port from a 12-entry table. All are `EXTERNAL` evidence and `INTERNET_FACING`.

scanners/nucleiscan.py

```
def find_nuclei() -> str | None
def nuclei_available() -> bool
def parse_nuclei_jsonl(text_or_path: str) -> list[Finding]
def run_nuclei_scan(target: str, fast_mode: bool = False, timeout: int = 300) -> list[Finding]
def merge_nuclei_with_nmap(nmap_findings: list[Finding],
                           nuclei_findings: list[Finding]) -> list[Finding]
```

- `find_nuclei()` — cross-platform: `shutil.which("nuclei")` first, then `~/go/bin`, `C:\Program Files\Nuclei\`, `/usr/local/bin`, `/opt/homebrew/bin`.
- `parse_nuclei_jsonl()` — accepts either a file path or raw JSONL text. Severity maps to CVSS: `critical` `9.5` · `high` `7.5` · `medium` `5.5` · `low` `3.0` · `info` `0.0`. Cross-references CISA KEV.
- `run_nuclei_scan()` [net] — subprocess with a 300 s default timeout. **Returns `[]` if the binary is absent** rather than raising.
- `merge_nuclei_with_nmap()` — correlates by `(host, port)`; upgrades matches to `CONFIRMED`.

scanners/openvasparse.py

```
def parse_openvas_xml(xml_file: str) -> list[Finding]
def merge_openvas_with_nmap(nmap_findings: list[Finding],
                            openvas_findings: list[Finding]) -> list[Finding]
```

Handles both `<report><results>` and bare `<results>` document shapes. Extracts CVE refs and CVSS from either the `<result>` or its `<nvt>`, and derives exploit status from CVSS plus the GVM threat level.

scanners/zapscan.py

```
def parse_zap_xml(xml_file: str) -> list[Finding]
def merge_zap_with_nmap(nmap_findings: list[Finding],
                        zap_findings: list[Finding]) -> list[Finding]
```

Web-layer findings only. Exposure is derived from the port and whether the site was HTTPS.

scanners/vulnersparse.py

```
def parse_vulners_from_nmap_xml(xml_file: str) -> list[Finding]
def merge_vulners_with_nmap(nmap_findings: list[Finding],
                            vulners_findings: list[Finding]) -> list[Finding]
```

Reads the NSE script output block already embedded in the Nmap XML — **no extra network call**. Filters by `VULNERS_MIN_CVSS` (5.0) and `VULNERS_STANDALONE_MIN_CVSS` (7.0).

scanners/vulnersenrich.py

```
def has_public_exploit(cve_id: str) -> bool
def get_exploit_status_from_vulners(cve_id: str, current_status: ExploitStatus) -> ExploitStatus
```

[net][key] `vulners.com/api/v3/search/lucene/`, 6 s timeout, cached per process. **Upgrade only:** never downgrades `ACTIVE_EXPLOITATION`, and no-ops silently without a key.

scanners/kevlookup.py

```
def fetch_kev_catalog() -> dict[str, dict]
def is_kev(cve_id: str) -> bool
def get_kev_entry(cve_id: str) -> dict | None
```

[net] The free CISA KEV JSON feed, 10 s timeout, cached in-process for the session. On failure the cache becomes `{}` — every lookup then returns `False`, `None`, silently.

scanners/epssscan.py

```
EPSS_PROMOTE_THRESHOLD = 0.50

def fetch_epss(cve_ids) -> dict
def enrich_findings_with_epss(findings: list, scores: dict | None = None) -> list
```

[net] `api.first.org/data/v1/epss`, batched **100 CVEs per request**, 15 s timeout, no key needed. `enrich_findings_with_epss()` attaches `epss_score`, `epss_percentile` and, at ≥ 0.50, promotes `UNKNOWN`/`NO_EXPLOIT` to `PUBLIC_EXPLOIT`, setting `raw_data["epss_promoted"] = True`. Passing `scores` explicitly bypasses the network — this is how the tests run offline.

scanners/dnsscan.py

```
def run_dns_scan(domain: str) -> list[Finding]
```

[net] DNS over `dnspython`, 8 s lifetime per query. **Returns `[]` immediately for a bare IP.** Produces findings for: missing SPF (5.3), multiple SPF records (4.0), `+all` (6.0), `~all` (3.5), missing DMARC (6.5), `p=none` (4.5), no DKIM across 12 common selectors (4.0, `INFERRED`), and no DNSSEC (3.0).

scanners/tlsscan.py

```
def run_tls_scan(domain: str, https_ports: list[int] | None = None) -> tuple[list[Finding], dict]
```

[net] Direct TLS connections (8 s timeout, certificate verification disabled so an expired or self-signed certificate can still be *inspected*), plus `crt.sh` for certificate-transparency subdomain discovery. Returns `([], {"skipped": ...})` for a bare IP. Port 443 is always included.

scanners/breachscan.py

```
def run_breach_scan(domain: str, ips: list[str]) -> tuple[list[Finding], dict]
```

[net] LeakIX `/domain/{domain}` and `/host/{ip}`, 12 s timeout, no key required. **Capped at the first 2 IPs** to avoid rate limiting. Event classification: credential/secret leak → CVSS 9.5 + `ACTIVE_EXPLOITATION`; exposed database → 8.5 + `ACTIVE_EXPLOITATION`; exposed `.git` → 7.5; exposed backup → 7.0; anything else → 6.0.

analysis/ — the reasoning layer

Pure and deterministic. No network calls anywhere except `brain_memory.ingest_kev()`.

analysis/schema.py

Declares `Finding` plus `ExposureLevel`, `DataSensitivity`, `ExploitStatus`, `DealTier`, `ScannerSource`, `EvidenceStrength`. Full reference: DATA_MODEL.md.

analysis/parser.py

```
def parse_nmap_xml(xml_file: str) -> list[Finding]
def analyze_nmap_file(xml_file: str) -> list[Finding]
```

Skips hosts that are not `up` and ports that are not `open`. Assigns a **flat CVSS of 3.5** — Nmap confirms a service is open, not that it is vulnerable — and sets `evidence_strength = CONFIRMED` (the port genuinely is open). Exposure is `PARTNER` for the eight services in `SERVICE_EXPOSED`, else `INTERNAL`. Twelve services have curated description and remediation text.

analysis/triage.py

```
EXPOSURE_SCORES: dict        # INTERNET_FACING 100 · PARTNER 60 · INTERNAL 30 · UNKNOWN 50
SENSITIVITY_SCORES: dict     # CROWN_JEWEL 100 · REGULATED 85 · SENSITIVE 55 · LOW 20 · UNKNOWN 50
EXPLOIT_SCORES: dict         # ACTIVE_EXPLOITATION 100 · PUBLIC_EXPLOIT 65 · UNKNOWN 30 · NO_EXPLO
    IT 10
EVIDENCE_MULTIPLIERS: dict   # CONFIRMED 1.00 · CORRELATED 0.95 · UNKNOWN 0.90 · INFERRED 0.85 · E
    XTERNAL 0.80
```

```
def check_override_rules(finding: Finding) -> tuple[bool, str]
def calculate_base_score(finding: Finding) -> float
def apply_evidence_adjustment(score: float, finding: Finding) -> float
def calculate_score(finding: Finding) -> float
def score_to_tier(score: float) -> DealTier
def triage(finding: Finding) -> Finding
def triage_all(findings: list[Finding]) -> list[Finding]
```

`triage()` mutates and returns the same object. `triage_all()` returns a **new list sorted by `risk_score` descending**. Override rules run before scoring and short-circuit to 100.

analysis/maturity.py

```
def score_domain(domain_key: str, answers: dict[str, int]) -> Optional[DomainScore]
def run_assessment(answers: dict[str, int], target: str = "") -> MaturityAssessment
def get_all_question_ids() -> list[str]
def get_domain_questions(domain_key: str) -> list[dict]
def get_all_domains() -> list[tuple[str, str]]
```

[cfg] Answers are clamped to 0–5. **Only answered questions contribute to the denominator**, so a partially completed questionnaire is not penalised as if the unanswered questions scored zero. The overall score weights all seven domains equally.

analysis/standardscompare.py

```
def compare_to_standard(assessment: MaturityAssessment) -> GapReport
def format_gap_summary(gap_report: GapReport) -> str
```

[cfg] Domains at `AT_TARGET` are excluded entirely. Each gap carries `catalog_key = f"{domain}_gap"`, which is what the cost engine looks up.

analysis/day1.py

```
def is_remote_access(finding, cfg: Optional[dict] = None) -> bool
def assign_phase(finding, cfg: Optional[dict] = None) -> str
def build_roadmap(findings: list, gap_report=None, cfg: Optional[dict] = None) -> dict
def build_pillars(findings: list, assessment=None, cfg: Optional[dict] = None) -> list[Pillar]
def build_gates(findings: list, assessment=None, cfg: Optional[dict] = None) -> list[IntegrationGa
    te]
def recommend_connectivity(findings: list, assessment=None,
                           gates: Optional[list[IntegrationGate]] = None,
                           cfg: Optional[dict] = None) -> tuple[str, str, str]
def build_day1_blueprint(findings: list, assessment=None, gap_report=None,
                         target: str = "") -> Day1Blueprint
```

[cfg] All behaviour is driven by `config/day1_blueprint.yaml`.

- `assign_phase()` walks the ordered `phase_rules` and returns the **first** match.
- `recommend_connectivity()` walks `["integrate", "federate", "broker", "isolate"]` and picks the

first tier whose gate passes — the most integrated posture the evidence justifies. `isolate` is the floor and always passes. The returned rationale names the blockers for the next tier up.

- `build_day1_blueprint()` degrades gracefully: with findings but no questionnaire it still produces posture, roadmap and the remote-access pillar, marking the other two "not assessed".

analysis/attackbrain.py

```
def analyze_attack_paths(findings: list, target: str = "") -> AttackPlan
def build_mindmap_svg(plan: AttackPlan, target: str = "") -> str
```

Offline MITRE ATT&CK expert system — no LLM, no network. Maps each finding to techniques by service and port (web → `T1190`, SSH → `T1110`, `T1133`, RDP → `T1133`, `T1110`, SMB → `T1210`, FTP/Telnet → `T1078`, databases → `T1190`, fallback → `T1595`). Builds four mind-map stages (Entry points, Exploitation, Lateral movement, Impact), each capped at four children. `build_mindmap_svg()` returns a raw SVG string for `rx.html`, sized 1000×820 with radii 205/322.

analysis/attackgraph.py

```
def analyze_graph(findings: list, target: str = "") -> GraphReport
```

Builds a `networkx.DiGraph`: `INTERNET → host → host:port service`, plus pivot edges from every internet-facing host to every internal host. Computes blast radius via `descendants()`, chokepoints by node-removal impact tie-broken on betweenness centrality (top 6), and shortest crown-jewel paths (top 4, crown jewels before regulated). **Degrades to an empty `GraphReport` if networkx is not importable.**

analysis/brainmemory.py

```
class BrainMemory:
    VERSION = 1
    def __init__(self, root: str | None = None)
    def recall(self, findings: list, plan) -> tuple[str, list[BrainInsight]]
    def learn_from_scan(self, findings: list, plan, target: str = "") -> None
    def stats(self) -> BrainStats
    def top_techniques(self, limit: int = 6) -> list[BrainInsight]
    def ingest_kev(self) -> tuple[bool, str, int]
    def export_seed(self, dest: str | None = None) -> str
```

[disk] Root resolution order: constructor argument → `$REDFLAG_BRAIN_DIR` → `~/RedFlag-Brain`. `recall()` must be called **before** `learn_from_scan()`. `ingest_kev()` [net] is the only network call in `analysis/`. `export_seed()` strips the `targets` map before writing. `_save()` and `_write_vault()` swallow all exceptions so the brain can never break a scan. Full detail: BRAIN_KNOWLEDGE_BASE.md.

analysis/parsers/excelassets.py

```
def parse_asset_excel(excel_file: str) -> dict[str, DataSensitivity]
def apply_sensitivity_to_findings(findings: list[Finding],
```

```
                                    asset_map: dict[str, DataSensitivity]) -> list[Finding]
```

Column detection is case-insensitive: host column from `ip`, `host`, `ip_address`, `hostname` / `address`; sensitivity column from any name containing `sensitiv` or `classif`, or exactly `tier`. **Raises `ValueError`** if either column is missing. `apply_sensitivity_to_findings()` **only upgrades findings that are currently `UNKNOWN`** — it never downgrades a classification.

analysis/graphbuilder.py

Legacy. Superseded by `attack_brain.py` and `attack_graph.py`; not imported anywhere in the codebase. See KNOWN_ISSUES_AND_BACKLOG.md §3.

---

cost/ — the costing engine

cost/catalog.py

```
def lookup_finding(finding) -> CostLineItem
def lookup_maturity_gap(gap) -> CostLineItem
```

[cfg] Resolution order for a finding: **CVE override → service entry → deal-tier fallback → `default`**. Anything resolving to `default` is flagged `ZERO_ESTIMATE` for human review.

cost/estimator.py

```
def estimate_from_findings(findings: list) -> list[CostLineItem]
def estimate_from_gaps(gap_report) -> list[CostLineItem]
```

Skips `UNSCORED` findings. Always adds the `DEAL_KILLER_ITEM` review flag to deal-killer items.

cost/deduplicator.py

```
def deduplicate(items: list[CostLineItem]) -> list[CostLineItem]
```

Buckets on `catalog_key::category`. Merged cost is deliberately **conservative**: `min(low)`, `max(base)`, `max(high)`. Items keyed `default` or already flagged `ZERO_ESTIMATE` are never merged — each needs individual scoping. Merged items gain the `DUPLICATE` flag, plus `HIGH_VARIANCE` if the spread exceeds 3×.

cost/scenarioengine.py

```
def build_scenarios(items: list[CostLineItem]) -> list[CostScenario]
```

Returns exactly three scenarios. `MIXED` CapEx/OpEx items split 50/50.

cost/day1costing.py

```
def estimate_from_day1(blueprint, headcount=None, catalog: dict | None = None,
                       overrides: dict | None = None) -> list[CostLineItem]
def cost_all_models(headcount=None, catalog: dict | None = None,
                    overrides: dict | None = None) -> dict[str, CostTriple]
def compute_accuracy(items: list, headcount_assumed: bool = False,
                     catalog: dict | None = None) -> tuple[float, float]
```

[cfg] Produces `bucket="integration"` items, kept separate from remediation and **never deduplicated against it**. Six scale modes: `fixed`, `per_user`, `per_user_year`, `per_user_month`, `per_priv_user_year`, `tsa_runrate`. `overrides` maps an item key to a firm vendor quote, which collapses the triple to a single figure and pins confidence to `HIGH`. `cost_all_models()` prices every rung so the UI can show the cost of integrating faster.

cost/simulation.py

```
def estimate_uncertainty(items, headcount_assumed: bool = False) -> Uncertainty
```

Closed-form, deterministic — **no Monte Carlo sampling**. Each item is a triangular distribution widened by its confidence `high 1.0 · medium 1.25 · low 1.6`). Items aggregate with **partial correlation (0.35)**, so independent errors partly cancel without the band becoming unrealistically tight. Yields a P10/P50/P90 80% interval (z = 1.2816), widened by 1.15× when the headcount was assumed, and clamped to a ±8–55% band.

cost/rollup.py

```
def build_rollup(items: list[CostLineItem], target: str = "",
                 headcount_assumed: bool = False) -> CostRollup
def run_cost_pipeline(findings: list, gap_report=None, target: str = "",
                      include_maturity_gaps: bool = True, blueprint=None,
                      include_integration: bool = True, headcount=None,
                      overrides: dict | None = None) -> CostRollup
```

`run_cost_pipeline()` is **the entry point** to the cost engine: estimate → deduplicate → (optionally) add the Day-1 integration budget → scenarios → rollup.

cost/exporters.py

```
def export_rollup_csv(rollup, output_path: Optional[str] = None) -> bytes
def export_rollup_xlsx(rollup, output_path: Optional[str] = None) -> bytes
```

[disk] when `output_path` is given; both always return the bytes.

narrative/ — deterministic prose

narrative/blocks.py

```
def select_block(section: str, context: dict) -> Optional[str]
def select_all_blocks(section: str, context: dict) -> list[str]
def list_sections() -> list[str]
```

[cfg] Blocks are sorted by `priority` ascending; **the first whose `when` conditions all match wins**. `{variable}` placeholders are substituted from the context; unknown placeholders are left untouched. Floats ≥ 1000 format with thousands separators, otherwise to one decimal place.

narrative/engine.py

```
def build_executive_summary(findings: list, target: str = "",
                            assessment=None, rollup=None) -> str
def build_maturity_narrative(assessment) -> str
def build_cost_narrative(rollup) -> str
def build_day1_narrative(blueprint) -> str
def build_finding_narrative(finding) -> str
def build_remediation_priority(finding) -> str
def build_full_context(findings: list, target: str = "",
                       assessment=None, rollup=None) -> dict
```

Each builder assembles a context dict from the engine objects, then calls `select_block()`. Same input, same sentence, every time — no randomness, no model.

narrative/reportbuilder.py

```
def build_report(findings: list, target: str = "", assessment=None, rollup=None) -> dict
```

reports/ — serialisation

reports/generator.py

```
def findings_to_dataframe(findings) -> pandas.DataFrame
def export_findings_csv(findings, output_dir="data/results") -> str
```

[disk] The DataFrame has 17 columns, with every enum rendered as `Title Case` text.

reports/pdfreport.py

```
def generate_pdf_report(findings: list, target: str, resolved_ip: str,
                        scan_date: str | None = None,
                        output_dir: str = "data/results") -> str
def generate_day1_section(pdf_path: str, blueprint, narrative_text: str = "") -> str
def generate_cost_section(pdf_path: str, rollup, narrative_text: str = "") -> str
```

[disk] fpdf2. The two `generate_*_section` functions **append to an existing PDF path**. All text passes through `_safe()`, which strips characters outside the embedded font — fpdf2 raises on an unencodable glyph, so new symbols must be checked before use.

> NOTE
>
> **Gap:** `generate_cost_section()` covers the remediation bucket only. The Day-1 integration budget, the ladder costs and the accuracy readout are shown in the UI but do not appear in the cost PDF. Tracked in KNOWN_ISSUES_AND_BACKLOG.md §3.

---

config/ — configuration

config/init.py

```
WEIGHT_CVSS = 0.25; WEIGHT_EXPOSURE = 0.25
WEIGHT_SENSITIVITY = 0.20; WEIGHT_EXPLOIT = 0.30
TIER_THRESHOLD_CRITICAL = 75; TIER_THRESHOLD_MODERATE = 50
SHODAN_MAX_CVES = 10
VULNERS_MIN_CVSS = 5.0; VULNERS_STANDALONE_MIN_CVSS = 7.0
NMAP_SCAN_ARGS = "-sV --open -T4 --max-retries 2"
NMAP_FAST_ARGS = "-sV --open -T4 --top-ports 200 --version-intensity 3 --max-retries 1"
```

Also re-exports the loader helpers so `from config import get_pricing_benchmarks` works.

config/loader.py

```
def reload_all() -> None
def get_maturity_questions() -> dict
def get_corporate_standard() -> dict
def get_pricing_benchmarks() -> dict
def get_remediation_catalog() -> dict
def get_narrative_blocks() -> dict
def get_day1_blueprint() -> dict
def get_day1_cost_catalog() -> dict
def get_labour_rate(role: str, scenario: str = "base") -> float
def get_effort_hours(task_key: str, scenario: str = "base") -> float
def get_tool_cost(tool_key: str, scenario: str = "base") -> float
def get_domain_standard(domain: str) -> dict
def get_overall_deal_blocker_threshold() -> float
```

[cfg] Every file is loaded once and cached for the process lifetime. **A YAML edit needs an app restart** — or `reload_all()`, which exists for the tests.

redflagui/ — presentation

redflagui/state.py

```
def build_view(findings: list, target: str,
               assessment=None, gap_report=None) -> dict      # pure; testable

class RedFlagState(rx.State):
    def run_scan(self)                                        # the pipeline (generator; yields to
    update the UI)
    async def upload_shodan|openvas|zap|nuclei|asset(self, files)
    def clear_shodan|openvas|zap|nuclei|asset(self)
    def submit_maturity(self, form_data: dict)
    def reset_maturity(self)
    def refresh_threat_intel(self)                            # → BrainMemory.ingest_kev()
    def set_cost_scenario|set_cost_scope|toggle_cost_maturity(self, ...)
    def set_cost_headcount(self, value)
    def apply_quotes(self, form_data: dict) / clear_quotes(self)
    def download_csv|download_pdf|download_day1_pdf|download_cost_pdf(self)
```

`build_view()` is a **module-level pure function**, which is what makes the view-model layer testable without Reflex.

Backend-only vars (leading underscore, never serialised to the browser): `_findings`, `_assessment`, `_gap_report`, `_rollup`, `_staged`, `_resolved_ip`.

`_SCAN_DIR = os.path.join(tempfile.gettempdir(), "redflag_scans")` — Nmap output **must** stay outside the worktree; see KNOWLEDGE_TRANSFER.md §2.

redflagui/redflagui.py

Constructs `rx.App` and registers nine routes: `/`, `/findings`, `/attack`, `/maturity`, `/day1`, `/cost`, `/export`, `/privacy`, `/contact`. Every page uses `on_load=RedFlagState.ensure_loaded`.

Related documents

- DATA_MODEL.md — the objects these functions operate on
- ARCHITECTURE.md — how they compose
- INTEGRATIONS.md — the endpoints behind every [net]
- CONFIGURATION.md — the YAML behind every [cfg]

Configuration

Every tunable knob in RedFlag: environment variables, the constants module, and all seven YAML files. Values shown are the real ones in the repository on 2026-07-27.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

> NOTE
>
> **All YAML is cached for the life of the process.** `config/loader.py` loads each file once. After editing any YAML you must **restart the app** for the change to take effect. In tests, call `config.loader.reload_all()`.

1. Environment variables (.env)

`.env` lives in the repository root and is **git-ignored**. Copy `.env.example` to create it.

VARIABLE	PURPOSE	REQUIRED	BEHAVIOUR IF UNSET
`SHODAN_API_KEY`	Live Shodan host lookups (1 credit per IP)	No	`lookup_host()` returns `{"success": False, "error": ...}`; the pipeline continues. Upload a Shodan JSON instead
`VULNERS_API_KEY`	Vulners exploit confirmation; also passed to the Nmap `vulners` NSE script	No	`get_exploit_status_from_vulners()` no-ops; the NSE script still runs unauthenticated if installed
`REDFLAG_BRAIN_DIR`	Overrides the knowledge-base location	No	Defaults to `~/RedFlag-Brain`

`.env.example` (committed, placeholders only):

```
SHODAN_API_KEY=your_shodan_api_key_here
VULNERS_API_KEY=your_vulners_api_key_here
```

> NOTE
>
> `REDFLAG_BRAIN_DIR` is read directly from `os.environ` in `analysis/brain_memory.py`, not from `.env`. Set it as a real environment variable if you need it.

2. config/init.py — scoring constants

The only tunables that are **not** YAML. Changing any of these changes every score RedFlag has ever produced, so treat them as a versioned decision.

```
# ── Scoring weights — MUST sum to 1.0 ──
WEIGHT_EXPLOIT     = 0.30   # highest: active/public exploitation is the primary triage signal
WEIGHT_EXPOSURE    = 0.25   # reachability; mirrors the CVSS Attack Vector dimension
WEIGHT_CVSS        = 0.25   # technical severity baseline
WEIGHT_SENSITIVITY = 0.20   # business / regulatory impact

# ── Score → tier thresholds ──
TIER_THRESHOLD_CRITICAL = 75   # >= 75 → CRITICAL
TIER_THRESHOLD_MODERATE = 50   # >= 50 → MODERATE, else MANAGEABLE

# ── Shodan ──
SHODAN_MAX_CVES = 10   # cap on standalone CVE findings created from a Shodan record

# ── Vulners NSE ──
VULNERS_MIN_CVSS            = 5.0   # ignore NSE CVEs below this
VULNERS_STANDALONE_MIN_CVSS = 7.0   # min CVSS for secondary CVEs on an already-matched port

# ── Nmap ──
NMAP_SCAN_ARGS = "-sV --open -T4 --max-retries 2"
NMAP_FAST_ARGS = "-sV --open -T4 --top-ports 200 --version-intensity 3 --max-retries 1"
```

`-T4` is aggressive timing (the `-T3` default is roughly twice as slow; `-T5` starts missing ports). Fast mode cuts a typical scan from 40–70 s to 15–25 s.

The **lookup tables** that these weights multiply live in `analysis/triage.py`, not in config — `EXPOSURE_SCORES`, `SENSITIVITY_SCORES`, `EXPLOIT_SCORES`, `EVIDENCE_MULTIPLIERS`. They are documented in DATA_MODEL.md §2.

> NOTE
>
> A root-level `config.py` also exists with the same values plus a longer rationale comment. Python resolves the `config/` **package** first, so that file is dead code. See KNOWN_ISSUES_AND_BACKLOG.md §3.

3. config/maturityquestions.yaml

Defines the questionnaire. **One top-level key: `domains`.** 7 domains, **23 questions**.

DOMAIN KEY	LABEL	QUESTIONS
`identity_access`	Identity & Access Management	4
`network_security`	Network Security	4
`endpoint_security`	Endpoint Security	3
`application_security`	Application Security	3

DOMAIN KEY	LABEL	QUESTIONS
`data_protection`	Data Protection	3
`incident_response`	Incident Response	3
`third_party_risk`	Third-Party & Supply Chain Risk	3

Each question has `id`, `text`, `weight` and six `options` — **the option index is the maturity level**, 0 to 5.

```
domains:
  identity_access:
    label: "Identity & Access Management"
    description: "Controls over who can access what, with what privileges."
    questions:
      - id: iam_mfa
        text: "Is multi-factor authentication (MFA) enforced for all privileged and remote access?
    "
        weight: 2.0          # MFA counts double within the domain
        options:
          - "No MFA in place"                                  # level 0
          - "MFA available but optional"                       # level 1
          - "MFA required for admin accounts only"             # level 2
          - "MFA required for all remote access"               # level 3
          - "MFA enforced everywhere with phishing-resistant methods (FIDO2/hardware)"
          - "Continuous authentication with risk-based step-up enforced"   # level 5
```

Effect of editing: the domain score is the weighted mean of *answered* questions only, so raising a `weight` increases that question's pull on the domain score. Adding a question changes the denominator and therefore every historical comparison. Question `id`s are used as form field keys — renaming one silently discards any saved answer.

4. config/corporatestandard.yaml

The acquirer's bar. Three thresholds per domain on the same 0–5 scale, plus one global value.

DOMAIN	ACCEPTABLEMIN	RECOMMENDED	DEALBLOCKER
`identity_access`	2	4	1
`network_security`	2	4	1
`endpoint_security`	2	4	1
`application_security`	2	3	1
`data_protection`	2	4	1
`incident_response`	2	3	**0**
`third_party_risk`	2	3	1

```
overall_deal_blocker_threshold: 1.5   # mean across all domains
```

Severity is assigned in `analysis/maturity.py:score_domain()`:

- `score <= deal_blocker` (and at least one answer) → **`DEAL_BLOCKER`**
- `score < acceptable_min` → **`BELOW_MIN`**
- `score < recommended` → **`ACCEPTABLE`**
- otherwise → **`AT_TARGET`** (excluded from the gap report entirely)

Note `incident_response` has `deal_blocker: 0` — a missing IR plan is a gap but never on its own a reason to walk away. Each domain also carries a `rationale` string, which is surfaced verbatim in the Day-1 roadmap and the narrative.

How to retune a deal-blocker threshold: edit the domain's `deal_blocker` value, restart the app, and re-run the assessment. Nothing else changes — the value flows to the maturity engine, the gap report, the Day-1 tier gates and the cost engine automatically.

---

5. config/pricingbenchmarks.yaml

Three sections, all in USD, each entry a `low`, `base`, `high` triple.

SECTION	ENTRIES	UNIT
`labour_rates`	8 roles	USD per hour
`tool_costs`	12 tools	per year, per endpoint, per user/month, or per engagement (each entry declares its `unit`)
`effort_hours`	12 tasks	hours

Representative values:

```
labour_rates:
  security_engineer: {low: 85,  base: 150, high: 250}
  security_architect: {low: 120, base: 200, high: 350}
  incident_responder: {low: 150, base: 250, high: 400}

tool_costs:
  edr_per_endpoint: {low: 30,    base: 60,    high: 120,    unit: "per endpoint per year"}
  siem_annual:      {low: 15000, base: 50000, high: 200000, unit: "per year"}
  mfa_per_user:     {low: 3,     base: 6,     high: 15,     unit: "per user per month"}

effort_hours:
  patch_single_system:      {low: 1,  base: 3,   high: 8}
  network_segmentation:     {low: 40, base: 120, high: 300}
  mfa_rollout_100_users:    {low: 8,  base: 20,  high: 40}
```

Accessed through `get_labour_rate(role, scenario)`, `get_tool_cost(tool_key, scenario)` and `get_effort_hours(task_key, scenario)`. All three fall back to `base`, then to a hard default (150 / 0 / 8) if the key is missing — so a typo produces a plausible-looking wrong number rather than an error. Check your key names.

How to retune for a different market: scale `labour_rates` and re-run. Everything downstream — line items, scenarios, the CapEx/OpEx split, the confidence interval — recomputes.

6. config/remediationcatalog.yaml

Maps a finding or maturity gap to a costed line item. Five top-level keys, resolved in this order by `cost/catalog.py:lookup_finding()`:

KEY	ENTRIES	MATCHED ON
`cve_overrides`	5	Exact `finding.cve_id`
`service_entries`	13	`finding.service`
`tier_entries`	4	`finding.deal_tier`
`maturity_entries`	7	`GapItem.catalog_key` `{domain}_gap`
`default`	1	Fallback — **flags the item `ZERO_ESTIMATE` for human review**

Shipped `cve_overrides`: `CVE-2017-0144` (EternalBlue), `CVE-2021-44228` (Log4Shell), `CVE-2023-44487` (HTTP/2 Rapid Reset), `CVE-2021-34527` (PrintNightmare), `CVE-2022-30190` (Follina).

Shipped `service_entries`: `ftp`, `telnet`, `rdp`, `smb`, `vnc`, `redis`, `elasticsearch`, `mongodb`, `docker`, `http`, `https`, `ssh`, `smtp`.

```
cve_overrides:
  CVE-2017-0144:
    title: "Patch MS17-010 (EternalBlue / WannaCry)"
    description: "Critical SMB RCE used by WannaCry and NotPetya ransomware campaigns."
    category: patching              # RemediationCategory enum value
    capex_opex: opex                # capex | opex | mixed
    labour_role: security_engineer  # key in pricing_benchmarks.labour_rates
    labour_hours_key: patch_single_system   # key in pricing_benchmarks.effort_hours
    confidence: high                # high | medium | low
    notes: "MS17-010 patch is available for all affected Windows versions."
```

How to add a pricing line item: add an entry under the appropriate section. `labour_role` and `labour_hours_key` must name existing keys in `pricing_benchmarks.yaml`; `category` must be a valid `RemediationCategory` value; `capex_opex` and `confidence` must be valid enum values. No code change is needed. Restart the app.

Note that dedup buckets on `catalog_key::category`, so ten findings resolving to the same `service_entries` key collapse into one line item — with `min(low)`, `max(base)`, `max(high)`.

7. config/narrativeblocks.yaml

Every sentence the tool writes. Six sections; blocks within a section are sorted by `priority` ascending and **the first whose `when` conditions all match wins**.

SECTION	BLOCKS
`executive_summary`	4
`maturity_summary`	3
`cost_summary`	4
`day1_summary`	4
`finding_detail`	6
`remediation_priority`	4

```
executive_summary:
  - id: ex_deal_killer_present
    priority: 1
    when: {has_deal_killers: true}
    text: >
      This assessment identified {deal_killer_count} deal-killer finding(s) that
      present unacceptable risk to the proposed acquisition of {target}.
```

`{variable}` placeholders are substituted from a context dict built by `narrative/engine.py`. Unknown placeholders are **left in place unchanged**, which is the visible symptom of a typo. Floats ≥ 1000 render with thousands separators; smaller floats render to one decimal place.

How to edit report wording: change the `text` of the relevant block. To add a new case, insert a block with a **lower** `priority` than the general one it should pre-empt, and a `when` condition specific enough not to fire otherwise. Keep the most general block at the highest priority number so it acts as the catch-all.

8. config/day1blueprint.yaml

Drives the entire Day-1 tab. Seven top-level keys.

KEY	WHAT IT CONTROLS
`remote_access_services`	What counts as a remote-access pathway — 16 service names and 15 ports (3389, 22, 23, 5900/5901, 21, 445, 139, 5985/5986, 1723, 1194, 500, 4500, 1701)
`phases`	The four timeline phases with `label`, `window` and `description`
`phase_rules`	Ordered finding-attribute → phase mapping; **first match wins**
`maturity_gap_phase`	Gap severity → phase
`pillars`	The three review pillars and their four status-keyed recommendation strings
`connectivity_models`	The four-rung ladder with `summary`, `controls` and cited `sources`
`tier_gates`	Entry criteria per tier

The phase rules, in evaluation order:

#	CONDITION	PHASE
1	`exploit_status: active_exploitation`	**P0** Pre-Connection Blocker
2	`deal_tier: deal_killer`	**P0**
3	`exposure: internet_facing` **and** `remote_access: true`	**P0**
4	`exposure: internet_facing` **and** `exploit_status: public_exploit`	**P0**
5	`exposure: internet_facing`	**P1** Day-1 Containment
6	`exposure: partner` **and** `remote_access: true`	**P1**
7	`deal_tier: critical`	**P2** Day 1–30 Stabilise
8	`exposure: partner`	**P2**
9	*(empty condition — always matches)*	**P3** Day 30–100 Integration-Ready

Maturity gaps map separately: `deal_blocker → P0`, `below_min → P2`, `acceptable → P3`.

Tier gate criteria. Two criterion types:

- `no_finding` — passes when **no** finding matches the `when` clause.
- `maturity_min` — passes when the named domain's score ≥ the named `level` (`acceptable_min` or `recommended`) from `corporate_standard.yaml`. **Fails if the domain was never assessed** — you cannot prove a posture you did not measure.

TIER	CRITERIA
`isolate`	*(none — always available; the floor)*
`broker`	No actively-exploited vulnerabilities
`federate`	The above, plus: no internet-facing remote access; `identity_access` ≥ acceptable_min; `network_security` ≥ acceptable_min
`integrate`	The above, plus: no internet-facing deal-killers; `identity_access` ≥ recommended; `network_security` ≥ recommended

How to retune: to make the tool more conservative about federating, add a criterion to `tier_gates.federate.criteria` or raise the `level` from `acceptable_min` to `recommended`. To change what counts as remote access, edit the `services` and `ports` lists — this affects both phase assignment and the gates.

9. config/day1costcatalog.yaml

Prices the connectivity ladder. Four top-level keys. Every item cites a real 2026 source in a `source` field.

```
assumptions:
  default_headcount: 250        # acquired-company users, if none is supplied
  opex_months: 12               # year-1 window for recurring items
  tsa_months: 12                # default TSA duration for the Isolate run-rate
  privileged_user_ratio: 0.05   # ~5% of users need PAM access
```

```
accuracy_bands: {high: 15, medium: 30, low: 50}   # confidence → ±% band
assumed_headcount_penalty: 8                       # extra ± when headcount is a guess
```

`models` holds one entry per rung, each a list of items:

MODEL	ITEMS
`isolate`	`cleanroom_setup`, `perimeter_ngfw`, `tsa_it_runrate`
`broker`	`perimeter_ngfw`, `vdi_daas`, `ztna`, `pam`, `siem_day1`, `broker_services`
`federate`	`ztna`, `identity_federation`, `edr_standardize`, `pam`, `siem_day1`, `federation_services`
`integrate`	`identity_federation`, `edr_standardize`, `tenant_migration`, `network_rearch`, `siem_unified`, `integration_pmo`

Each item declares `key`, `title`, `category`, `capex_opex`, `scale`, `low`, `base`, `high`, `confidence` and `source`. Six scale modes:

SCALE	MULTIPLIER APPLIED TO LOW/BASE/HIGH
`fixed`	1
`per_user`	headcount
`per_user_year`	headcount
`per_user_month`	headcount × `opex_months`
`per_priv_user_year`	`round(headcount × privileged_user_ratio)`, minimum 1
`tsa_runrate`	headcount × `tsa_months`

How to tighten the accuracy readout: replace a `base` with a real vendor quote and set `confidence: high`. Better still, enter the quote in the UI's vendor-quote field — that collapses the item's triple to a single figure, pins it to `HIGH`, and clears its high-variance flag, without editing YAML.

10. Retuning recipes

GOAL	CHANGE	FILE
Make exploitability matter more than severity	Raise `WEIGHT_EXPLOIT`, lower `WEIGHT_CVSS` (keep the four summing to 1.0)	`config/__init__.py`
Classify more findings as Critical	Lower `TIER_THRESHOLD_CRITICAL` from 75	`config/__init__.py`
Raise the bar for a maturity deal-blocker	Raise a domain's `deal_blocker` value	`corporate_standard.yaml`
Price for a different labour market	Scale `labour_rates`	`pricing_benchmarks.yaml`
Add a cost for a new CVE or service	Add a `cve_overrides` / `service_entries` entry	`remediation_catalog.yaml`

GOAL	CHANGE	FILE
Change report wording	Edit a block's `text`	`narrative_blocks.yaml`
Treat a new port as remote access	Add it to `remote_access_services.ports`	`day1_blueprint.yaml`
Be stricter about federating identity	Add a criterion, or raise `level` to `recommended`	`day1_blueprint.yaml`
Use a real vendor quote in the budget	Enter it in the UI, or set `base + confidence: high`	`day1_cost_catalog.yaml`
Scan faster	Toggle **Fast mode** in the UI, or edit `NMAP_FAST_ARGS`	`config/__init__.py`
Store the brain elsewhere	Set `REDFLAG_BRAIN_DIR`	environment

After any YAML edit, restart the app. The loader cache is per-process.

Related documents

- DATA_MODEL.md — the lookup tables these weights multiply
- RESULTS_INTERPRETATION.md — the effect of these values on output
- MODULE_REFERENCE.md — the loader accessors
- ACCESS_AND_CREDENTIALS.md — key provisioning

Integrations

Every external tool, binary and API RedFlag touches: what it is used for, what leaves your machine, what it costs, and exactly how it behaves when it is unavailable.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Summary table

#	INTEGRATION	TYPE	MODE	KEY NEEDED	COST	DATA SENT
1	**Nmap**	Local binary	Live	No	Free	Packets to the target
2	**Shodan**	REST API	Live **or** upload	Optional	1 credit / IP	Target IP
3	**Nuclei**	Local binary	Live **or** upload	No	Free	Requests to the target
4	**OpenVAS / GVM**	XML upload	Upload only	No	Free	Nothing
5	**OWASP ZAP**	XML upload	Upload only	No	Free	Nothing
6	**Vulners NSE**	Nmap script	Live (in-scan)	Optional	Free	CVE lookups from the NSE script
7	**Vulners API**	REST API	Live	Optional	Free tier	CVE IDs
8	**CISA KEV**	Public JSON feed	Live	No	Free	Nothing (download only)
9	**EPSS (FIRST.org)**	REST API	Live	No	Free	CVE IDs
10	**NVD (NIST)**	REST API	Live	No	Free	CVE IDs
11	**DNS**	DNS resolver	Live	No	Free	DNS queries for the target domain
12	**TLS + crt.sh**	Socket + REST	Live	No	Free	TLS handshake to the target; domain to crt.sh
13	**LeakIX**	REST API	Live	No	Free	Target domain and IPs
14	**Asset inventory**	Excel upload	Upload only	No	Free	Nothing

A consolidated view of what leaves the machine is in SECURITY_AND_PRIVACY.md §2.

2. Nmap

- **Module:** `scanners/nmap_scan.py` · **Library:** `python-nmap 0.7.1`
- **Purpose:** the base layer. Open ports, service names, product/version banners. Every other scanner either enriches these findings or adds to them.
- **Mode:** active scan against the target. This is the one integration that **touches the target directly and at volume** — see AUTHORIZED_USE.md.
- **Arguments:** `-sV --open -T4 --max-retries 2`; fast mode adds `--top-ports 200 --version-intensity 3 --max-retries 1`.
- **Output:** `{output_dir}/nmap_{target}_{YYYYMMDD_HHMMSS}.xml`, written to `%TEMP%/redflag_scans` at runtime.
- **Binary discovery:** two hard-coded Windows paths only — `C:\Program Files (x86)\Nmap\nmap.exe`, then `C:\Program Files\Nmap\nmap.exe`. It does **not** consult `PATH`, so a non-standard Windows install or a macOS/Linux install (`/usr/local/bin/nmap`) will not be found by `find_nmap()`.
- **Degradation: this is the one integration that raises.** `run_nmap_scan()` throws `FileNotFoundError` if the binary is missing, and `run_scan` surfaces it as

"*Scan failed: Could not find nmap.exe*". Everything else in the pipeline degrades silently.

- **Evidence strength produced:** `CONFIRMED` — the port genuinely is open. CVSS is a flat **3.5**, because an open port is not itself a vulnerability.

---

3. Shodan

- **Module:** `scanners/shodan_scan.py` · **Library:** `shodan 1.31.0`
- **Endpoint:** the Shodan host API, via the official client.
- **Purpose:** confirm which ports are visible **from the internet**, and pull organisation, ASN, ISP, geography, hostnames and known CVEs.
- **Key:** `SHODAN_API_KEY`. **Cost: 1 credit per IP queried.**
- **Data sent:** the target IP address, nothing else.
- **Two modes:**
  - *Live* — `lookup_host(ip)`. Requires a resolved IPv4 address, so `run_scan` calls `socket.gethostbyname()` first.
  - *Upload* — `parse_shodan_json(data)` reads a Shodan host record from the **Shodan JSON** upload slot. **The upload takes priority: if one is staged, the live API is never called.** Useful when the target supplies its own export, and it costs nothing.
- **What it changes:** for findings whose port Shodan also observed, `exposure` becomes `INTERNET_FACING` and `evidence_strength` becomes `CORRELATED`. This is the single biggest score mover in the product — exposure is worth 25% of the weighting, and the jump from `INTERNAL` (30) to `INTERNET_FACING` (100) is large.
- **It also creates standalone findings:** up to `SHODAN_MAX_CVES` (10) CVE findings plus one per risky observed port, from a 12-port table (21, 23, 25, 3389, 445, 5900, 6379, 9200, 27017, 5000, 8080, 2375). These carry `EXTERNAL` evidence (×0.80).
- **Degradation:** no key, or an API error, or an unresolvable hostname → the enrichment step is skipped entirely.

Findings keep the `PARTNER`, `INTERNAL` exposure that `analysis/parser.py` assigned, and score correspondingly lower.

4. Nuclei

- **Module:** `scanners/nuclei_scan.py` · **Binary:** ProjectDiscovery Nuclei (optional)
- **Purpose:** template-based DAST. Confirms **actual** vulnerabilities — CVEs, exposed admin panels, default credentials, misconfigurations — rather than just open ports.
- **Mode:** live subprocess (300 s default timeout) **or** an uploaded `nuclei -jsonl` file.
- **Binary discovery:** `shutil.which("nuclei")` first — genuinely cross-platform — then `~/go/bin/nuclei[.exe]`, `C:\Program Files\Nuclei\nuclei.exe`, `/usr/local/bin/nuclei`, `/opt/homebrew/bin/nuclei`.
- **Severity → CVSS:** `critical` `9.5` · `high` `7.5` · `medium` `5.5` · `low` `3.0` · `info` `0.0`.
- **Cross-references CISA KEV** for each CVE it reports.
- **Merge:** `merge_nuclei_with_nmap()` correlates by `(host, port)` and upgrades the matched Nmap finding to `CONFIRMED` with the higher CVSS and exploit status.
- **Degradation:** binary absent → `run_nuclei_scan()` returns `[]` silently. You can still upload JSONL produced on any other machine, so no local install is ever required.

5. OpenVAS / GVM

- **Module:** `scanners/openvas_parse.py` · **Mode:** XML upload only
- **Purpose:** verified CVEs and configuration flaws from an authenticated or credentialed scan — the depth RedFlag cannot reach on its own.
- **Data sent:** none. Parsing is entirely local.
- **Format:** handles both `<report><results>` and bare `<results>` document shapes. Extracts CVE references and CVSS from either the `<result>` or its `<nvt>` element, and derives exploit status from CVSS combined with the GVM threat level.
- **Merge:** correlates by `host:port`, upgrading matched Nmap findings. Unmatched OpenVAS findings are added on their own.
- **Fixture:** `tests/fixtures/mock_openvas.xml` contains EternalBlue, PrintNightmare, Log4Shell, default credentials, Telnet, Redis exposure, TLS misconfiguration and missing security headers.
- **Degradation:** not applicable — nothing runs unless a file is uploaded.

6. OWASP ZAP

- **Module:** `scanners/zap_scan.py` · **Mode:** XML upload only
- **Purpose:** web-application-layer findings — SQL injection, XSS, IDOR, missing CSRF protection, directory

listing, cookie flags, vulnerable libraries, exposed actuator endpoints.

- **Data sent:** none. Local parsing.
- **Exposure derivation:** from the port and whether the site was HTTPS.
- **Merge:** correlates by `host:port` into the Nmap layer.
- **Fixture:** `tests/fixtures/mock_zap.xml`.
- **Degradation:** not applicable — upload only.

---

7. Vulners NSE script

- **Module:** `scanners/vulners_parse.py` (parsing); `scanners/nmap_scan.py` (invocation)
- **Purpose:** CVE-to-service mapping produced **during** the Nmap scan.
- **How it runs:** if `vulners.nse` is present in Nmap's `scripts/` directory, `run_nmap_scan()` appends `--script vulners --script-args vulners.mincvss=5.0`, plus `,api_key=<key>` when `VULNERS_API_KEY` is set.
- **Parsing:** `parse_vulners_from_nmap_xml()` reads the script output already embedded in the XML — **no additional network call from RedFlag**. The NSE script itself contacts Vulners during the scan.
- **Thresholds:** `VULNERS_MIN_CVSS = 5.0`; `VULNERS_STANDALONE_MIN_CVSS = 7.0` for secondary CVEs on an already-matched port.
- **Degradation:** script not installed → the argument is omitted, a `[INFO]` line is printed, and no Vulners findings are produced. Nothing fails.

---

8. Vulners API

- **Module:** `scanners/vulners_enrich.py`
- **Endpoint:** `https://vulners.com/api/v3/search/lucene/` (POST), 6 s timeout
- **Query:** `cvelist:{CVE} AND type:exploit`, size 1 — RedFlag only asks *whether* an exploit exists, not for its content.
- **Data sent:** the CVE ID and your API key.
- **Key:** `VULNERS_API_KEY`. Free tier available.
- **Caching:** per-CVE, in-process, for the session.
- **Upgrade only:** `get_exploit_status_from_vulners()` never downgrades `ACTIVE_EXPLOITATION` and only ever raises `UNKNOWN` → `PUBLIC_EXPLOIT`.
- **Degradation:** no key → returns the current status unchanged, silently. `exploit_status` simply stays `UNKNOWN` (which still scores 30) unless KEV or EPSS supplies it.

---

9. CISA KEV

- **Modules:** `scanners/kev_lookup.py`; `analysis/brain_memory.py:ingest_kev()`

- **Endpoint:** `https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json`
- **Purpose:** the authoritative list of vulnerabilities **confirmed exploited in the wild**. A KEV hit sets `ACTIVE_EXPLOITATION`, which is a deal-killer override — score forced to 100.
- **Data sent:** none. It is a public file download.
- **Key:** none required.
- **Caching:** the whole catalogue is fetched once per process and held in `_kev_cache`.
- **Also used by the brain:** the **Refresh threat intel** button on the Attack path tab calls `BrainMemory.ingest_kev()`, which stores the full CVE list in `brain.json` and back-fills the `kev` flag on every CVE the brain already knows.
- **Degradation:** unreachable → the cache becomes `{}`, and every `is_kev()` returns `False`.

This is the most consequential silent degradation in the product: with no KEV feed, an actively-exploited CVE will not be flagged as a deal killer. EPSS provides partial cover.

---

10. EPSS (FIRST.org)

- **Module:** `scanners/epss_scan.py`
- **Endpoint:** `https://api.first.org/data/v1/epss`, 15 s timeout, **batched 100 CVEs per request**
- **Purpose:** the probability that a CVE will be exploited **in the next 30 days**. This is what makes the risk model forward-looking rather than purely retrospective.
- **Data sent:** CVE IDs only.
- **Key:** none required.
- **Promotion rule:** `EPSS ≥ 0.50` on a finding whose status is `UNKNOWN` or `NO_EXPLOIT` promotes it to `PUBLIC_EXPLOIT` and sets `raw_data["epss_promoted"] = True`. It never downgrades. Rationale: ADR-0007.
- **Position in the pipeline:** runs **last before triage**, because promotion changes the score.
- **Degradation:** unreachable → findings pass through untouched; `epss_score` stays `None` and no promotion occurs. Tests inject a `scores` dict to bypass the network entirely.

---

11. NVD (NIST)

- **Module:** `scanners/shodan_scan.py` → `fetch_cvss_from_nvd()` / `fetch_cvss_batch()`
- **Endpoint:** `https://services.nvd.nist.gov/rest/json/cves/2.0`, 8 s timeout
- **Purpose:** real CVSS base scores for CVEs that Shodan reports without one.
- **Data sent:** CVE IDs only.
- **Key:** none used. **Rate limit: 5 requests / 30 s unauthenticated**, which is why `fetch_cvss_batch()` caps its thread pool at 5 workers and skips anything already cached.
- **Score preference:** CVSS v3.1 → v3.0 → v2.
- **Degradation:** any failure falls back to a **default CVSS of 6.5**. Note this is a *silent substitution of a plausible number*, not an error — if NVD is unreachable, a batch of CVEs will all score 6.5.

12. DNS

- **Module:** `scanners/dns_scan.py` · **Library:** `dnspython 2.8.0`
- **Purpose:** email-security posture, which becomes the acquirer's liability on day one.
- **Queries sent:** `TXT` on the domain (SPF), `TXT` on `_dmarc.{domain}`, `TXT` on `{selector}._domainkey.{domain}` for **12 common selectors** `default`, `google`, `mail`, `selector1`, `selector2`, `dkim`, `k1`, `s1`, `s2`, `email`, `mandrill`, `protonmail`), and `DNSKEY` (DNSSEC). 8 s lifetime per query.
- **Key:** none. Uses the system resolver.
- **Findings produced:** missing SPF (5.3) · multiple SPF records (4.0) · `+all` (6.0) · `~all` (3.5) · missing DMARC (6.5) · `p=none` (4.5) · no DKIM found (4.0, `INFERRED`) · no DNSSEC (3.0).
- **Known limitation:** DKIM selectors are arbitrary names. A domain using a custom selector will produce a false "No DKIM Selector Found" — which is why that finding alone is marked `INFERRED` (×0.85) rather than `CONFIRMED`.
- **Degradation:** returns `[]` immediately for a bare IP address. Any resolver failure returns an empty record list, which reads as "not configured" — a resolver outage can therefore produce false positives.

13. TLS and crt.sh

- **Module:** `scanners/tls_scan.py` · **Library:** `cryptography 48.0.0`
- **Two parts:**

1. **Direct TLS connection** to each HTTPS port Nmap found (443 always included), 8 s timeout. Reads certificate expiry, issuer, subject, SANs, negotiated TLS version and cipher.

Certificate verification is deliberately disabled `CERT_NONE`) so that an expired or self-signed certificate can still be inspected — the point is to *report* the problem.

2. **crt.sh certificate-transparency query** — `https://crt.sh/?q=%.{domain}&output=json`, 12 s timeout. Returns every subdomain that has ever had a certificate issued, revealing forgotten and unmonitored hosts.

- **Data sent:** a TLS handshake to the target; the bare domain name to crt.sh.
- **Key:** none.
- **Weak versions flagged:** TLS 1.0, TLS 1.1, SSLv2, SSLv3 — deprecated and a PCI-DSS violation.
- **Degradation:** returns `([], {"skipped": ...})` for a bare IP. crt.sh is frequently slow or rate-limited; a failure returns an empty subdomain list, and the certificate checks still run.

14. LeakIX

- **Module:** `scanners/breach_scan.py`
- **Endpoints:** `https://leakix.net/domain/{domain}` and `https://leakix.net/host/{ip}`, 12 s timeout

- **Purpose:** answers the single most important acquisition question — *has this company already been compromised?* LeakIX indexes live exposures: unauthenticated MongoDB/Elasticsearch/Redis, leaked configuration and credential dumps, exposed `.git` directories and backups.
- **Data sent:** the target domain and up to **2 IP addresses** (capped to avoid rate limiting).
- **Key:** none required for basic queries.
- **Classification:** credential/secret leak → CVSS 9.5 + `ACTIVE_EXPLOITATION` · exposed database → 8.5 + `ACTIVE_EXPLOITATION` · exposed `.git` → 7.5 + `PUBLIC_EXPLOIT` · exposed backup → 7.0 · generic exposure → 6.0. The first two therefore trigger the deal-killer override.
- **Evidence strength:** `CONFIRMED` — the exposure has been *observed* by a public scanner, not inferred.
- **Degradation:** any error is captured into the returned summary dict and the findings list is simply shorter. Never raises.

15. Asset inventory (Excel)

- **Module:** `analysis/parsers/excel_assets.py` · **Library:** `openpyxl 3.1.5` via pandas
- **Purpose:** the only source of `data_sensitivity`. Without it, every finding stays `UNKNOWN` and scores the neutral 50 on the dimension worth 20% of the weighting — **and two of the three deal-killer override rules can never fire**, because both require `CROWN_JEWEL` or `REGULATED`.
- **Format:** any `.xlsx` with a host column (`ip`, `host`, `ip_address`, `hostname`, `address`) and a sensitivity column (any header containing `sensitiv` or `classif`, or exactly `tier`). Matching is case-insensitive and space-tolerant.
- **Accepted values:** `crown_jewel` / `crown jewel` / `crownjewel`, `regulated`, `sensitive`, `low`, `unknown`. Anything unrecognised becomes `UNKNOWN`.
- **Upgrade only:** a finding is stamped **only if its sensitivity is currently `UNKNOWN`**.
- **Degradation: raises `ValueError`** if a required column is missing — surfaced as an upload error in the UI. Not uploading a file is fine; uploading a malformed one tells you so.

16. Running with no API keys at all

RedFlag is designed to produce a complete assessment with zero keys. This is a deliberate constraint, not a fallback — see ADR-0003.

WHAT YOU LOSE	WHAT STILL WORKS
Live Shodan lookup	Upload a Shodan host JSON instead — it takes priority over the live call anyway
Vulners API exploit confirmation	KEV and EPSS both supply exploit intelligence with no key
(nothing else)	Nmap, Nuclei, OpenVAS, ZAP, CISA KEV, EPSS, NVD, DNS, TLS, crt.sh, LeakIX, Excel

Ten of the fourteen integrations require no key at all. The two optional keys only sharpen enrichment; neither gates any feature.

17. Failure-mode reference

INTEGRATION	IF UNAVAILABLE	VISIBLE SYMPTOM	RISK TO THE ASSESSMENT
Nmap	**Raises**	*"Scan failed: Could not find nmap.exe"*	Total — no base findings
Shodan	Silent skip	Exposure stays `PARTNER`, `INTERNAL`	**High** — scores materially understate risk
Nuclei	Silent `[]`	Fewer `CONFIRMED` findings	Medium
CISA KEV	Silent `{}`	No deal-killer overrides fire	**High** — an exploited CVE goes unflagged
EPSS	Silent skip	No `epss_score`, no promotion	Medium
NVD	Falls back to **6.5**	Plausible but wrong CVSS	Medium — *silent substitution*
Vulners	Silent no-op	`exploit_status` stays `UNKNOWN`	Low — KEV/EPSS cover most of it
DNS	Empty records	Reads as "not configured"	Low — but can cause **false positives**
TLS / crt.sh	Empty list	No subdomain discovery	Low
LeakIX	Error into summary	No breach findings	Medium
Excel	**Raises `ValueError`**	Upload error shown	None — it tells you

> NOTE
>
> **Consequence.** A scan that produces a clean report is not evidence that the target is clean; it may instead reflect an unreachable KEV feed. The feeds should be confirmed live before an assessment is relied upon. See LIMITATIONS.md.

Related documents

- SECURITY_AND_PRIVACY.md — the consolidated egress view
- AUTHORIZED_USE.md — which of these touch the target directly
- LICENSES_AND_ATTRIBUTION.md — service terms and attribution
- MODULE_REFERENCE.md — the functions behind each integration
- TROUBLESHOOTING.md — what to do when one of these fails

Brain Knowledge Base

RedFlag's self-improving attacker memory: what it is, where it lives, what it stores, and what is safe to commit.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Concept — learning by remembering

The brain gets sharper with every scan. It does this **without training a model**.

There is no neural network, no fine-tuning, no GPU, no training loop, and no paid API. The brain is a knowledge base on disk that **accumulates** what every scan has seen — techniques, CVEs, services, ports, attack paths, deal tiers — and **retrieves** the relevant parts when a new scan runs. Value compounds because the corpus grows, not because weights are adjusted.

The practical payoff on scan *n* is context that scan 1 could not have:

> NOTE
>
> *"CVE-2021-44228 — seen in 3 prior scans · KEV"* *"This exact kill-chain has been seen before."*

Why this design rather than a model: ADR-0005.

Implementation: `analysis/brain_memory.py`, pure standard library, ~340 lines, no third-party dependency.

2. Storage layout

The store lives **outside the project tree**. Two reasons: it is long-term memory that must survive a `git clean`, and writing inside the worktree trips Reflex's dev file-watcher, which hot-reloads the backend and destroys state mid-scan.

Root resolution order (`BrainMemory.__init__`):

1. an explicit `root` constructor argument
2. the `REDFLAG_BRAIN_DIR` environment variable
3. `~/RedFlag-Brain` (the default)

```
~/RedFlag-Brain/
├── brain.json          the index — everything the brain knows
└── vault/              a real Obsidian vault
    ├── README.md       written once, on first learn
    ├── Scans/          one note per scan: {target}-{YYYYMMDD-HHMMSS}.md
    ├── Techniques/     one note per MITRE technique: T-T1190.md
    └── CVEs/           one note per CVE: CVE-2021-44228.md
```

brain.json schema

```
{
  "version": 1,
  "scans": 12,                            // total scans learned
  "first_seen": "2026-06-29T10:14:02Z",
  "last_seen":  "2026-07-02T16:41:55Z",

  "techniques": { "T1190": { "count": 9, "name": "Exploit Public-Facing Application" } },
  "cves":       { "CVE-2021-44228": { "count": 3, "cvss": 10.0,
                                      "kev": true, "services": ["http"] } },
  "services":   { "ssh":  { "count": 14, "inet": 6 } },   // inet = times internet-facing
  "ports":      { "3389": { "count": 4 } },
  "paths":      { "T1190 → T1078 → T1021 → T1567": 2 },   // kill-chain signature → times seen
  "tiers":      { "critical": 41, "deal_killer": 3 },

  "targets":    { "example.com": { "scans": 2, "last": "..." } },   // ← STRIPPED from the seed
  "kev":        { "cves": ["CVE-..."], "count": 1284, "updated": "..." }
}
```

A **kill-chain signature** is the ordered list of MITRE technique IDs from the attack plan, joined with `→`. It only counts when there are at least two steps. That signature is what lets the brain recognise "this exact attack path has occurred before" across different targets.

The Obsidian vault

The `vault/` directory is a genuine Obsidian vault. Open it as a vault and use **Graph View** to see the accumulated knowledge as an actual graph: scan notes link to technique and CVE notes via `[[wikilinks]]`, so recurring CVEs and techniques become visible hubs.

The `README.md` inside the vault notes that anything *you* add by hand becomes part of the brain's knowledge too — there is no training step to re-run.

---

3. Recall, then learn

Order matters, and it is enforced by the caller in `redflag_ui/state.py:_learn_and_recall()`:

```
plan = analyze_attack_paths(self._findings, tgt or "target")
recall_summary, recall_items = brain.recall(self._findings, plan)   # 1. read prior knowledge
brain.learn_from_scan(self._findings, plan, tgt or "target")        # 2. then fold this scan in
```

If you learned first, every CVE in the current scan would report "seen in 1 prior scan" — the insight would be tautological. Recall must reflect what the brain knew **coming in**.

recall(findings, plan) -> (summary, insights)

- For each CVE in the current findings, looks up its prior `count`. Zero-count CVEs are skipped.
- Returns up to **6 insights**, sorted by prevalence descending, each labelled `"seen in N prior scans"` and suffixed `" · KEV"` when the CVE is known-exploited.

- Checks whether the current kill-chain signature has been seen before.
- On the very first run (`scans == 0`) it returns an explanatory first-scan message instead.

learnfromscan(findings, plan, target) [disk]

Increments the scan counter and, for every finding, bumps:

BUCKET	WHAT IS INCREMENTED
`services[svc]`	`count`, plus `inet` when the finding is internet-facing
`ports[port]`	`count`
`cves[cve]`	`count`; `cvss` takes the running **maximum**; `kev` latches to `true`
`tiers[tier]`	`count`
`techniques[tid]`	`count`, from the attack plan's steps
`paths[signature]`	`count`, when the chain has ≥ 2 steps
`targets[target]`	`scans` and `last` timestamp

Then it writes `brain.json` atomically (temp file + `os.replace`) and refreshes the vault notes.

Prevalence weighting is monotonic — the brain never forgets and never ages knowledge. A CVE seen once in 2026 carries the same evidential weight as one seen last week. Whether recency *should* matter is an open design question; see KNOWLEDGE_TRANSFER.md §4.

Read helpers

```
brain.stats() -> BrainStats           # scans, techniques, cves, services, paths, kev_known, vaul
    t_path
brain.top_techniques(limit=6)         # most prevalent MITRE techniques across the corpus
```

`stats()` is what renders the brain-memory panel line on the **Attack path** tab:

> NOTE
>
> *Learned from 12 scans · 9 techniques · 34 CVEs tracked · 1284 known-exploited*

---

4. The shipped seed

A fresh clone does not start with an empty brain.

The repository ships `analysis/brain_seed/brain.json` (~44 KB). On the first run on a machine with no `~/RedFlag-Brain/brain.json`, `BrainMemory._load()` calls `_bootstrap_from_seed()`, which copies the seed into place. From then on the local brain diverges and grows independently.

Refreshing the seed

```
python -m analysis.brain_memory
```

This calls `export_seed()`, which snapshots **this machine's** accumulated brain back into `analysis/brain_seed/brain.json` and prints a summary. Commit the result to share the knowledge with everyone who clones.

What sanitisation does — and what it does not

```
def export_seed(self, dest=None) -> str:
    data = json.loads(json.dumps(self.data))   # deep copy
    data["targets"] = {}                       # drop who-was-scanned
    ...
```

`export_seed()` strips **exactly one field**: `targets`. That is the only place in `brain.json` where a scanned host or IP is named. Everything else — technique, CVE, service, port, path and tier prevalence, plus the KEV list — is aggregate pattern knowledge that reveals nothing about who was assessed.

> NOTE
>
> **If you add a new field to `brain.json`, you must decide whether it identifies a target, and strip it in `export_seed()` if so.** This one function is the entire boundary between "local memory" and "safe to publish". It is the highest-consequence six lines in the codebase.

Note that a *local* brain is **not** sanitised — `~/RedFlag-Brain/brain.json` and the vault's `Scans/` notes do contain target names. They are outside the repository and therefore never committed, but they are real records of who you scanned. Treat that directory accordingly.

---

5. Threat-intel refresh

```
brain.ingest_kev() -> (ok: bool, message: str, count: int)
```

Triggered by the **Refresh threat intel** button on the Attack path tab (`RedFlagState.refresh_threat_intel`). It:

1. Downloads the free CISA Known Exploited Vulnerabilities feed (`https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json`, 10 s timeout, no key).
2. Stores the sorted CVE list in `brain.json` under `kev`.
3. **Back-fills** `kev: true` on every CVE the brain already knows that appears in the feed.

This is the **only network call anywhere in `analysis/`**, and it is manual and optional. Offline, it returns `(False, "Couldn't reach the CISA KEV feed (offline?) — …", 0)` and changes nothing.

6. Configuration and data retention

ASPECT	BEHAVIOUR
Location	`~/RedFlag-Brain`, overridable with `REDFLAG_BRAIN_DIR`
Committed to git	**Only** `analysis/brain_seed/brain.json` (sanitised)
Local brain in git	Never — it lives outside the repository entirely
Retention	Indefinite. Nothing expires, decays, or is pruned
Deletion	Delete `~/RedFlag-Brain`. The next run re-bootstraps from the seed
Portability	Plain JSON and Markdown. Copy the directory to move it
Failure mode	`_save()` and `_write_vault()` swallow **all** exceptions — the brain can never break a scan, but a failure is also invisible

Privacy note. The local brain accumulates a durable record of every target you have assessed: hostnames in `targets`, and scan notes naming them in `vault/Scans/`. If you assess third-party targets under an engagement agreement, that agreement may govern how long you may retain such records. Deleting `~/RedFlag-Brain` removes them completely. See SECURITY_AND_PRIVACY.md and AUTHORIZED_USE.md.

Debugging. If the brain "isn't learning", check in this order: does `~/RedFlag-Brain` exist, is it writable, and does `brain.json` have a recent `last_seen`? Because every write is wrapped in a bare `except: pass`, a permissions problem produces no error message at all.

Related documents

- ADR-0005 — why accumulate rather than train
- ADR-0001 — why there is no model here
- MODULE_REFERENCE.md — the `BrainMemory` API
- ARCHITECTURE.md — where the brain sits in the pipeline
- SECURITY_AND_PRIVACY.md — committed vs. local data

PART III

Operations

Installation, operation and diagnostics

Installation

Developer Onboarding

Deployment Runbook

Troubleshooting

Installation

A standalone guide to getting RedFlag running on Windows or macOS.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Prerequisites

REQUIREMENT	WHY	WINDOWS	MACOS
Python 3.11+	The whole application	python.org/downloads	`brew install python`
Git	Cloning the repository	git-scm.com	Pre-installed, or `brew install git`
Nmap	**Required.** The base scanner — without it a live scan fails	nmap.org/download	`brew install nmap`
Node.js 18+	Reflex compiles a Next.js frontend	Installed automatically by Reflex on first run, or nodejs.org	Same
Nuclei	*Optional.* Template DAST	ProjectDiscovery releases	`brew install nuclei`
Shodan API key	*Optional.* Live exposure lookups	Free tier at shodan.io	Same
Vulners API key	*Optional.* Exploit confirmation	Free tier at vulners.com	Same

> NOTE
>
> **You can run a complete assessment with no API keys and no Nuclei binary.** See §6.

> NOTE
>
> **Nmap discovery is Windows-path-based.** `scanners/nmap_scan.py:find_nmap()` checks only `C:\Program Files (x86)\Nmap\nmap.exe` and `C:\Program Files\Nmap\nmap.exe`. It does not consult `PATH`, so a Homebrew install at `/usr/local/bin/nmap` will not be found. On macOS you can still use every upload-driven feature; see §7.

2. Installation — Windows

Open **PowerShell** and run these in order.

Step 1 — Verify Python

```
python --version
```

Expect `Python 3.11.x` or higher. If not, install from python.org/downloads and **tick "Add Python to PATH"** during setup.

Step 2 — Install Nmap

Download the **stable self-installer** from nmap.org/download.html, run it with the default options (which install to `C:\Program Files (x86)\Nmap`), then verify:

```
nmap --version
```

Step 3 — Clone the repository

```
git clone https://github.com/adityaa206/redflag.git
cd redflag
```

> NOTE
>
> **Do not clone into OneDrive, Dropbox, or any synced folder.** The sync engine fights Reflex's Node/Vite build and produces `EBUSY` errors and a blank page. Use a path like `C:\Users\<you>\Redflag`.

Step 4 — Create and activate a virtual environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Your prompt will show `(venv)`. **Activate it in every new terminal.**

If PowerShell blocks the activation script, allow it for the current session only:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Step 5 — Install dependencies

```
pip install -r requirements.txt
```

Step 6 — Configure API keys (optional)

```
copy .env.example .env
notepad .env
```

Fill in whichever keys you have, save, and close. Both are optional — see §6.

Step 7 — Launch

```
python -m reflex run
```

Open <http://localhost:3000>. **The first run compiles the frontend — give it about a minute.**

---

3. Installation — macOS

Step 1 — Install Homebrew (if needed)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2 — Install Python, Git and Nmap

```
brew install python git nmap
python3 --version   # expect 3.11+
nmap --version
```

Step 3 — Clone the repository

```
git clone https://github.com/adityaa206/redflag.git
cd redflag
```

Step 4 — Create and activate a virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

Step 5 — Install dependencies

```
pip install -r requirements.txt
```

Step 6 — Configure API keys (optional)

```
cp .env.example .env
nano .env          # or: open -e .env
```

Step 7 — Launch

```
python -m reflex run
```

Open <http://localhost:3000>.

---

4. Running after the first install

Windows

```
cd C:\Users\<you>\Redflag
.\venv\Scripts\Activate.ps1
python -m reflex run
```

macOS

```
cd ~/redflag
source venv/bin/activate
python -m reflex run
```

Subsequent launches skip the frontend compile and start in a few seconds.

---

5. Configuration

Edit `.env` in the repository root:

```
# Optional — live Shodan lookups (1 credit per IP queried)
SHODAN_API_KEY=your_shodan_api_key_here

# Optional — per-CVE exploit confirmation via Vulners
VULNERS_API_KEY=your_vulners_api_key_here
```

- **Shodan** — account.shodan.io → API Key
- **Vulners** — vulners.com/userinfo → API Keys

`.env` is git-ignored and must never be committed. There is one further, non-secret variable set as a real environment variable rather than in `.env`:

VARIABLE	PURPOSE	DEFAULT
`REDFLAG_BRAIN_DIR`	Where the knowledge base is stored	`~/RedFlag-Brain`

Everything else is configured through the YAML files in `config/` — see CONFIGURATION.md.

6. Installing with no API keys

Skip step 6 entirely. RedFlag runs without a `.env` file at all.

WHAT YOU LOSE	WORKAROUND
Live Shodan lookups	Upload a Shodan host JSON in the **Shodan JSON** slot
Vulners exploit confirmation	Findings still score; CISA KEV and EPSS still supply exploit intelligence with no key
Nothing else	NVD, KEV, EPSS, DNS, TLS, crt.sh and LeakIX all work keyless

Ten of the fourteen integrations need no key. Full detail: INTEGRATIONS.md §16.

7. Installing the optional extras

Nuclei (template DAST)

```
# macOS
brew install nuclei

# Any platform, with Go installed
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
```

RedFlag finds it via `PATH` first, then `~/go/bin`, `C:\Program Files\Nuclei\`, `/usr/local/bin` and `/opt/homebrew/bin`. If it is absent, live Nuclei scanning is skipped silently — you can still upload `nuclei -jsonl` output produced anywhere.

Vulners NSE script (CVE lookup during the Nmap scan)

Download `vulners.nse` from the vulscan/vulners project and place it in Nmap's `scripts/` directory (`C:\Program Files (x86)\Nmap\scripts\` on Windows). RedFlag detects it automatically and adds `--script vulners` to the scan.

macOS without a Windows-path Nmap

`find_nmap()` only probes the two Windows paths, so a Homebrew Nmap will not be found and a live scan will raise `FileNotFoundError`. Two options:

1. **Use upload-only mode.** Leave the target field empty, stage OpenVAS/ZAP/Nuclei/Shodan files, and click **Run scan**. The whole pipeline runs on the uploads.
2. **Patch the path list.** Add your Nmap location to `NMAP_PATHS` in `scanners/nmap_scan.py`:

```
NMAP_PATHS = [
    r"C:\Program Files (x86)\Nmap\nmap.exe",
    r"C:\Program Files\Nmap\nmap.exe",
    "/opt/homebrew/bin/nmap",     # Apple Silicon
    "/usr/local/bin/nmap",        # Intel
]
```

8. Verifying the installation

```
# 1. Dependencies resolve and the engines import
python -c "import reflex, pydantic, pandas, networkx, fpdf, nmap, shodan, dns, cryptography, yaml;
    print('ok')"

# 2. The engine test suite passes — expect "143 passed"
pytest tests/ -v

# 3. Nmap is reachable (Windows)
python -c "from scanners.nmap_scan import find_nmap; print(find_nmap())"

# 4. The app starts
python -m reflex run
```

A green test run is the strongest single signal that the installation is sound — it exercises the scoring, maturity, cost, narrative, Day-1, EPSS, Nuclei, graph and integration engines without needing a network or a target.

9. What gets created on first run

PATH	WHAT	COMMITTED?
`.web/`	The compiled Next.js frontend and `node_modules`	No (git-ignored)
`venv/`	The Python virtual environment	No
`.states/`, `reflex.lock/`	Reflex runtime state	No
`%TEMP%/redflag_scans/`	Nmap XML output	No — outside the repository
`~/RedFlag-Brain/`	The knowledge base, bootstrapped from the shipped seed	No — outside the repository

The first launch is slow because `.web/` is being built. Later launches reuse it.

Related documents

- DEPLOYMENT_RUNBOOK.md — day-to-day running and operational cautions
- TROUBLESHOOTING.md — when a step above fails
- DEVELOPER_ONBOARDING.md — the next step for a developer
- USER_GUIDE.md — the next step for an analyst
- CONFIGURATION.md — everything configurable
- AUTHORIZED_USE.md — **read before your first scan**

Developer Onboarding

Getting productive in the RedFlag codebase.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Setup

```
git clone https://github.com/adityaa206/redflag.git
cd redflag
python -m venv venv

# Windows
.\venv\Scripts\Activate.ps1
# macOS / Linux
source venv/bin/activate

pip install -r requirements.txt
cp .env.example .env        # optional — both keys are optional
pytest tests/ -v            # expect: 143 passed
```

Full detail, including prerequisites and the optional binaries: INSTALLATION.md.

> NOTE
>
> **Do not put the checkout in OneDrive or any synced folder.** The sync engine fights Reflex's Node build.

2. Project layout tour

```
Redflag/
├── rxconfig.py           Reflex config (app_name="redflag_ui")
├── requirements.txt      Pinned dependencies
├── .env                  Secrets (git-ignored)

├── redflag_ui/           PRESENTATION — no business logic
│   ├── redflag_ui.py     rx.App + 9 routes
│   ├── state.py          RedFlagState: the pipeline + all view-models
│   ├── components/       shell (nav, scan bar, 5 upload slots), ui helpers
│   └── pages/            one module per route

├── scanners/             COLLECTION — the only layer that touches the network
├── analysis/             REASONING — pure, deterministic, offline
│   ├── schema.py         the Finding model + 6 enums
│   ├── triage.py         the scoring model
├── cost/                 Costing: catalog → estimator → dedupe → scenarios → rollup
```

```
── narrative/            Deterministic template prose
── reports/              CSV + PDF serialisation
── config/               loader.py + 7 YAMLs + scoring constants
── assets/               redflag.css — the entire visual design
── tests/                143 tests + fixtures
```

The full responsibility table is in ARCHITECTURE.md §5.

3. Running in development

```
python -m reflex run
```

Frontend <http://localhost:3000>, backend <http://localhost:8000>. Hot reload is on: editing a Python file under `redflag_ui/` recompiles and refreshes the browser.

Two consequences of hot reload:

1. **A compile error crashes the dev server.** Before a risky component edit, render-check it in the venv instead of relying on the live server:

```
from redflag_ui.pages.day1 import day1
day1().render()      # surfaces VarTypeError / binding errors immediately
```

2. **Writing inside the worktree during a scan resets backend state.** The file-watcher sees the write, hot-reloads, and the findings vanish. This is exactly why Nmap output goes to `%TEMP%/redflag_scans` and the brain to `~/RedFlag-Brain`. Never change that.

Alternate ports:

```
python -m reflex run --frontend-port 3001 --backend-port 8001
```

4. Running the tests

```
pytest tests/ -v                  # everything — 143 tests, ~9 seconds
pytest tests/test_triage.py -v    # one module
pytest tests/ -k "day1"           # by keyword
pytest tests/ -x                  # stop at the first failure
```

The suite is fast and needs **no network and no target** — every external call is either injected (EPSS takes a `scores` dict) or replaced by a fixture. Keep it that way: a test that hits the network is a test that will fail on a plane.

Coverage map and what is deliberately untested: TEST_PLAN.md.

5. Where business logic lives — and where it must not

KIND OF CHANGE	WHERE IT GOES
Scoring, weights, tiers	`analysis/triage.py` + `config/__init__.py`
A new evidence source	`scanners/` + a `ScannerSource` member
Maturity questions or thresholds	`config/maturity_questions.yaml`, `config/corporate_standard.yaml`
Day-1 rules, gates, models	`config/day1_blueprint.yaml`
Prices	`config/pricing_benchmarks.yaml`, `config/remediation_catalog.yaml`, `config/day1_cost_catalog.yaml`
Report wording	`config/narrative_blocks.yaml`
How something is *displayed*	`redflag_ui/pages/`, `redflag_ui/components/`, `assets/redflag.css`
Flattening engine output for display	`redflag_ui/state.py` view-models

The governing rule: a change that would alter a number in a report does not belong in `redflag_ui/`. The Streamlit to Reflex migration was inexpensive because this boundary had been maintained from the outset.

6. Conventions used in this codebase

Match what is there rather than importing your own style.

- **`from __future__ import annotations`** at the top of most modules; modern generics `list[Finding]`, `str | None`) throughout.
- **Dataclasses for engine outputs**, Pydantic for models that need validation `Finding`, the `cost/` schema).
- **Section banner comments** mark logical blocks:

```
# ── Phase assignment + roadmap ──────────────────────────────────────
```

- **A leading underscore means private to the module** `_matches`, `_v`, `_worst_tier`).
- **Enum normalisation is always** `str(getattr(x, "value", x))` — usually a local `_v()` helper.

Never `str(x)`: on an enum member that yields `"DealTier.CRITICAL"` and silently breaks every comparison.

- **Scanners never raise.** Wrap network I/O and return `[]` or an unchanged input. The two deliberate exceptions are `run_nmap_scan()` (missing binary) and `parse_asset_excel()` / `parse_shodan_json()` (malformed upload), which raise because the user needs to know.
- **Upgrade, never downgrade.** Merge and enrichment helpers use `_higher_exploit()` and `max()` on CVSS so a correlation can only strengthen a finding.
- **Config is read through `config/loader.py`**, never by opening a YAML directly, and never from `redflag_ui/`.
- **Backend-only Reflex vars carry a leading underscore** so raw engine objects are not serialised to the

browser.

- Docstrings are the module- and public-function-level narrative kind — explaining *why*, not restating the signature.

---

7. How to add things

A new scanner

1. `scanners/my_scanner.py` producing `Finding` objects.
2. Either a standalone `run_my_scan(target) -> list[Finding]` (like DNS/TLS) **or** a `parse_my_x()` + `merge_my_with_nmap()` pair (like OpenVAS/ZAP/Nuclei).
3. Add a `ScannerSource` member in `analysis/schema.py`.
4. Set `evidence_strength` honestly — it multiplies the score. `CONFIRMED` means *verified*, not *observed*.
5. Wrap all I/O so failure returns `[]`.
6. Call it from `RedFlagState.run_scan` at the right position — before `triage_all()`, and before EPSS if it can add CVEs.
7. Need an upload slot? Add to `_SLOTS` in `redflag_ui/components/shell.py` and add `upload_*`, `clear_*` handlers in `state.py`.
8. Add a fixture under `tests/fixtures/` and tests against it.

A new config knob

Add the key to the relevant YAML; add a loader accessor if it needs one; read it in the engine; document it in CONFIGURATION.md. Remember the loader caches per process — restart to see the change.

A new page

Create `redflag_ui/pages/my_page.py` returning `shell("Title", ...)`, then add it to `_PAGES` in `redflag_ui/redflag_ui.py`. Build the view-model in `state.py`; keep the page module free of logic.

A new cost item or report sentence

No code needed. Add an entry to `config/remediation_catalog.yaml` (or `config/day1_cost_catalog.yaml`), or a block to `config/narrative_blocks.yaml`.

---

8. Principal source files

FILE	LINES	SIGNIFICANCE
`analysis/schema.py`	91	The `Finding` model and all six enums — the vocabulary shared by every other module
`analysis/triage.py`	115	The scoring model: lookup tables, the weighted formula, the override rules
`redflag_ui/state.py`	1339	`RedFlagState.run_scan` — the pipeline; and every view-model
`analysis/day1.py`	423	The connectivity ladder, tier gates, review pillars and phase rules
`cost/rollup.py`	168	`run_cost_pipeline` — the entry point to the cost engine
`tests/test_triage.py`	258	The executable specification for the scoring model

Corresponding reference material: DATA_MODEL.md, RESULTS_INTERPRETATION.md, ARCHITECTURE.md and KNOWLEDGE_TRANSFER.md.

9. Candidate first changes

Items 1 to 3 of KNOWN_ISSUES_AND_BACKLOG.md §5 are self-contained and suitable as an initial contribution:

1. Add the missing `LICENSE` file.
2. Commit `tests/fixtures/mock_nuclei.jsonl` and correct the README's stale test count.
3. Remove the dead `analysis/graph_builder.py` and root `config.py`, having first ported the scoring-weight rationale comments into `config/__init__.py`.

The branch, test and pull-request workflow is defined in CONTRIBUTING.md.

Related documents

- CONTRIBUTING.md — branch, test and PR workflow
- ARCHITECTURE.md — structure and extension points
- MODULE_REFERENCE.md — signatures and side effects
- TEST_PLAN.md — the suite
- TROUBLESHOOTING.md — when the dev server misbehaves

Deployment Runbook

Operating RedFlag day to day: starting it, stopping it, where it writes, and what to be careful about.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Deployment model

RedFlag is a local, single-user desktop application. There is no server deployment, no container, no database, no authentication layer, and no multi-user support.

ASPECT	REALITY
Hosting	None. Runs on the operator's machine
Persistence	In-memory per session, plus local files
Users	One, on `localhost`
Authentication	None — the app assumes a trusted local machine
Network exposure	Binds `localhost` only
CI/CD	None configured

There is no hosted instance of RedFlag anywhere. The repository contains no Dockerfile, no CI workflows, no infrastructure-as-code and no deployment configuration, and `rxconfig.py` declares no remote API or deploy URL.

Do not expose the backend port to a network. There is no authentication, and anyone who can reach it can launch an active Nmap scan against an arbitrary target from your machine and your IP address. See AUTHORIZED_USE.md.

2. Starting the application

```
# Windows
cd C:\Users\<you>\Redflag
.\venv\Scripts\Activate.ps1
python -m reflex run
```

```
# macOS / Linux
cd ~/redflag
source venv/bin/activate
python -m reflex run
```

PORT	SERVICE
3000	Frontend (Next.js) — open this in a browser
8000	Backend (WebSocket state server)

Both are needed; the frontend talks to the backend over a WebSocket.

To change ports:

```
python -m reflex run --frontend-port 3001 --backend-port 8001
```

First launch compiles the Next.js frontend into `.web/`. This takes roughly a minute, needs Node.js 18+, and Reflex will offer to install Node if it is missing. Subsequent launches reuse the build and start in seconds.

3. Stopping the application

`Ctrl+C` in the terminal running `reflex run` stops both processes.

If a port is left occupied by an orphaned process:

```
# Windows — find and kill whatever holds port 3000
netstat -ano | findstr :3000
taskkill /PID <pid> /F
```

```
# macOS / Linux
lsof -ti:3000 | xargs kill -9
```

4. Where things are written

PATH	CONTENTS	LIFETIME	IN THE REPOSITORY?
`%TEMP%/redflag_scans/` **(** `$TMPDIR/redflag_scans` **on macOS)**	Nmap XML, one file per scan	Until the OS clears temp	**No**
`~/RedFlag-Brain/brain.json`	The knowledge-base index	Permanent	**No**
`~/RedFlag-Brain/vault/`	Obsidian notes: Scans, Techniques, CVEs	Permanent	**No**
`.web/`	Compiled frontend and `node_modules`	Until deleted	No (git-ignored)
`.states/`, `reflex.lock/`	Reflex runtime state	Session	No (git-ignored)
`uploaded_files/`	Reflex's upload staging area	Session	No

PATH	CONTENTS	LIFETIME	IN THE REPOSITORY?
Downloads	CSV and PDF exports go to the browser's download folder	Permanent	No

> NOTE
>
> **The two runtime paths outside the repository are deliberate and load-bearing.** Reflex's development file-watcher monitors the worktree; a write inside it triggers a hot reload that **resets backend state mid-scan** and loses the findings. The tell-tale is `Compiling...` appearing in the terminal during a scan. Do not "tidy" these paths back into the project.

Scan output and the brain may contain sensitive target data. Both are outside version control, but they are real records — see §7.

5. Operational cautions

Authorisation comes first

RedFlag performs **active scanning**. Nmap sends packets to the target; Nuclei sends application requests. Run it **only** against systems you own or have explicit written authorisation to assess. This is not a formality — unauthorised scanning is a criminal offence in many jurisdictions. Read AUTHORIZED_USE.md before your first scan and keep a record of the authorisation.

Know which feeds were live

A clean report is not proof the target is clean. If CISA KEV was unreachable, no deal-killer override will fire for an actively-exploited CVE, and the report will look reassuring. If Shodan was unavailable, exposure stays `PARTNER` / `INTERNAL` and every score is understated. Before relying on an assessment, confirm the feeds responded. The full failure-mode table is in INTEGRATIONS.md §17.

Scan timing and courtesy

`-T4` is aggressive timing. On a fragile target, or one behind an IDS, a full scan can trip alerts or degrade a service. **Fast mode** (top 200 ports) is both quicker and gentler. Coordinate timing with the target's operations team.

Credit consumption

A live Shodan lookup costs **1 credit per IP**. Staging a Shodan JSON upload skips the API call entirely and costs nothing — and the upload takes priority over the live call automatically.

Configuration changes need a restart

`config/loader.py` caches every YAML for the life of the process. Editing a YAML while the app is running has no

effect until you restart it.

6. Routine operations

Refresh threat intelligence

Attack path tab → **Refresh threat intel**. Pulls the current CISA KEV feed into the brain and back-fills the `kev` flag on every CVE it already knows. Worth doing before a significant assessment.

Refresh the shipped brain seed

```
python -m analysis.brain_memory
```

Snapshots your local brain into `analysis/brain_seed/brain.json`, **stripping target identities**, so a fresh clone starts pre-loaded. Commit the result if you want to share it.

Reset the brain

```
Remove-Item -Recurse -Force $HOME\RedFlag-Brain
```

The next run re-bootstraps from the shipped seed.

Clear scan output

```
Remove-Item -Recurse -Force $env:TEMP\redflag_scans
```

Force a clean frontend rebuild

```
Remove-Item -Recurse -Force .web
python -m reflex run
```

Do this if the UI renders blank or stale after a dependency change.

7. Data retention

Two locations accumulate records of who you assessed:

- `~/RedFlag-Brain/brain.json` → the `targets` map (hostnames and scan counts)
- `~/RedFlag-Brain/vault/Scans/` → one Markdown note per scan, named after the target

Nothing expires or is pruned. If you assess third-party targets under an engagement agreement, that agreement may govern how long you may keep such records. Deleting `~/RedFlag-Brain` removes them entirely.

Note that `analysis/brain_seed/brain.json` — the only brain data in the repository — has the `targets` map stripped by `export_seed()`. See BRAIN_KNOWLEDGE_BASE.md §4.

8. Logging and observability

There is **no logging framework**. Diagnostics are:

SOURCE	WHAT IT SHOWS
The `reflex run` terminal	`[INFO]` lines from the Nmap scanner (binary path, mode, NSE detection, host count), plus Reflex compile output
The UI notice bar	Scan result summary — finding count, deal-killer count, and which sources were fused
The UI error bar	`Scan failed: …` when the pipeline raises
Browser dev tools	Frontend and WebSocket errors
A log file	Only if `reflex run` output is explicitly redirected to one. None is created by default, and `*.log` is git-ignored

Because `run_scan` wraps each scanner in `except Exception: pass`, a scanner failing produces **no message at all** — it simply contributes nothing. To debug one, call it directly:

```
from scanners.dns_scan import run_dns_scan
print(run_dns_scan("example.com"))
```

Improving this — logging the swallowed exception rather than passing — is item 6 in KNOWN_ISSUES_AND_BACKLOG.md §5.

9. Health checks

```
# Engines healthy?
pytest tests/ -v                    # expect 143 passed

# Nmap reachable?
python -c "from scanners.nmap_scan import find_nmap; print(find_nmap())"

# Nuclei present? (optional)
python -c "from scanners.nuclei_scan import nuclei_available; print(nuclei_available())"

# Brain healthy?
python -c "from analysis.brain_memory import BrainMemory; print(BrainMemory().stats())"

# Config parses?
python -c "from config.loader import get_day1_blueprint; print(len(get_day1_blueprint()))"
```

Related documents

- INSTALLATION.md — first-time setup
- TROUBLESHOOTING.md — symptom → fix
- AUTHORIZED_USE.md — **required reading before operating**
- INTEGRATIONS.md — failure modes per feed
- BRAIN_KNOWLEDGE_BASE.md — the persistent store

Troubleshooting

Symptom -> cause -> fix.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Quick reference

SYMPTOM	LIKELY CAUSE	FIX
`'nmap' is not recognized` / `Could not find nmap.exe`	Nmap not installed, or installed outside the two paths RedFlag probes	§2.1
`ModuleNotFoundError` **on launch**	Virtual environment not activated	§2.2
`python: command not found` **(macOS)**	macOS ships `python3`	Use `python3`, or `brew install python`
Port 3000 or 8000 already in use	An earlier instance is still running	§2.3
First launch is very slow / asks to install Node	Reflex is building the frontend	Expected once, ~1 min. §2.4
Scan completes but returns no findings	Nmap missing, or the target has no open ports	§2.5
Scan runs, then findings disappear	Something wrote inside the worktree and triggered a hot reload	§2.6
Blank page / "cannot resolve react"	The checkout is inside OneDrive or another synced folder	§2.7
Everything scores low; nothing is internet-facing	No Shodan key and no Shodan upload	§3.1
No deal killers even with known-exploited CVEs	CISA KEV feed unreachable	§3.2
All CVEs show CVSS exactly 6.5	NVD unreachable — 6.5 is the silent fallback	§3.3
Every finding shows sensitivity "Unknown"	No asset-inventory Excel uploaded	§3.4
Excel upload fails with a column error	Missing host or sensitivity column	§3.5
Shodan JSON upload rejected	Not a Shodan *host* record	§3.6
Nuclei never contributes anything	Binary not installed	§3.7
"No DKIM Selector Found" on a domain that has DKIM	Custom selector name	§3.8
Day-1 tab says "awaits data"	No scan run and no questionnaire completed	§4.1
Day-1 always recommends Isolate	A gate criterion is failing	§4.2
Maturity gates fail even with good scores	Questionnaire not submitted	§4.3
Cost is $0 or every item flagged for review	Findings unscored, or nothing matched the catalogue	§4.4

SYMPTOM	LIKELY CAUSE	FIX
Brain shows 0 scans / never updates	`~/RedFlag-Brain` missing or not writable	§5.1
A YAML edit has no effect	The loader cache is per-process	§5.2
PDF export raises an encoding error	An unsupported character reached fpdf2	§5.3
Dev server dies on save	A compile error in a component	§5.4

2. Installation and startup

2.1 Nmap not found

```
Scan failed: Could not find nmap.exe. Check whether Nmap is installed in Program Files.
```

`scanners/nmap_scan.py:find_nmap()` probes **only** two paths and does **not** consult `PATH`:

- `C:\Program Files (x86)\Nmap\nmap.exe`
- `C:\Program Files\Nmap\nmap.exe`

Windows: reinstall Nmap with the default install location, then verify:

```
python -c "from scanners.nmap_scan import find_nmap; print(find_nmap())"
```

macOS/Linux: a Homebrew Nmap at `/opt/homebrew/bin/nmap` will not be found. Either use upload-only mode (leave the target blank and stage files), or add your path to `NMAP_PATHS` — see INSTALLATION.md §7.

2.2 ModuleNotFoundError

The virtual environment is not active. The prompt should show `(venv)`.

```
.\venv\Scripts\Activate.ps1     # Windows
source venv/bin/activate        # macOS / Linux
```

If PowerShell blocks the activation script:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

If it persists after activating, reinstall: `pip install -r requirements.txt`.

2.3 Port already in use

```
python -m reflex run --frontend-port 3001 --backend-port 8001
```

Or free the port:

```
netstat -ano | findstr :3000
taskkill /PID <pid> /F
```

```
lsof -ti:3000 | xargs kill -9
```

2.4 Slow first launch

Expected. Reflex compiles a Next.js frontend into `.web/` on first run — about a minute — and needs Node.js 18+. Let it install Node, or install it yourself from nodejs.org. Later launches reuse the build.

2.5 Scan returns no findings

In order of likelihood:

1. **Nmap is not installed** — see §2.1. This is by far the most common cause.
2. **The target genuinely has no open ports** in the scanned range. Try full mode instead of fast mode (fast mode only covers the top 200 ports).
3. **You are in upload-only mode with nothing staged.** With no target and no uploads, the scan button reports "*Enter a target host or IP, or stage an intelligence file, first.*"
4. **A firewall is dropping the scan.** Verify manually: `nmap -sV --open <target>`.

2.6 Findings vanish mid-scan

The tell-tale is `Compiling...` appearing in the `reflex run` terminal **during** a scan.

Reflex's development file-watcher monitors the worktree. Any write inside it triggers a hot reload, which **resets backend state** — the scan finishes but `_findings` has been wiped.

This is why Nmap writes to `%TEMP%/redflag_scans` and the brain to `~/RedFlag-Brain`. If you have changed either path to something inside the project, change it back. If an editor or a sync client is touching files in the worktree during a scan, stop it.

2.7 Blank page or broken frontend

Symptoms: a blank page, `cannot resolve react`, `EBUSY` errors, or npm repeatedly pruning.

Cause: the checkout is inside OneDrive, Dropbox or another synced folder. The sync engine locks and reshuffles files under `.web/node_modules` while Vite is building.

Fix: move the checkout outside the synced folder, for example to `C:\Users\<you>\Redflag`, then force a clean rebuild:

```
Remove-Item -Recurse -Force .web
python -m reflex run
```

3. Scanner and enrichment problems

Almost every scanner **degrades silently** — a failure produces no error, just less data. The symptoms below are how those silences look.

3.1 No Shodan data

Symptom: every finding shows exposure `Partner` or `Internal`; nothing is `Internet-facing`; scores look uniformly low.

Cause: no `SHODAN_API_KEY`, an API error, or an unresolvable hostname (Shodan needs a resolved IP).

Why it matters a lot: exposure carries 25% of the weighting, and the jump from `INTERNAL` (30) to `INTERNET_FACING` (100) is the single biggest score mover in the product. Without Shodan, scores materially understate risk.

Fix: add a key to `.env`, or upload a Shodan host JSON in the **Shodan JSON** slot — the upload takes priority over the live call and costs no credits.

3.2 KEV feed unreachable

Symptom: no deal-killer findings even though you know a known-exploited CVE is present.

Cause: `scanners/kev_lookup.py` could not reach the CISA feed. Its cache becomes `{}` and every `is_kev()` returns `False`, so the `ACTIVE_EXPLOITATION` override never fires.

Check:

```
from scanners.kev_lookup import fetch_kev_catalog
print(len(fetch_kev_catalog()))    # 0 means the feed failed; expect ~1000+
```

Fix: restore connectivity and rescan. EPSS provides partial cover but does not replace KEV.

3.3 Every CVSS is 6.5

`fetch_cvss_from_nvd()` returns **6.5 on any failure** — a silent substitution, not an error. If NVD is unreachable or rate-limited (5 requests / 30 s unauthenticated), a whole batch of CVEs will score exactly 6.5.

Check:

```
from scanners.shodan_scan import fetch_cvss_from_nvd
print(fetch_cvss_from_nvd("CVE-2021-44228"))   # expect 10.0, not 6.5
```

3.4 All findings "Unknown" sensitivity

Expected behaviour without an asset inventory. `data_sensitivity` is set from exactly one source: the **Asset inventory (Excel)** upload.

Consequences: the dimension worth 20% of the weighting scores a neutral 50, and **two of the three deal-killer override rules can never fire** — both require `CROWN_JEWEL` or `REGULATED`.

Fix: upload an `.xlsx` with a host column and a sensitivity column. See §3.5 for the format.

3.5 Asset Excel rejected

```
Excel must have an IP/host column and a sensitivity column. Found: [...]
```

Column detection is case-insensitive:

- **Host column:** header must be one of `ip`, `host`, `ip_address`, `hostname`, `address`
- **Sensitivity column:** header must contain `sensitiv` or `classif`, or be exactly `tier`

Accepted values: `crown_jewel` / `crown jewel` / `crownjewel`, `regulated`, `sensitive`, `low`, `unknown`. Anything else becomes `UNKNOWN`.

Also: host values must match the finding's `host` **exactly** — usually the IP address Nmap reported, not a friendly hostname.

3.6 Shodan JSON rejected

```
Could not find 'ip_str' in the uploaded file.
```

The file must be a Shodan **host record**, not a search-results page or an export of multiple hosts. Produce one with:

```
shodan host <ip> --save
```

or from `api.host(ip)` in the Shodan Python client.

3.7 Nuclei does nothing

`run_nuclei_scan()` returns `[]` silently when the binary is absent.

Check:

```
from scanners.nuclei_scan import nuclei_available, find_nuclei
print(nuclei_available(), find_nuclei())
```

Fix: install Nuclei (`brew install nuclei`, or `go install`), **or** run it elsewhere and upload the JSONL:

```
nuclei -u https://target -jsonl -o out.jsonl
```

Then stage `out.jsonl` in the **Nuclei JSONL** slot. No local binary is ever required.

3.8 False DKIM finding

RedFlag probes 12 common DKIM selector names (`default`, `google`, `mail`, `selector1`, `selector2`, `dkim`, `k1`, `s1`, `s2`, `email`, `mandrill`, `protonmail`). Selectors are arbitrary, so a domain using a custom name produces a false "No DKIM Selector Found".

This is why that finding alone carries `INFERRED` evidence (×0.85) rather than `CONFIRMED`. Verify manually:

```
dig TXT <your-selector>._domainkey.<domain>
```

To reduce false positives permanently, add your selector to `_DKIM_SELECTORS` in `scanners/dns_scan.py`.

Related: a DNS resolver outage returns empty record lists, which read as "not configured" — so a resolver problem can produce a burst of false SPF/DMARC/DKIM findings.

---

4. Analysis and output problems

4.1 Day-1 tab empty

> NOTE
>
> *Day 1 plan awaits data.*

The tab needs **either** a completed scan **or** a submitted maturity questionnaire. Run one.

4.2 Day-1 always recommends Isolate

`recommend_connectivity()` picks the highest tier whose gate **passes**, and `isolate` is the floor that always passes. Being stuck on Isolate means the Broker gate is failing.

Look at the **Tier gates** section — each criterion shows pass/blocked with its reason. The Broker gate has exactly one criterion: *no actively-exploited vulnerabilities*. One KEV finding pins you to Isolate.

Federate additionally requires no internet-facing remote access and `identity_access` + `network_security` maturity at or above `acceptable_min` — and a **maturity criterion fails if the domain was never assessed**, which is the most common blocker. See §4.3.

4.3 Maturity gates fail "unassessed"

> NOTE
>
> *Not assessed*

A `maturity_min` criterion fails when the domain has no answers — you cannot prove a posture you did not measure. Complete and submit the **Maturity** questionnaire. Only answered questions count toward a domain score, so you do not have to answer all 23 — but the relevant domain must have at least one answer.

4.4 Cost is zero or everything flagged

Cost is $0: the cost engine skips findings whose `deal_tier` is `UNSCORED`. If triage has not run, there is nothing to price. Run a scan first.

Everything flagged for review: findings are falling through to the catalogue's `default` entry, which sets the

`ZERO_ESTIMATE` flag deliberately so a human scopes it. Add matching entries under `service_entries` or `cve_overrides` in `config/remediation_catalog.yaml`.

Deal-killer items are always flagged by design (`DEAL_KILLER_ITEM`) — that is not a fault.

The accuracy band looks wide: it widens by 1.15× when the headcount is a default guess. Enter a real headcount in the Cost tab's What-If controls, and enter vendor quotes where you have them — a quote pins the item to `HIGH` confidence and collapses its spread.

5. Runtime and development problems

5.1 Brain not learning

Symptom: the Attack path tab shows 0 scans, or the count never rises.

Cause: `BrainMemory._save()` and `_write_vault()` swallow **all** exceptions so the brain can never break a scan — which means a permissions or disk problem is completely silent.

Check:

```
from analysis.brain_memory import BrainMemory
b = BrainMemory()
print(b.root)          # where it thinks the brain is
print(b.stats())       # scans, techniques, cves, last_seen
```

Then confirm the directory exists and is writable. If `REDFLAG_BRAIN_DIR` is set, check it points somewhere real.

Reset: delete `~/RedFlag-Brain`; the next run re-bootstraps from the shipped seed.

5.2 Config change not taking effect

`config/loader.py` caches every YAML for the **life of the process**. Restart the app. In tests, call `config.loader.reload_all()`.

5.3 PDF export fails

fpdf2 raises on a character outside the embedded font. All PDF text passes through `_safe()` in `reports/pdf_report.py`, which strips non-ASCII — but new content added without going through it will raise at export time.

If you add a symbol (an arrow, a star, a dash variant) to PDF output, either route it through `_safe()` or verify the font contains the glyph.

5.4 Dev server crashes on edit

A compile error in a Reflex component takes the whole dev server down. Before a risky component edit, render-check it in the venv:

```
from redflag_ui.pages.day1 import day1
day1().render()      # surfaces VarTypeError / binding errors without the live server
```

Two recurring causes:

- Passing a `Var` as `class_name` to `rx.upload` → "*Cannot iterate over Var*". Keep it static and put the conditional class on a wrapping `rx.el.div`.
- Building an f-string over a `Var` in a component. Precompute the string in `state.py` instead `bar_w="60%"`, `donut_gradient="conic-gradient(...)"`).

6. Diagnostic commands

```
# Full engine health — expect "143 passed"
pytest tests/ -v

# Binaries
python -c "from scanners.nmap_scan import find_nmap, vulners_nse_available; print(find_nmap(), vul
    ners_nse_available())"
python -c "from scanners.nuclei_scan import find_nuclei; print(find_nuclei())"

# Feeds
python -c "from scanners.kev_lookup import fetch_kev_catalog; print(len(fetch_kev_catalog()))"
python -c "from scanners.epss_scan import fetch_epss; print(fetch_epss(['CVE-2021-44228']))"
python -c "from scanners.shodan_scan import fetch_cvss_from_nvd; print(fetch_cvss_from_nvd('CVE-20
    21-44228'))"

# Individual scanners (bypasses the pipeline's exception swallowing)
python -c "from scanners.dns_scan import run_dns_scan; print(len(run_dns_scan('example.com')))"

# Config parses
python -c "from config.loader import get_day1_blueprint, get_remediation_catalog; print(len(get_da
    y1_blueprint()), len(get_remediation_catalog()))"

# Brain
python -c "from analysis.brain_memory import BrainMemory; b=BrainMemory(); print(b.root); print(b.
    stats())"
```

Related documents

- INSTALLATION.md — setup steps referenced above
- DEPLOYMENT_RUNBOOK.md — routine operations and health checks
- INTEGRATIONS.md §17 — the full failure-mode table
- LIMITATIONS.md — behaviour that is a limitation, not a bug
- KNOWLEDGE_TRANSFER.md §2 — the gotchas behind several of these

PART IV

User

Operating the application and interpreting its output

User Guide

Interpreting the Results

User Guide

How to run an assessment in RedFlag, tab by tab.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

> NOTE
>
> **Authorisation.** RedFlag performs active scanning. It may be run only against systems the operator owns or has explicit written authorisation to assess. See AUTHORIZED_USE.md.

1. Running a scan

Open <http://localhost:3000>. The scan bar sits at the top of every page.

Step 1 — Enter a target (optional)

A hostname (`example.com`) or an IP address. The target field is **optional**: if you only want to process uploaded scanner output, leave it blank and RedFlag skips the live scanners entirely.

Providing a target runs: Nmap, Vulners NSE (if installed), Shodan (if keyed and no upload staged), Nuclei (if installed), DNS, TLS and the breach check.

Step 2 — Stage supplementary files (optional)

Five upload slots. Each shows a filename chip once staged; a clear button removes it.

SLOT	FORMAT	WHAT IT ADDS
Shodan JSON	`.json`	Internet exposure, org/ASN/geo, known CVEs. **Takes priority over the live API — costs 0 credits**
OpenVAS XML	`.xml`	Verified CVEs and configuration flaws from a credentialed scan
OWASP ZAP XML	`.xml`	Web-application findings — SQLi, XSS, IDOR, headers, cookies
Nuclei JSONL	`.jsonl`	Confirmed vulnerabilities. Run `nuclei -jsonl` anywhere; no local binary needed
Asset inventory	`.xlsx`	Classifies hosts as Crown Jewel / Regulated / Sensitive — **the only source of data sensitivity**

Uploads do not replace the live scan — they correlate into it. An OpenVAS result matching a port Nmap already found upgrades that existing finding's evidence strength, CVSS and exploit status rather than creating a second entry. You get one ranked list, not five.

Step 3 — Choose the scan mode (optional)

Fast mode scans the top 200 ports with lower version-probe intensity — roughly 15–25 seconds versus 40–70 for a full scan. It is also gentler on a fragile target. Full mode finds more.

Step 4 — Click Run scan

DNS, TLS and breach checks run automatically alongside Nmap. When it finishes, the notice bar reports what was found and **which sources were fused** — for example:

> NOTE
>
> *Scan complete — 2 deal-killer findings across 47 total findings · sources: live scan, OpenVAS, asset inventory.*

The source list records the evidence on which the assessment is based.

Step 5 — Complete the maturity questionnaire (recommended)

Several capabilities stay locked until you do. Without it:

- Two of the three Day-1 review pillars read "*Not assessed*".
- The Federate and Integrate tier gates **fail automatically** — you cannot prove a posture you did not measure.
- No maturity gaps feed the roadmap or the cost model.

Go to **Maturity**, answer what you can, and submit. Only answered questions count, so a partial questionnaire is not penalised as if the unanswered ones scored zero.

2. Trying it without a live scan

Use the committed fixtures to exercise the whole pipeline offline:

FILE	UPLOAD INTO
`tests/fixtures/mock_openvas.xml`	OpenVAS XML
`tests/fixtures/mock_zap.xml`	OWASP ZAP XML
`tests/fixtures/mock_nuclei.jsonl`	Nuclei JSONL *(present on disk; see note below)*
`tests/fixtures/sample_assessment.json`	Reference data for tests — not an upload

Leave the target field **blank**, stage one or more files, and click **Run scan**.

The mock OpenVAS file contains EternalBlue, PrintNightmare, Log4Shell, default credentials, Telnet, Redis exposure, TLS misconfiguration and missing security headers. The mock ZAP file contains SQL injection, reflected XSS, IDOR, missing CSRF protection, directory listing, cookie flags, a vulnerable jQuery and an exposed Spring Actuator endpoint.

> NOTE
>
> `mock_nuclei.jsonl` exists in the working tree but has not been committed, so it may be absent in a fresh clone. See KNOWN_ISSUES_AND_BACKLOG.md §1.4.

3. Tab guide

Overview

The summary view. It presents:

- **The verdict** — Deal Killer / Critical / Moderate / Manageable, with a one-line explanation. A single deal-killer finding sets the verdict to Deal Killer regardless of the average.
- **Tier counts** across all findings.
- **A risk-breakdown donut** showing the proportional split by tier.
- **The findings table**, already sorted highest risk first.

This tab is the summary view; the remaining tabs provide the supporting detail.

Findings

Every finding with its CVE, CVSS, EPSS percentage, host and port, exposure level, evidence strength and risk score. Filterable by tier.

Each row reads: *what it is · how severe · how likely to be exploited · where it is · how well corroborated · what that adds up to*. How to interpret those: RESULTS_INTERPRETATION.md.

Attack path

Four sections:

- **The attacker's summary** — plain-language: how many internet-facing hosts give a way in, how many findings are weaponisable, how many internal hosts are reachable by pivoting.
- **The mind-map** — a radial diagram centred on the target with four branches: Entry points → Exploitation → Lateral movement → Impact, coloured by deal tier.
- **The MITRE playbook** — numbered attack steps, each linked to its technique page on attack.mitre.org `T1190`, `T1110`, `T1078`, `T1021`, `T1567` …).
- **Graph analysis** — the quantitative counterpart: **chokepoints** (fix this one host and *N* attack paths die), **blast radius** (how many assets are reachable from the internet), and the

shortest path to crown-jewel data.

- **Brain memory** — what RedFlag remembers from previous scans: "*CVE-2021-44228 — seen in 3 prior scans · KEV*", "*this exact kill-chain has been seen before*". The **Refresh threat intel** button pulls the current CISA KEV feed.

The mind-map is descriptive; the graph is quantitative. The chokepoint ranking is a remediation-leverage ranking and is the appropriate basis for prioritisation.

Maturity

23 questions across 7 domains: Identity & Access, Network Security, Endpoint Security, Application Security, Data Protection, Incident Response, Third-Party Risk.

Each question offers six options; **the selected option index is the maturity level, 0 to 5**. The scores determine the Day-1 tier gates and feed the cost model, so accuracy matters more than optimism.

Results show each domain's score against the acquirer's `acceptable_min` and `recommended` thresholds, with a RAG status. Domains at or below their `deal_blocker` threshold are flagged separately from the scan findings.

The scan sees what is exposed. The questionnaire sees what is governed. The Day-1 plan needs both.

Day 1 plan

The tab that turns the assessment into a decision for the moment the deal closes.

- **Recommended posture** — one of **Isolate → Broker → Federate → Integrate**. RedFlag recommends the *highest* tier whose entry gate passes: the most integrated posture the evidence actually justifies.
- **The ladder** — all four tiers, each marked Cleared, Active or Locked.
- **Tier gates** — what unlocks each tier, criterion by criterion, each showing Pass or Blocked with its reason. This is where you find out *why* you are stuck on a tier.
- **Review pillars** — Identity Sources, Network Boundaries and Remote Access Pathways, each with a RAG status, the evidence behind it, and a concrete recommendation.
- **Fix-first roadmap** — every finding and maturity gap sequenced into four phases:

PHASE	WINDOW	MEANING
P0 Pre-Connection Blocker	Before any link	Must be fixed or formally mitigated before the networks touch
P1 Day-1 Containment	At cutover	Handle behind an isolation boundary
P2 Stabilise	First 30 days	First-month uplift
P3 Integration-Ready	Day 30–100	Hardening to unlock the next tier

- **Architecture catalog** — all four models with their controls and cited industry sources.

The P0 list is the most actionable output in the product. It is the answer to "*what must be true before we connect?*"

Cost

Two buckets, kept separate:

- **Remediation** — fixing the findings and maturity gaps.
- **Integration** — standing up the recommended Day-1 connectivity model.

Every figure is a **low / base / high** triple with a CapEx/OpEx split, never a single number. Alongside it sits an **estimate accuracy** readout — an 80% confidence interval derived from each line item's confidence, so you can see how much to trust the total.

What-If controls:

- **Scenario** — switch between low, base and high.
- **Scope** — restrict to deal-killers, or to deal-killers plus criticals.
- **Include maturity gaps** — toggle whether questionnaire gaps are priced.
- **Headcount** — the acquired company's user count. Several integration costs scale per user, so entering a real number materially tightens the accuracy band (a default guess widens it by 15%).
- **Vendor quotes** — replace a benchmark with a firm quote. That pins the item to high confidence, collapses its range to a single figure, and clears its high-variance flag.

The **ladder cost** view prices every rung, so you can see what integrating faster would cost.

Items flagged for human review — high variance, deal-killer linked, or no catalogue match — are listed explicitly. That gate exists so a machine-generated number is never exported as if a human had checked it.

Export

EXPORT	CONTENTS
CSV	Every finding, 17 columns: title, CVE, host, port, service, scanner, CVSS, risk score, deal tier, exposure, sensitivity, exploit status, evidence, description, remediation, override reason, timestamp
Full PDF	Executive summary, verdict, findings detail
Day-1 PDF	The Safe Harbor Blueprint: posture, gates, pillars, roadmap
Cost PDF	The remediation cost model

> NOTE
>
> The cost PDF currently covers the **remediation bucket only** — the Day-1 integration budget, the ladder costs and the accuracy readout appear in the UI but not in that export. Tracked in KNOWN_ISSUES_AND_BACKLOG.md §3.

4. A worked sequence

For a real assessment, in this order:

1. **Get written authorisation.** Keep a copy.
2. **Complete the maturity questionnaire** — ideally with the target's IT lead. It unlocks the Day-1 gates.
3. **Gather artefacts** — ask the target for any recent OpenVAS/ZAP reports, a Shodan export, and an asset inventory with data classifications. The asset inventory matters most: without it, nothing is ever classified as crown-jewel or regulated, and two of the three deal-killer rules can never fire.
4. **Stage the uploads, enter the target, run the scan.**

5. **Read Overview → Attack path (graph analysis) → Day 1 plan (P0 list) → Cost.**
6. **Review the flagged cost items** before exporting anything.
7. **Export** and attach to the diligence pack.

5. Getting more out of it

WANT	DO
More accurate risk scores	Upload an **asset inventory** — sensitivity is 20% of the weighting
Correct internet exposure	Supply a Shodan key or upload a Shodan JSON — exposure is 25%
Higher-confidence findings	Upload OpenVAS/ZAP/Nuclei output — evidence strength multiplies the score
A tighter cost estimate	Enter a real headcount and any vendor quotes you have
Day-1 gates to evaluate at all	Complete the maturity questionnaire
Fresher exploit intelligence	Click **Refresh threat intel** on the Attack path tab
A faster, gentler scan	Enable **Fast mode**

Related documents

- RESULTS_INTERPRETATION.md — what every number means
- AUTHORIZED_USE.md — **required reading**
- LIMITATIONS.md — what RedFlag cannot tell you
- TROUBLESHOOTING.md — when a tab is empty or a scan misbehaves
- INSTALLATION.md — getting it running

Interpreting the Results

What every number RedFlag prints actually means, and how far to trust it.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

Every value in this document is taken from `analysis/triage.py` and `config/__init__.py`, not from the README.

1. The risk score

Each finding gets a **0–100 risk score**. It is not a severity rating — it is an estimate of commercial risk to the deal, which is why a CVSS 9.8 on an unreachable internal box can score lower than a CVSS 7.5 on an internet-facing system holding regulated data.

Two steps: a weighted base score, then an evidence adjustment.

Step 1 — the weighted base

```
cvss_normalized = (cvss_score / 10.0) * 100        # 0–10 → 0–100

base_score = (exploit_score     * 0.30)    # can it actually be exploited?
           + (exposure_score    * 0.25)    # can an attacker reach it?
           + (cvss_normalized   * 0.25)    # how severe is it technically?
           + (sensitivity_score * 0.20)    # what is at risk behind it?
```

The four weights sum to 1.0, so the base score is itself 0–100.

Why exploitability outweighs severity. Aligned with CISA's SSVC methodology and EPSS research: a known-exploited vulnerability demands action regardless of its CVSS, while a theoretical CVSS 10 with no exploit and no reachability is a backlog item. Exposure and CVSS are weighted equally because a high CVSS on an unreachable service is genuinely less urgent than a moderate CVSS on an internet-facing one.

Step 2 — the evidence adjustment

```
risk_score = round(base_score * evidence_multiplier, 2)
```

The multiplier is 0.80–1.00, so the worst case is a 20% discount. Two findings with identical severity, exposure and exploitability differ by up to 20 points depending on how well corroborated they are — a verified OpenVAS result outranks a banner-only inference. Rationale: ADR-0006.

2. The lookup tables

All four verified against `analysis/triage.py`.

Exposure — can an attacker reach it?

LEVEL	SCORE	SET BY
Internet Facing	**100**	Shodan port confirmation; DNS, TLS, breach and Shodan-standalone findings
Partner	**60**	`analysis/parser.py` for commonly exposed services (http, https, ssh, rdp, ftp, telnet, vnc, smb, netbios-ssn)
Internal	**30**	`analysis/parser.py` default for everything else
Unknown	**50**	Not established

The `INTERNAL` → `INTERNET_FACING` jump is worth **17.5 points** of base score on its own. This is the largest single mover in the model, and it usually comes from Shodan. **Without Shodan data, scores materially understate risk.**

Data sensitivity — what is at risk?

LEVEL	SCORE	MEANING
Crown Jewel	**100**	The organisation's most valuable data or systems
Regulated	**85**	GDPR / HIPAA / PCI-DSS scope
Sensitive	**55**	Confidential but not regulated
Low	**20**	Public or low-value
Unknown	**50**	Not classified

Set from exactly one source: the **asset-inventory Excel upload**. Without it every finding sits at the neutral 50 — **and two of the three deal-killer override rules become unreachable**, because both require Crown Jewel or Regulated.

Exploit status — is it actually exploitable?

STATUS	SCORE	SOURCE
Active Exploitation	**100**	CISA KEV, or a LeakIX credential/database exposure
Public Exploit	**65**	Vulners confirmation, or **EPSS ≥ 0.50 promotion**
Unknown	**30**	Nothing established
No Exploit	**10**	Confirmed no known exploit

Note that **Unknown (30) scores higher than No Exploit (10)**. That is deliberate: absence of evidence is treated as more dangerous than evidence of absence.

Evidence strength — how well corroborated?

STRENGTH	MULTIPLIER	TYPICAL SOURCE
Confirmed	**1.00**	Nmap confirmed the port open; OpenVAS/ZAP/Nuclei verified the vulnerability; LeakIX observed the exposure
Correlated	**0.95**	Two independent sources agree — usually Nmap plus Shodan
Unknown	**0.90**	Not established
Inferred	**0.85**	Deduced rather than observed — e.g. no DKIM found across 12 common selectors
External	**0.80**	Third-party intelligence, unverified — Shodan's own CVE list

3. Deal-killer overrides

Four conditions **bypass scoring entirely**. When one fires, `risk_score` is forced to exactly **100.0**, the tier becomes `DEAL_KILLER`, and `override_reason` explains why.

#	CONDITION	WHY
1	`exploit_status == ACTIVE_EXPLOITATION`	The CVE is in CISA KEV — confirmed exploited in the wild. Weaponised code exists now
2	`data_sensitivity == CROWN_JEWEL` **and** `cvss >= 9.0` **and** `exposure == INTERNET_FACING`	The most valuable asset, critically vulnerable, publicly reachable
3	`data_sensitivity == REGULATED` **and** `cvss >= 9.5` **and** `exposure == INTERNET_FACING`	Regulatory liability at critical severity, publicly reachable
4	`override_reason` contains the phrase `"active compromise"`	Manual analyst flag

Reading a deal killer: it is not "a very high score". It is a categorical statement that this finding alone should stop the deal until it is resolved or contractually mitigated. Rules 2 and 3 can only fire if you uploaded an asset inventory.

4. The four deal tiers

Score thresholds from `config/__init__.py`.

TIER	TRIGGER	WHAT IT MEANS	RECOMMENDED ACTION
Deal Killer	Any override rule	Unacceptable pre-close risk	Escalate before signing. Remediate or contractually mitigate
Critical	Score **≥ 75**	Severe exposure	Remediate within 30 days of close; obtain a written commitment
Moderate	Score **≥ 50**	Material gap	90-day post-close security roadmap

TIER	TRIGGER	WHAT IT MEANS	RECOMMENDED ACTION
Manageable	Score **< 50**	Standard hygiene	Normal backlog
Unscored	Triage has not run	Not yet assessed	Skipped by the cost engine

The Overview verdict

The headline verdict is not simply the worst finding:

VERDICT	CONDITION
Deal Killer	At least one deal-killer finding — *regardless of the average*
Critical	No deal killers, but the **average** score ≥ 75
Moderate	No deal killers, average ≥ 50
Manageable	No deal killers, average < 50

One deal killer among a hundred benign findings still produces a Deal Killer verdict. That is correct — a single actively-exploited internet-facing CVE is not diluted by good hygiene elsewhere.

5. EPSS promotion

EPSS (Exploit Prediction Scoring System, from FIRST.org) gives the probability that a CVE will be exploited **in the next 30 days**.

RedFlag fetches it for every CVE and displays it on each finding (`EPSS 92%`). Where it changes the answer:

> NOTE
>
> If a CVE has **EPSS ≥ 0.50** and its exploit status is `UNKNOWN` or `NO_EXPLOIT`, that status is promoted to `PUBLIC_EXPLOIT` — moving its exploit component from 30 (or 10) to 65.

The effect on the score is **+10.5 points** from `UNKNOWN`, or **+16.5** from `NO_EXPLOIT`.

Why: CISA KEV is retrospective — a CVE enters it after exploitation is observed. EPSS is predictive. A CVE that is 92% likely to be exploited this month should be treated like one with a known public exploit, not parked because nobody has filed the paperwork yet. Rationale: ADR-0007.

Promotion is marked in the finding's `raw_data` as `epss_promoted: true`. It never downgrades a finding, and never touches one already at `ACTIVE_EXPLOITATION`.

6. Reading the Attack Path tab

The mind-map narrates; the graph measures. Use them for different questions.

The MITRE playbook

Numbered attacker steps, each mapped to a MITRE ATT&CK technique with a link to its page. It describes the **single worst path** — the highest-risk entry point followed through to impact — not every possible path.

This is derived from your findings by a rule-based expert system, not observed. It answers *"what would a competent attacker most likely do?"*, not *"what has happened."*

Graph analysis

METRIC	QUESTION IT ANSWERS	HOW TO USE IT
Chokepoints	Which single host or service, if fixed, severs the most attack paths?	**The remediation-leverage ranking.** "Patch this one box and 6 paths die"
Blast radius	How many assets are reachable from the public internet?	The scale of the exposure
Crown-jewel paths	The shortest route from the internet to sensitive data	Fewer hops = more urgent

A caveat on the graph model: lateral movement is modelled as *any* internet-facing host being able to pivot to *any* internal host. That is a deliberately pessimistic assumption. It is right for a flat network and overstates risk in a well-segmented one — but you rarely know which you have before you look, and segmentation is exactly what the maturity questionnaire asks about.

Brain memory

Prevalence across previous scans: *"seen in 3 prior scans · KEV"*. This is corpus context, not a judgement about this target. A CVE seen often is a CVE that recurs across the estates you assess — useful for pattern recognition, irrelevant to this finding's severity.

7. Reading the Maturity tab

Scores are **0–5 per domain**, a weighted mean of the questions you answered.

LEVEL	ROUGH MEANING
0	Nothing in place
1	Ad hoc, undocumented
2	Basic controls, inconsistently applied
3	Defined and mostly consistent
4	Managed, measured, reviewed
5	Optimised, automated, continuously improved

Each domain is compared against three thresholds from `config/corporate_standard.yaml`:

THRESHOLD	MEANING
`deal_blocker`	At or below this → flagged as a maturity deal blocker
`acceptable_min`	Below this → a gap requiring an agreed improvement plan
`recommended`	The 12-month post-acquisition target

Most domains are set to `acceptable_min: 2`, `recommended: 3` or `4`, `deal_blocker: 1`. **Incident Response is the exception** `deal_blocker: 0`): a missing IR plan is a real gap, but on its own it is not a reason to walk away.

Only answered questions count. A partial questionnaire is not penalised as if the unanswered questions scored zero — but an unanswered *domain* has no score at all, which causes the Day-1 tier gates that depend on it to fail as "*Not assessed.*"

Two things trigger an overall maturity deal-blocker: any single domain at or below its `deal_blocker` threshold, **or** the mean across all seven domains falling to 1.5 or below.

8. Reading the Day-1 ladder

RedFlag recommends the **highest tier whose entry gate passes** — the most integrated posture the evidence justifies, not the safest possible one.

TIER	WHAT IT MEANS	GATE REQUIREMENTS
Isolate	No network route at all. The business runs standalone under a TSA; data moves through a governed clean room	*(none — always available)*
Broker	Users reach apps through a brokered virtual desktop. Pixels in, no data egress, no network route — a compromised target cannot pivot into the parent	No actively-exploited vulnerabilities
Federate	Limited identity federation (one-way trust or Entra B2B) plus per-app ZTNA access, replacing legacy VPN/RDP	The above, plus: no internet-facing remote access; Identity and Network maturity ≥ acceptable_min
Integrate	Full directory consolidation and routed networks	The above, plus: no internet-facing deal-killers; Identity and Network maturity at **recommended**

Reading a blocked gate. Each criterion shows Pass or Blocked with its reason. The tab also names exactly what would unlock the next tier up. A `maturity_min` criterion **fails when the domain was never assessed** — you cannot prove a posture you did not measure, so an incomplete questionnaire pins you to a lower tier.

The P0–P3 roadmap

Every finding and maturity gap lands in exactly one phase, by the **first matching rule**:

PHASE	WINDOW	WHAT LANDS HERE
P0 Pre-Connection Blocker	Before any link	Actively exploited · deal killers · internet-facing remote access · internet-facing public exploits · maturity deal-blockers

PHASE	WINDOW	WHAT LANDS HERE
P1 Day-1 Containment	At cutover	Other internet-facing exposure · partner-exposed remote access
P2 Stabilise	First 30 days	Critical-tier findings · partner-exposed services · below-minimum maturity gaps
P3 Integration-Ready	Day 30–100	Everything else — internal hygiene

The P0 list is the single most actionable output in the product. It is the literal answer to "*what must be true before these two networks touch?*"

9. Reading the Cost tab

Never a single number

Every figure is a **low / base / high** triple:

- **low** — best case: minimal scope, in-house labour
- **base** — most likely: benchmark rates, standard scope
- **high** — worst case: consultant rates, full scope, overtime

Use **base** for planning and **high** for negotiating a price adjustment.

Two buckets

BUCKET	WHAT IT PRICES
Remediation	Fixing the findings and closing the maturity gaps
Integration	Standing up the recommended Day-1 connectivity model

They are deliberately separate and never deduplicated against each other — they answer different questions: *what does the debt cost* versus *what does connecting safely cost*.

CapEx vs OpEx

CapEx is one-time (tools, infrastructure, architecture); OpEx is recurring (labour, subscriptions, retainers). Items marked `MIXED` split 50/50. The distinction matters for how the cost is treated in the deal model.

Estimate accuracy

The accuracy readout is a **variance-based 80% confidence interval** (P10 / P50 / P90), not an average of the ranges.

Its honesty is worth understanding. Each line item is treated as a triangular distribution, widened by its confidence (`high` `×1.0`, `medium` `×1.25`, `low` `×1.6`). Items are then aggregated with **partial correlation (0.35)** — independent estimating errors partly cancel out, which tightens the band, but a common-mode market factor

stops it becoming unrealistically tight. The result is clamped to ±8–55%. If the headcount was a default guess rather than a real number, the band widens by a further 15%.

This means "±22%" is a genuine statistical statement, not a gesture. To tighten it: enter the real headcount, and enter vendor quotes where you have them — a quote pins that item to high confidence and collapses its range to a single figure.

Review flags

FLAG	MEANING
`HIGH_VARIANCE`	The high/low spread exceeds 3× — scope this manually
`ZERO_ESTIMATE`	No catalogue entry matched; it fell through to the default
`DEAL_KILLER_ITEM`	Linked to a deal-killer finding — always flagged
`DUPLICATE`	Merged from several findings needing the same fix
`MANUAL_REQUIRED`	Needs human judgement

The review gate exists so that a machine-generated number is never exported as if a human had checked it. **Read the flagged items before exporting.**

What-If

Switching scenario, restricting scope to deal-killers, toggling maturity gaps, changing headcount and entering quotes all recompute the totals live — the pipeline is deterministic and fast enough to re-run on every change.

10. How much to trust a RedFlag number

High confidence: an open port from Nmap; a certificate expiry date; a missing DMARC record; a LeakIX-indexed exposure. These are observed facts.

Medium confidence: the risk score. The formula is deterministic and the weights are defensible, but the inputs vary in quality — and two of the four dimensions depend on data you may not have supplied.

Read with care:

- **Any assessment run without an asset inventory.** Sensitivity sits at a neutral 50 everywhere, and two of the four deal-killer rules cannot fire.
- **Any assessment run without Shodan data.** Exposure is understated, and exposure is 25% of the weighting.
- **A clean report when a feed was down.** If CISA KEV was unreachable, no deal-killer override fires for an actively-exploited CVE, and the report looks reassuring. If NVD was unreachable, every CVE silently scores 6.5. Check the notice bar's source list, and see INTEGRATIONS.md §17.
- **The absence of a finding.** RedFlag reports what it observed. It cannot report what it could not see — authenticated flaws, internal systems, application logic, insider risk.

RedFlag is not a penetration test and not a certified audit. It is a structured, repeatable first pass that tells you where to point the expensive people. See LIMITATIONS.md.

Related documents

- USER_GUIDE.md — how to produce these results
- LIMITATIONS.md — the honest caveats in full
- DATA_MODEL.md — the fields behind every value here
- CONFIGURATION.md — how to retune the weights and thresholds
- ADR-0006, ADR-0007

PART V

Testing

Verification coverage and accuracy caveats

Test Plan

What is tested, how to run it, and — just as importantly — what is not covered.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Summary

Framework	pytest 9.0.3
Test files	11
Test cases	**143**
Status	**143 passed** (verified 2026-07-27)
Runtime	~9 seconds
Network required	**No**
Target required	**No**
External binaries required	**No**

The suite is a pure **engine** suite. It exercises the deterministic layers — scoring, maturity, Day-1, cost, narrative, EPSS, Nuclei parsing and graph analytics — and deliberately excludes the Reflex UI and all live network behaviour. That is why it runs in nine seconds on a plane.

2. How to run it

```
# Everything
pytest tests/ -v

# One module
pytest tests/test_triage.py -v
pytest tests/test_day1.py -v
pytest tests/test_estimator.py -v

# By keyword across the suite
pytest tests/ -k "day1"
pytest tests/ -k "epss or nuclei"

# Stop at the first failure
pytest tests/ -x

# Quiet — just the count
pytest tests/ -q
```

Activate the virtual environment first, or the imports will fail.

3. Coverage map

FILE	TESTS	COVERS
`test_triage.py`	**24**	The scoring model: lookup tables, the weighted formula, the evidence multiplier, all deal-killer overrides, tier thresholds, and `triage_all` sort order
`test_day1.py`	**26**	The Day-1 engine end to end: remote-access detection by service and by port, all nine phase rules, roadmap construction with maturity gaps, tier-gate evaluation, connectivity recommendation, review pillars, and graceful degradation without a questionnaire
`test_estimator.py`	**23**	The cost engine: `CostTriple` arithmetic, catalogue lookup (CVE override → service → tier → default), deal-killer flagging, deduplication and its conservative merge, rollup totals, CapEx/OpEx split, scenarios, and the review gate
`test_maturity.py`	**17**	Domain scoring including weighting, partial answers and clamping; severity classification; overall assessment; and gap-report comparison against the corporate standard
`test_narrative_engine.py`	**15**	Block selection, priority ordering, condition matching, variable substitution, and each narrative builder
`test_day1_cost.py`	**10**	The integration budget: catalogue completeness, per-user scaling, ladder costing for all four tiers, pipeline wiring, accuracy readout, headcount-assumed penalty, and vendor-quote overrides
`test_integration.py`	**7**	End to end from a fixture through maturity → cost → narrative → CSV export
`test_epss.py`	**6**	EPSS attachment, promotion at threshold, non-promotion below it, never downgrading `ACTIVE_EXPLOITATION`, and no-op cases
`test_attack_graph.py`	**5**	Blast radius, chokepoint ranking, crown-jewel path discovery, no-internet and empty cases
`test_nuclei.py`	**5**	JSONL parsing, host/port/CVE extraction, severity→CVSS baseline, info-level filtering, and the correlation merge
`test_simulation.py`	**5**	The variance-based confidence interval and accuracy percentage

4. What each module proves

`test_triage.py` — the most important file in the suite. It pins the exact numbers in `EXPOSURE_SCORES`, `SENSITIVITY_SCORES`, `EXPLOIT_SCORES` and `EVIDENCE_MULTIPLIERS`, verifies the weighted formula, and checks every deal-killer override independently. It constitutes the executable specification for the scoring model and constrains any change to `analysis/triage.py`.

`test_day1.py` — the largest file. It walks every phase rule in order `active_exploitation` → P0, `internet + remote_access` → P0, `partner + remote_access` → P1, plain internal → P3…) and asserts the recommendation

logic in both directions: `test_recommend_integrate_when_clean_and_mature` proves the ladder climbs when the evidence allows it, and `test_internet_rdp_blocks_federate_even_with_max_maturity` proves that a single internet-facing RDP service pins you below Federate *no matter how good the questionnaire looks*. There are explicit tests for degradation without a questionnaire and with no findings at all.

`test_estimator.py` — proves the catalogue resolution order and, importantly, that deduplication is **conservative**: merged items take `min(low)`, `max(base)`, `max(high)`, so collapsing ten identical findings never understates the fix.

`test_epss.py` — runs entirely offline by injecting a `scores` dict, which is the pattern the whole suite follows for external data. It proves promotion fires at the threshold, does not fire below it, and **never downgrades** an actively-exploited finding.

`test_day1_cost.py` — proves the integration budget scales with headcount, that all four ladder rungs price above zero, that the accuracy band widens when headcount is assumed, and that a vendor quote pins its item to high confidence and tightens the overall accuracy.

`test_integration.py` — the only test that runs several engines in sequence, driven by `tests/fixtures/sample_assessment.json`. It also asserts the fixture exists and has the required keys, so a missing fixture fails loudly rather than skipping silently.

5. Fixtures

FILE	SIZE	PURPOSE	COMMITTED
`tests/fixtures/mock_openvas.xml`	9.8 KB	OpenVAS/GVM report for upload testing and manual demos	Yes
`tests/fixtures/mock_zap.xml`	14.2 KB	OWASP ZAP report	Yes
`tests/fixtures/sample_assessment.json`	4.1 KB	Drives `test_integration.py`	Yes
`tests/fixtures/mock_nuclei.jsonl`	4.2 KB	Nuclei JSONL for upload testing	Yes

Contents of the mocks — `mock_openvas.xml` contains EternalBlue, PrintNightmare, Log4Shell, default credentials, Telnet, Redis exposure, TLS misconfiguration and missing security headers. `mock_zap.xml` contains SQL injection, reflected XSS, IDOR, missing CSRF protection, directory listing, insecure cookie flags, a vulnerable jQuery and an exposed Spring Actuator endpoint.

All four fixtures are committed, so a fresh clone can reproduce every upload workflow described in USER_GUIDE.md. Note that `test_nuclei.py` does **not** read `mock_nuclei.jsonl` — it injects its JSONL inline — so the fixture serves the manual upload walkthrough rather than the suite.

6. Testing philosophy

Three rules, visible throughout the suite:

1. **No network, ever.** External data is injected — EPSS takes an explicit `scores` dict; Nuclei takes inline JSONL;

OpenVAS and ZAP take fixture files. A test that reaches the internet is a test that fails on a plane and lies in CI.

2. **No mocking framework.** The engines are pure functions over plain objects, so tests construct real `Finding` objects and assert on real output. If a module needs heavy mocking to test, that is a signal it has the wrong dependencies.
3. **Test the contract, not the implementation.** Tests assert scores, tiers, phases and totals — the things a user sees — rather than internal call sequences. This is what let the Streamlit → Reflex migration happen without touching a single test.

7. What is NOT covered

Stated plainly, because it matters more than the coverage that exists.

The Reflex UI layer — untested

`redflag_ui/` has **no test coverage at all**. That includes:

- `RedFlagState.run_scan` — the entire pipeline orchestration
- All upload handlers and staging logic
- Every view-model flattening function
- All page components and rendering

`build_view()` is written as a module-level pure function specifically so it *could* be tested without Reflex. It currently is not. This is the largest single coverage gap.

Note that the earlier Streamlit app *did* have UI tests; they were removed during the Reflex migration and never replaced.

Live network behaviour — untested

No test exercises a real Nmap scan, a real Shodan lookup, a live KEV or EPSS fetch, a DNS query, a TLS handshake, a crt.sh query, or a LeakIX call. The scanners' **parsers** are tested against fixtures; their **network paths and failure modes** are not.

That matters because the failure modes are silent — for example, `fetch_cvss_from_nvd()` returning 6.5 on any error. Nothing verifies that behaviour.

Also untested

AREA	NOTE
PDF generation (`reports/pdf_report.py`)	440 lines, entirely uncovered. The `_safe()` character-stripping path is a known source of runtime errors
The brain (`analysis/brain_memory.py`)	Recall, learn, vault writing and `export_seed()` are all uncovered — including the sanitisation that decides what is safe to commit
`analysis/parser.py`	Nmap XML parsing has no direct test; it is only exercised indirectly

AREA	NOTE
The Shodan enrichment path	`enrich_findings_with_shodan()` and `create_shodan_findings()` are uncovered
DNS / TLS / breach scanners	No fixture-driven tests
XLSX export (`cost/exporters.py`)	CSV export is covered by `test_integration.py`; XLSX is not
`analysis/parsers/excel_assets.py`	Column detection and the upgrade-only rule are uncovered

Highest-value tests to add

1. **`export_seed()` sanitisation** — assert that `targets` is stripped and that no other field contains a hostname. This is the boundary between local memory and published data, and it is currently unguarded.
2. **`build_view()`** — pure and already isolated; a handful of tests would cover a large surface.
3. **`apply_sensitivity_to_findings()`** — assert it upgrades `UNKNOWN` and never downgrades.
4. **`reports/pdf_report.py` smoke test** — generate a PDF from fixture findings and assert it is non-empty. Would catch the encoding failures.
5. **Scanner degradation** — assert each scanner returns `[]` (rather than raising) when its dependency is missing.

8. Coverage measurement

Coverage is not currently measured. To generate a report:

```
pip install pytest-cov
pytest tests/ --cov=analysis --cov=cost --cov=narrative --cov=scanners --cov-report=term-missing
```

Expect high coverage on `analysis/triage.py`, `analysis/day1.py`, `analysis/maturity.py` and the `cost/` package, and near-zero on `redflag_ui/`, `reports/pdf_report.py` and the network paths in `scanners/`.

`pytest-cov` is not in `requirements.txt` — it is a development-only tool.

9. Continuous integration

None configured. There is no GitHub Actions workflow, no pre-commit hook, and no automated gate. Tests are run manually.

Adding a workflow that runs `pytest tests/ -v` on push would be a small, high-value change: the suite needs no network, no secrets and no binaries, so it would work in a stock runner with only `pip install -r requirements.txt`.

Related documents

- LIMITATIONS.md — accuracy caveats that are by design, not gaps in testing
- DEVELOPER_ONBOARDING.md — running tests while developing
- KNOWN_ISSUES_AND_BACKLOG.md — the fixture and count defects
- MODULE_REFERENCE.md — what each tested function does

Limitations

An honest account of what RedFlag cannot tell you, and where its output should not be trusted without corroboration.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. RedFlag is not a penetration test

State this plainly to anyone who receives a RedFlag report.

RedFlag **observes and reasons**. It never exploits anything. It does not attempt authentication, does not chain vulnerabilities to prove they are reachable, does not test application logic, and does not verify that a theoretically exploitable finding is exploitable **on this target**.

A penetration test proves impact. RedFlag estimates likelihood and prices consequence. They answer different questions, and one is not a substitute for the other.

It is also **not a certified audit**. It produces no assurance opinion and satisfies no compliance requirement on its own.

What it is for: a structured, repeatable, low-cost first pass that tells you where to point the expensive people, and gives a deal team something concrete to negotiate against.

2. False positives

RedFlag will sometimes report a problem that is not one.

SOURCE	WHY IT HAPPENS	MITIGATION IN THE PRODUCT
Banner-based service inference	Nmap identifies a service from its banner. Banners can be wrong, spoofed, or belong to a patched build	Nmap findings carry a flat CVSS of **3.5** — an open port is not a vulnerability
DKIM "not found"	Selectors are arbitrary names. RedFlag probes 12 common ones; a custom selector reads as absent	That finding alone is marked `INFERRED` (×0.85) rather than `CONFIRMED`
Shodan CVE lists	Shodan associates CVEs with a host from version fingerprints, not verification. The service may be patched or not affected	Marked `EXTERNAL` evidence (×0.80)
DNS resolver failure	An empty record set reads as "not configured", so a resolver outage can produce a burst of false SPF/DMARC findings	None — check the resolver if DNS findings appear unexpectedly
Uploaded scanner output	RedFlag inherits whatever OpenVAS or ZAP reported, including their false positives	Evidence strength reflects the source, but RedFlag cannot re-verify

The evidence-strength multiplier is the systemic answer. A finding RedFlag merely inferred scores up to 20%

below one that a scanner verified. It does not remove false positives, but it stops them outranking confirmed problems.

3. False negatives — the more dangerous direction

The absence of a finding is not evidence of security. RedFlag reports what it observed from outside, with the data it was given.

It cannot see:

- Anything behind authentication — application flaws, authenticated API surface, admin panels
- Internal systems not reachable from the scan origin
- Application and business logic flaws
- Configuration weaknesses that produce no external signal
- Insider risk, physical security, social engineering exposure
- Anything on a port outside the scanned range (**Fast mode covers only the top 200 ports**)
- Anything a firewall or IDS blocked during the scan

Upload slots for OpenVAS, ZAP and Nuclei exist precisely because those tools reach places RedFlag cannot.

4. Silent degradation — the most important caveat in this document

Every scanner except Nmap **fails silently by design**, so that one dead feed cannot fail a twelve-integration pipeline. The consequence is that **a report produced with broken feeds looks identical to one produced with healthy feeds — just cleaner.**

FEED UNAVAILABLE	WHAT SILENTLY CHANGES	CONSEQUENCE
CISA KEV	Every `is_kev()` returns `False`	**No deal-killer override fires for an actively-exploited CVE.** The most dangerous finding class becomes invisible
Shodan	Exposure stays `PARTNER` / `INTERNAL`	Exposure is 25% of the weighting; the `INTERNAL` – `INTERNET_FACING` gap is 17.5 points of base score. **Every score is understated**
NVD	CVSS silently falls back to **6.5**	A batch of CVEs all score exactly 6.5 — a plausible-looking wrong number, not an error
EPSS	No probability, no promotion	Loses the forward-looking signal
Vulners	Exploit status stays `UNKNOWN`	Partly covered by KEV and EPSS
LeakIX	No breach findings	The "have they already been compromised?" question goes unanswered

Before relying on an assessment, confirm the feeds responded. The notice bar lists the sources that were fused. Diagnostic commands are in TROUBLESHOOTING.md §6; the full failure table is in INTEGRATIONS.md §17.

5. What the scoring model depends on that you may not have supplied

Two of the four scoring dimensions come from inputs that are optional.

Data sensitivity (20% of the weighting) comes from exactly one source: the asset-inventory Excel upload. Without it every finding sits at a neutral 50 — **and two of the four deal-killer override rules become unreachable**, because both require Crown Jewel or Regulated classification.

Exposure (25% of the weighting) is corrected to `INTERNET_FACING` primarily by Shodan. Without Shodan data, `analysis/parser.py`'s conservative `PARTNER`, `INTERNAL` defaults stand.

An assessment run with neither is not wrong, but it is systematically conservative in a way the report does not announce. Say so when you present it.

6. Model assumptions worth challenging

These are defensible choices, not facts. A reader entitled to disagree should know they exist.

ASSUMPTION	WHERE	WHY IT MIGHT BE WRONG
The four weights (0.30 / 0.25 / 0.25 / 0.20)	`config/__init__.py`	Reasoned from SSVC and EPSS methodology, but not empirically validated against breach outcomes
`UNKNOWN` **exploit status scores 30, above** `NO_EXPLOIT` **at 10**	`analysis/triage.py`	Deliberate precaution — absence of evidence treated as more dangerous than evidence of absence. Inflates scores where enrichment simply did not run
Any internet-facing host can pivot to any internal host	`analysis/attack_graph.py`	Correct for a flat network; overstates blast radius in a well-segmented one
The EPSS promotion threshold is 0.50	`scanners/epss_scan.py`	A round number, not a calibrated one
Nmap findings get a flat CVSS of 3.5	`analysis/parser.py`	Reasonable for "a port is open", but the same value for Telnet and for an HTTPS endpoint
Merged cost items take the maximum base	`cost/deduplicator.py`	Conservative by design; overstates cost where a single fix genuinely covers many findings
Cost line items correlate at 0.35	`cost/simulation.py`	A judgement about how much estimating errors cancel. Not measured
Pricing benchmarks reflect a 2026 US market	`config/pricing_benchmarks.yaml`, `day1_cost_catalog.yaml`	Materially wrong for other regions or years. Retunable in YAML
The corporate standard thresholds	`config/corporate_standard.yaml`	One acquirer's bar. Yours may differ — that is what the YAML is for

7. Scope boundaries

BOUNDARY	DETAIL
Single target per run	One hostname or IP. No subnet, no portfolio, no comparison view
Single-host Shodan	`lookup_host()` queries one IP. No `net:` search support
Point in time	Not continuous monitoring. Results age from the moment the scan ends
No authenticated scanning	RedFlag ingests OpenVAS/ZAP output but performs none itself
No compliance mapping	No ISO 27001 / NIST CSF / SOC 2 / GDPR / PCI-DSS control mapping
No secret scanning	Public-repository credential scanning is not implemented
Self-reported maturity	The questionnaire is unverified. It measures what someone said, not what is true
English only	No localisation
Local, single user	No multi-user access, no persistence between sessions, no audit trail
Breach data is one source	LeakIX only. No HIBP, no dark-web monitoring, no paid intelligence

8. Coverage limitations

The 143-test suite covers the deterministic engines thoroughly and the following **not at all**:

- The entire Reflex UI layer, including the `run_scan` pipeline orchestration
- All live network paths and their failure modes
- PDF generation
- The brain — including `export_seed()`, the function that decides what is safe to commit

Full detail: TEST_PLAN.md §7.

9. Conditions for presenting a report

A RedFlag report should be accompanied by the following disclosures.

#	DISCLOSURE
1	The nature of the assessment: a structured first-pass technical review, not a penetration test and not an audit
2	The inputs supplied — which uploads and feeds were used, and whether an asset inventory was provided. The notice bar's source list is the record
3	The areas not examined: authenticated surface, internal systems and application logic
4	The distinction between observed and inferred findings, as recorded in the evidence-strength column

#	DISCLOSURE
5	Costs stated as ranges with their accuracy band. The confidence interval is a statistical statement and is lost if the base figure is quoted alone
6	Confirmation that flagged cost items were reviewed before export, so that no machine-generated figure is presented as reviewed
7	A recommendation for professional penetration testing where the findings warrant it

Related documents

- RESULTS_INTERPRETATION.md — how to read each number
- AUTHORIZED_USE.md — the legal boundaries of running it
- TEST_PLAN.md — what is and is not verified
- INTEGRATIONS.md — per-feed degradation behaviour
- CONFIGURATION.md — retuning the assumptions in §6

PART VI

Legal & Security

Authorised use, licensing and data handling

Authorized Use

The legal and ethical boundaries governing the operation of RedFlag.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

> NOTE
>
> **This document is not legal advice.** It sets out the operating rules I hold this project to and the reasoning behind them. Anyone relying on RedFlag in a commercial engagement should have the terms of that engagement reviewed by qualified counsel.

1. Authorized use only

RedFlag may be run only against systems that you own, or that you have explicit, written, current authorization to assess.

There is no other permitted use. Specifically, the following are **not** authorization:

- The target being publicly reachable on the internet.
- A verbal "go ahead", a corridor conversation, or an assumption of consent.
- Being engaged on a deal that involves the target.
- The target being a prospective acquisition, supplier, competitor or partner.
- Someone else on the deal team saying it is fine.
- Curiosity, research interest, or the belief that a scan is "harmless".

If you cannot point to a written authorization covering **this target**, **this scope** and **this time window**, do not run the scan.

Before every engagement, confirm:

#	CHECK
1	Written authorization exists, signed by someone with authority to grant it
2	It names the specific hosts, domains or IP ranges in scope
3	It covers the dates on which you intend to scan
4	It explicitly permits **active scanning**, not merely passive research
5	You have a copy, stored somewhere you can retrieve it later
6	You know who to contact if a scan causes a problem

2. Legal warning

Unauthorized scanning of computer systems is a **criminal offence** in most jurisdictions, and frequently a civil wrong as well. Depending on where you and the target are located, it may violate:

JURISDICTION	LEGISLATION
United States	**Computer Fraud and Abuse Act (CFAA)**, 18 U.S.C. § 1030, plus state computer-crime statutes
United Kingdom	**Computer Misuse Act 1990**, particularly section 1 (unauthorised access)
European Union	**Directive 2013/40/EU** on attacks against information systems, as implemented in each member state
India	**Information Technology Act 2000**, sections 43 and 66
Canada	**Criminal Code**, section 342.1
Australia	**Criminal Code Act 1995**, Part 10.7

Many of these statutes turn on **access without authorization**, and several do not require proof of damage, intent to cause harm, or that any data was obtained. Port scanning alone has been prosecuted.

The operator bears the responsibility. Not the tool, not its author, not the employer who asked for the assessment. If you run the scan, you are the one who scanned.

Cross-border scanning multiplies the exposure. A scan originating in one country against a target in another may engage the laws of both, and possibly of the countries whose networks the traffic transits. Cloud-hosted targets may sit in a jurisdiction neither party expected. Cloud providers additionally impose their own acceptable-use terms that may require separate notification before security testing.

3. The M&A context specifically

M&A due diligence is an environment where the temptation to scan without authorization is unusually strong — the information is valuable, the timeline is short, and the target's cooperation may be limited or politically awkward. Resist it.

Obtain written authorization from the appropriate deal parties before scanning. Practical guidance:

- **Route it through the deal process.** Scanning authorization normally belongs in the diligence request list or an access agreement, alongside data-room permissions — not in a side channel.
- **Get it from someone with authority over the systems.** The seller's corporate development lead may not be authorized to consent to testing of the target's production infrastructure. The target's CTO or CISO usually is.
- **Watch for third-party infrastructure.** A target's systems frequently run on a cloud provider, a managed hosting company, or a SaaS platform. **The target may not be able to authorize scanning of infrastructure it does not own.** Identify this before scanning, not afterwards.
- **Beware of scope creep.** RedFlag's TLS scanner queries crt.sh and discovers subdomains you did not know existed. Those newly discovered hosts are **not automatically in scope**. Scanning them is a fresh decision requiring fresh authorization.
- **Deal collapse does not retroactively authorize anything.** Nor does it un-authorize a scan that was properly

permitted at the time — but it may trigger obligations to delete the results. Check the authorization's terms.

- **Keep the record.** Retain the authorization for as long as you retain the results, and at least as long as any applicable limitation period. If a question is raised two years later, the document is your answer.

4. Passive versus active — what actually touches the target

Not every RedFlag integration behaves the same way. Understanding which is which matters when scoping an authorization.

Actively touches the target

INTEGRATION	WHAT IT DOES
Nmap	Sends packets directly to the target's ports. This is the loudest thing RedFlag does — it will appear in the target's logs and may trigger IDS alerts
Nuclei	Sends application-layer requests (template probes) directly to the target
TLS scanner	Opens TLS connections to each HTTPS port
DNS scanner	Queries the target's DNS records — low impact, but still directed at the target's infrastructure

These require explicit authorization to perform active scanning. A permission to "look at public information" does not cover them.

Queries third-party indexes about the target

INTEGRATION	WHAT IS SENT, AND TO WHOM
Shodan	The target IP → Shodan
LeakIX	The target domain and IPs → LeakIX
crt.sh	The target domain → the certificate-transparency log
NVD, CISA KEV, EPSS, Vulners	CVE identifiers only — nothing target-identifying

These do not touch the target. They are OSINT queries against public indexes.

But treat them as in-scope activity anyway. Three reasons: they still generate a record of your interest in the target on a third party's systems; an engagement agreement or NDA may restrict them independently of any computer-crime statute; and a target's security team monitoring its own Shodan or certificate-transparency footprint may notice.

Touches nothing

Uploaded OpenVAS XML, ZAP XML, Nuclei JSONL and asset-inventory Excel files are parsed entirely locally. No network activity results.

RedFlag can be run in a fully passive, upload-only mode — leave the target field blank and process staged files. Where authorization for active scanning is not available or not yet granted, this is the correct way to use the tool.

5. Operational responsibilities

- **Scan timing.** `-T4` is aggressive timing. Against a fragile system, a scan can degrade performance or trip alerting. Coordinate with the target's operations team, and prefer **Fast mode** where thoroughness is not essential.
- **Do not expose the application to a network.** RedFlag has no authentication. Anyone who can reach its backend port can launch an active scan against an arbitrary target *from your machine and your IP address*. Keep it on `localhost`.
- **Handle the output as confidential.** A RedFlag report is a map of a company's weaknesses. It is more sensitive than most of the material in a data room, and should be distributed accordingly.
- **Mind what the brain retains.** RedFlag keeps a durable local record of every target assessed — hostnames in `~/RedFlag-Brain/brain.json` and one Markdown note per scan in `~/RedFlag-Brain/vault/Scans/`. Nothing expires. If an engagement agreement limits how long you may retain records about the target, that obligation covers this directory too. Deleting `~/RedFlag-Brain` removes it. See BRAIN_KNOWLEDGE_BASE.md.
- **Never publish an unsanitised brain.** `export_seed()` strips the `targets` map before writing the shipped seed. Do not commit `~/RedFlag-Brain/brain.json` itself, and do not commit vault notes.

6. No warranty — informational only

RedFlag's output is **informational**. It is not a certified security audit, not a penetration test, not an assurance opinion, and not a compliance attestation. It satisfies no regulatory requirement on its own.

Specifically:

- **A clean report is not a certificate of security.** It may reflect broken feeds rather than a secure target. Several scanners fail silently by design — if CISA KEV was unreachable, no actively-exploited CVE will be flagged, and the report will look reassuring. See LIMITATIONS.md §4.
- **Findings may be false positives.** Banner-based inference, third-party CVE association and ingested scanner output can all be wrong.
- **Absent findings prove nothing.** RedFlag sees an external, unauthenticated, point-in-time slice.
- **Costs are estimates.** Benchmark-derived ranges with a stated confidence interval, not quotes.
- **Maturity scores are self-reported and unverified.**
- **Results age immediately.** A scan describes the moment it ran.

Do not represent a RedFlag report as an audit, a penetration test, or a guarantee of any security property. Where the findings warrant it, recommend a professional penetration test — telling you where that spend is worthwhile is precisely what RedFlag is for.

7. Disclaimer of liability

RedFlag is provided **as is**, without warranty of any kind, express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, accuracy and non-infringement.

The authors and contributors accept **no liability** for any claim, damage, loss or other liability arising from the use of this software or from any decision taken on the basis of its output — including, without limitation:

- Any consequence of scanning a system without proper authorization
- Any disruption, degradation or outage caused by a scan
- Any deal decision made in reliance on a RedFlag report
- Any loss arising from a false positive or a missed finding
- Any breach of contract, engagement terms or applicable law by an operator

Responsibility for lawful and authorized use rests entirely with the operator.

This section is reinforced by the MIT licence's own warranty disclaimer, in `LICENSE` at the repository root — see LICENSES_AND_ATTRIBUTION.md §1.

8. Pre-scan checklist

Work through this before every engagement.

#	CHECK	YES
1	Written authorization obtained, signed by someone with authority	[ ]
2	It names the specific hosts, domains or IP ranges in scope	[ ]
3	It covers today's date	[ ]
4	It explicitly permits **active scanning**	[ ]
5	Third-party-hosted infrastructure identified and separately authorized, or excluded	[ ]
6	Timing agreed with the target's operations team	[ ]
7	An escalation contact is available if the scan causes a problem	[ ]
8	A copy of the authorization is stored where you can retrieve it	[ ]
9	Retention obligations for the results — and for the brain — understood	[ ]
10	Everyone receiving the report understands it is not a penetration test	[ ]

If any box is unticked, use upload-only mode — leave the target field blank and process staged scanner output instead. It exercises the full analysis pipeline without touching the target at all.

Related documents

- LIMITATIONS.md — what the output cannot tell you
- SECURITY_AND_PRIVACY.md — exactly what data leaves your machine
- INTEGRATIONS.md — per-integration behaviour and endpoints
- LICENSES_AND_ATTRIBUTION.md — licence position and service terms
- DEPLOYMENT_RUNBOOK.md §5 — operational cautions
- USER_GUIDE.md — running an assessment

Licenses & Attribution

The project's licensing position, a dependency inventory, and the terms attached to every external service RedFlag contacts.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Project licence

RedFlag is released under the MIT licence. The licence text lives in `LICENSE` at the repository root, and `README.md` declares the same.

Why the file matters as much as the declaration. Under copyright law the default position is **all rights reserved**. Source published with a licence *claim* but no licence *text* leaves anyone who forks, reuses or contributes without a grant they can rely on — ambiguous rather than permissive. The committed text is the standard MIT licence:

```
MIT License

Copyright (c) 2026 Adityaa

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

The MIT licence's warranty disclaimer also reinforces AUTHORIZED_USE.md §7, which previously stood alone.

What MIT means here: it is permissive. Anyone may use, modify, distribute and sell the software, commercially, provided the copyright notice and licence text are preserved. There is no copyleft obligation and no requirement to publish modifications.

2. Dependency inventory

From `requirements.txt`, all versions pinned exactly. Licences verified against each project's published metadata.

PACKAGE	VERSION	PURPOSE	LICENCE	NOTES
`reflex`	0.9.5.post2	Web UI framework (compiles to Next.js/React)	Apache-2.0	Pulls a large transitive tree — see §3
`pydantic`	2.13.4	Data validation for `Finding` and the cost models	MIT	
`pandas`	3.0.3	DataFrames for CSV export	BSD-3-Clause	
`fpdf2`	2.8.7	PDF generation	LGPL-2.1-or-later	**See §4**
`openpyxl`	3.1.5	Excel read (asset inventory) and XLSX export	MIT	
`python-nmap`	0.7.1	Nmap wrapper	GPL-3.0-or-later	**See §4**
`shodan`	1.31.0	Shodan API client	BSD-3-Clause	
`dnspython`	2.8.0	DNS queries for SPF/DMARC/DKIM/DNSSEC	ISC	
`cryptography`	48.0.0	X.509 certificate parsing	Apache-2.0 **or** BSD-3-Clause (dual)	
`networkx`	3.5	Attack-graph analytics	BSD-3-Clause	
`PyYAML`	6.0.3	Config loading	MIT	
`python-dotenv`	1.2.2	`.env` loading	BSD-3-Clause	
`requests`	2.34.2	HTTP for NVD, KEV, Vulners, crt.sh, LeakIX	Apache-2.0	
`pytest`	9.0.3	Test framework	MIT	Development only

The licences above were read from each project's published package metadata. The table covers the **direct** dependencies only; to enumerate the transitive tree as well, and to read licences from the installed distributions rather than from upstream metadata:

```
pip install pip-licenses
pip-licenses --format=markdown --with-urls
```

3. Transitive dependencies

`requirements.txt` lists 14 direct dependencies. The installed environment contains many more, mostly pulled in by Reflex (FastAPI, Starlette, Uvicorn, Pydantic-core, Rich, Typer, Alembic, SQLModel and others) and by pandas (NumPy, python-dateutil, pytz).

Reflex additionally downloads **Node.js and an npm dependency tree** into `.web/` on first run, including Next.js and React. Those are not Python packages and do not appear in any pip listing, but they are third-party code executing on the developer's machine.

A machine-readable SBOM can be produced from any installed environment, in CycloneDX format:

```
pip install cyclonedx-bom
cyclonedx-py environment -o sbom.json
```

4. Copyleft flags

Two direct dependencies carry copyleft terms. **Neither is a problem for RedFlag as it stands**, but both matter if the project is ever distributed as a binary or bundled into a proprietary product.

python-nmap — GPL-3.0-or-later

The strongest obligation in the tree. GPL-3.0 requires that derivative works, when distributed, be licensed under GPL-3.0 and their source made available.

Current position: RedFlag is distributed as source under a permissive licence, and users install `python-nmap` themselves via pip. This is the ordinary "aggregation with an interpreted dependency" arrangement and does not trigger the copyleft obligation.

When it would matter: bundling `python-nmap` into a distributed binary, container image, or closed-source product would raise a genuine question about whether the combined work must be GPL-3.0. Take advice before doing that.

Note also: the **Nmap binary itself** is not a pip dependency — the user installs it separately. Nmap is distributed under the Nmap Public Source License, which is GPL-derived and **explicitly restricts redistribution and commercial embedding**. Nmap's own FAQ is clear that commercial redistribution requires a separate licence from Nmap Software LLC. Do not bundle the Nmap binary with RedFlag.

fpdf2 — LGPL-2.1-or-later

Weaker copyleft. The LGPL permits use in a work under a different licence provided the library itself remains replaceable and its own source is available.

Current position: installed via pip and imported at runtime. Fine.

When it would matter: static bundling into a distributed binary would require preserving the user's ability to replace the library. Alternatives with permissive licences exist `reportlab` — BSD) if this ever becomes a constraint.

5. External services

Every service RedFlag contacts, what it sends, and the terms attached.

SERVICE	WHAT IS SENT	ATTRIBUTION / TERMS
Nmap (local binary)	Packets to the target	Nmap Public Source License. Free for internal use; commercial redistribution requires a licence

SERVICE	WHAT IS SENT	ATTRIBUTION / TERMS
Nuclei (local binary)	Requests to the target	MIT (ProjectDiscovery)
Shodan	Target IP	Commercial API. Governed by Shodan's Terms of Service. Free tier available; **1 credit per IP**
NVD (NIST)	CVE IDs	Public US government data, no licence restriction. Rate-limited to 5 req/30 s unauthenticated. NVD terms
CISA KEV	Nothing — file download	US government public data, free to use and redistribute
EPSS (FIRST.org)	CVE IDs	Free. FIRST asks that EPSS be **cited by name** when scores are published — RedFlag does this in the UI and this documentation
Vulners	CVE IDs + API key	Commercial API with a free tier. Vulners terms
crt.sh	Target domain	Free public certificate-transparency service operated by Sectigo. No formal terms; use courteously
LeakIX	Target domain and up to 2 IPs	Free API. LeakIX terms
MITRE ATT&CK	Nothing — knowledge encoded locally	ATT&CK® is a registered trademark of The MITRE Corporation. Used under MITRE's terms of use, which permit reuse with attribution. RedFlag links every technique to its attack.mitre.org page
Obsidian	Nothing	The brain writes a plain-Markdown vault. Obsidian itself is not a dependency and is not bundled

Privacy summary. Only three services ever receive target-identifying data: Shodan (the IP), LeakIX (the domain and IPs), and crt.sh (the domain). Everything else receives only CVE identifiers or nothing at all. Consolidated view: SECURITY_AND_PRIVACY.md §2.

6. Attributions

RedFlag builds on public security data and frameworks. Attribution where it is due:

- **MITRE ATT&CK®** — the technique taxonomy underpinning the attacker-brain. ATT&CK is a registered trademark of The MITRE Corporation. RedFlag encodes technique mappings locally and links each one to its canonical page.
- **CISA Known Exploited Vulnerabilities catalog** — the authoritative source for `ACTIVE_EXPLOITATION` status, published free by the US Cybersecurity and Infrastructure Security Agency.
- **EPSS**, by the FIRST.org EPSS Special Interest Group — the exploitation-probability model behind RedFlag's forward-looking scoring.
- **SSVC** (Stakeholder-Specific Vulnerability Categorization), CISA and Carnegie Mellon SEI — the methodology the weighting scheme is aligned with.
- **NVD**, NIST — CVSS base scores.
- **Nmap**, by Gordon Lyon and the Nmap Project.
- **Nuclei**, by ProjectDiscovery.

- **OWASP ZAP** and **OpenVAS/Greenbone** — supported through their report formats.
- **Reflex** — the UI framework.

Industry sources cited in `config/day1_blueprint.yaml` for the connectivity models: EY and Deloitte (M&A clean rooms), VMware (Day-0 VDI productivity), Microsoft Entra (identity federation and B2B trust options), and Cybersecurity Insiders (ZTNA in M&A). Pricing sources for `config/day1_cost_catalog.yaml` are cited per line item in that file.

7. Licence compliance checklist

#	ITEM	STATUS
1	A `LICENSE` file exists in the repository root	Yes MIT — §1
2	The README's licence claim matches the actual licence	Yes Both state MIT
3	Copyleft dependencies identified	Yes `python-nmap` GPL-3.0, `fpdf2` LGPL-2.1
4	No copyleft code is statically bundled or redistributed	Yes pip-installed at runtime
5	The Nmap binary is not redistributed	Yes Installed separately by the user
6	EPSS is cited by name where scores are shown	Yes
7	MITRE ATT&CK is attributed and linked	Yes
8	No API key or secret is committed	Yes `.env` is git-ignored
9	Direct dependency licences recorded	Yes All 14 — §2
10	Transitive tree and SBOM reproducible on demand	Yes Commands in §2 and §3

Related documents

- AUTHORIZED_USE.md — the legal boundaries of operating the tool
- SECURITY_AND_PRIVACY.md — the data-egress view
- INTEGRATIONS.md — technical detail per service
- KNOWN_ISSUES_AND_BACKLOG.md §1.1 — the missing LICENSE

Security & Privacy

What RedFlag stores, what leaves your machine, and how secrets are handled.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

1. Data handling posture

RedFlag processes only the data you explicitly give it. It has no telemetry, no analytics, no crash reporting, no update check, and no account system. It never phones home.

ASPECT	REALITY
Telemetry	**None**
Analytics	**None**
Cloud storage	**None** — everything is local
Database	**None**
Authentication	**None** — assumes a trusted local machine
Network binding	`localhost` only
Multi-user	Not supported

Where data is written

PATH	CONTENTS	SENSITIVITY	VERSION-CONTROLLED
`%TEMP%/redflag_scans/`	Nmap XML — hosts, ports, service banners	**High** — a map of the target's attack surface	No — outside the repository
`~/RedFlag-Brain/brain.json`	Aggregate knowledge **plus a `targets` map naming every host assessed**	**High**	No — outside the repository
`~/RedFlag-Brain/vault/Scans/`	One Markdown note per scan, **named after the target**	**High**	No
`~/RedFlag-Brain/vault/{Techniques,CVEs}/`	Technique and CVE notes — no target identities	Low	No
`.env`	API keys	**Secret**	No — git-ignored
`.web/`, `.states/`, `uploaded_files/`	Reflex build output, runtime state, upload staging	Low–medium	No — git-ignored
Browser downloads	Exported CSV and PDF reports	**High**	No
`analysis/brain_seed/brain.json`	Aggregate knowledge, `targets` **stripped**	Low	**Yes — committed**

Both runtime paths sit outside the repository deliberately. The brain is long-term memory that must survive a `git clean`, and — for both paths — a write inside the worktree trips Reflex's dev file-watcher, which

hot-reloads the backend and destroys scan state mid-run.

2. Egress table — exactly what leaves the machine

DESTINATION	DATA SENT	WHEN	KEY REQUIRED
The target itself	Nmap packets; Nuclei template requests; TLS handshakes; DNS queries	Live scan with a target	No
Shodan	The target **IP address**	Live lookup — *skipped entirely if a Shodan JSON is staged*	Yes (optional)
LeakIX	The target **domain** and up to **2 IPs**	Live scan with a target	No
crt.sh	The target **domain**	TLS scan	No
NVD (NIST)	**CVE IDs only**	CVSS enrichment	No
CISA KEV	**Nothing** — a public file download	KEV lookup, threat-intel refresh	No
EPSS (FIRST.org)	**CVE IDs only**, batched 100 per request	EPSS enrichment	No
Vulners API	**CVE IDs** and your API key	Exploit confirmation	Yes (optional)
Vulners (via the Nmap NSE script)	Service versions, from the NSE script during the scan	Only if `vulners.nse` is installed	Optional

Only three services ever receive target-identifying data: Shodan, LeakIX and crt.sh. Everything else receives CVE identifiers or nothing.

Running with minimal egress

- **Do not supply a Shodan key** and upload a Shodan JSON instead — the upload takes priority and the API is never called.
- **Use upload-only mode.** Leave the target field blank and process staged OpenVAS/ZAP/Nuclei/ Excel files. **Nothing leaves the machine at all** — those parsers are entirely local. The only network activity would be CVE-only enrichment (NVD, KEV, EPSS), which sends no target-identifying data.
- **Work offline entirely.** Every feed degrades silently, so the pipeline completes — but read LIMITATIONS.md §4 first, because a report built with dead feeds looks identical to one built with live ones, only cleaner.

3. Secrets management

RULE	DETAIL
Where secrets live	`.env` in the repository root, and nowhere else
Git status	`.env` is in `.gitignore`. Verify with `git check-ignore -v .env`

RULE	DETAIL
Committed template	`.env.example` — placeholders only, no real values
In documentation	Never. No document in this set contains a key, and none ever should
In logs	Keys are never printed. Note that `VULNERS_API_KEY` **is** passed on the Nmap command line as `--script-args ...,api_key=<key>`, so it may appear in process listings on a shared machine

Provisioning:

```
copy .env.example .env      # Windows
```

```
cp .env.example .env        # macOS / Linux
```

Both keys are optional — see ACCESS_AND_CREDENTIALS.md for the inventory, rotation procedures, and the transfer checklist.

4. Secrets in the repository

No API key, token or credential appears in any tracked file. `.env` is git-ignored and `.env.example` contains only placeholders. A pattern scan across every tracked file returns no credential material, and the two API keys RedFlag can use are both optional — the tool runs a complete assessment with neither.

The design point behind this is that RedFlag never *needs* a shared secret. Where a key is used it belongs to one operator, is read from a local `.env` at runtime, and is never written to a scan output, a report, a log line or the knowledge base. Rotation procedures for both services are in ACCESS_AND_CREDENTIALS.md §4; the safest position for a new operator is to issue their own keys rather than inherit any.

5. What is committed versus what stays local

Committed to the repository

- All source code, configuration YAML, and this documentation set
- `.env.example` — placeholders only
- Test fixtures — synthetic data, no real target information
- **`analysis/brain_seed/brain.json`** — the only brain data in version control

Never committed

- `.env` — real API keys

- `~/RedFlag-Brain/` — the local brain, including target identities
- `%TEMP%/redflag_scans/` — Nmap XML
- Exported CSV and PDF reports
- `.web/`, `.states/`, `venv/`, `uploaded_files/`

How the shipped seed is made safe

`BrainMemory.export_seed()` strips **exactly one field** before writing the seed:

```
data = json.loads(json.dumps(self.data))   # deep copy
data["targets"] = {}                        # drop who-was-scanned
```

`targets` is the only place in `brain.json` where a scanned host or IP is named. What remains is aggregate pattern knowledge — technique, CVE, service, port, path and tier prevalence, plus the KEV list — which reveals nothing about who was assessed.

> NOTE
>
> **This function is the entire boundary between "local memory" and "safe to publish".** If you add a field to `brain.json` that could identify a target, you must strip it here too. Six lines, high consequence, and currently **untested** — see TEST_PLAN.md §7.

Note that the *local* brain is not sanitised. `~/RedFlag-Brain/brain.json` and the vault's `Scans/` notes do name targets. They are outside version control, but they are real records.

.gitignore and the documentation

The `.gitignore` carries a blanket `*.md` rule with explicit negations for the published documentation, `CONTRIBUTING.md`, `SECURITY.md`, `CODE_OF_CONDUCT.md` and `CHANGELOG.md`. Agent and scratch notes remain ignored. Verify with:

```
git check-ignore -v docs/README.md
git status --short docs/
```

6. Data retention

DATA	RETENTION	HOW TO DELETE
Nmap XML	Until the OS clears its temp directory	`Remove-Item -Recurse -Force $env:TEMP\redflag_scans`
The brain	**Indefinite — nothing expires or is pruned**	`Remove-Item -Recurse -Force $HOME\RedFlag-Brain`
Exported reports	Wherever the browser saved them	Delete manually

DATA	RETENTION	HOW TO DELETE
In-session findings	Lost on restart — nothing persists between sessions	Automatic

The brain is the retention risk worth understanding. It accumulates a durable, permanent record of every target you have assessed. If you assess third-party targets under an engagement agreement, that agreement may govern how long you may retain such records — and that obligation covers `~/RedFlag-Brain`. Deleting the directory removes it completely; the next run re-bootstraps from the sanitised shipped seed.

7. Application security posture

RedFlag is a local single-user tool and is built accordingly. Be aware of the following.

CONSIDERATION	POSITION
No authentication	Anyone who can reach the backend port can drive the application. **Never expose it to a network** — they could launch an active scan against an arbitrary target from your machine and your IP
Uploaded file parsing	XML is parsed with the standard library's `xml.etree.ElementTree`, which does **not** expand external entities by default; Excel via openpyxl; JSON via the standard library. All parse untrusted input, so only upload files from sources you trust
Subprocess execution	`run_nuclei_scan()` invokes an external binary. The target string is passed as an argument, not through a shell
Broad exception handling	`run_scan` wraps each scanner in `except Exception: pass`. This is deliberate resilience, but it means a genuine fault is swallowed silently
Output sensitivity	A RedFlag report is a map of a company's weaknesses — more sensitive than most data-room material. Distribute accordingly
Dependency currency	All dependencies are pinned. Pinning gives reproducibility but means security updates require an explicit bump. Review periodically (`pip list --outdated`)

To report a vulnerability **in RedFlag itself**, see SECURITY.md.

8. Privacy checklist for an engagement

#	CHECK	YES
1	Written authorization obtained — see AUTHORIZED_USE.md	[ ]
2	You know which services will receive target-identifying data (§2)	[ ]
3	The engagement permits OSINT queries to Shodan, LeakIX and crt.sh	[ ]
4	Retention obligations for the results are understood	[ ]
5	Retention obligations for **the brain** are understood (§6)	[ ]

#	CHECK	YES
6	Exported reports will be stored and shared securely	[ ]
7	The brain will be cleared if the engagement requires it	[ ]
8	`.env` is confirmed git-ignored and contains no shared key	[ ]

Related documents

- AUTHORIZED_USE.md — the legal boundaries
- ACCESS_AND_CREDENTIALS.md — key inventory and rotation
- INTEGRATIONS.md — per-service technical detail
- BRAIN_KNOWLEDGE_BASE.md — the store and its sanitisation
- SECURITY.md — reporting a vulnerability in RedFlag

PART VII

Process & History

Version history, forward plan and decision records

Changelog

All notable changes to RedFlag are documented here.

The format is based on Keep a Changelog, and this project aims to follow Semantic Versioning.

> NOTE
>
> **Note on version history.** The repository carries **no git tags and no formal releases**. The entries below are reconstructed from the commit history on `main` and grouped into dated milestones. Only the `1.0.0 — Handover` entry represents a deliberate version boundary; the earlier headings are development milestones, not published releases.

[Unreleased]

Added

- Complete documentation set under `docs/` — handover, technical, operations, user, testing, legal and process groups, plus eight Architecture Decision Records.
- Root community files: `CONTRIBUTING.md`, `SECURITY.md`, `CODE_OF_CONDUCT.md`, this changelog.
- `tools/` — `mdpdf.py` and `build_docs_pdf.py`, which render `docs/` into a bookmarked PDF and one PDF per document.
- Module docstrings and explanatory comments across `analysis/`, `scanners/`, `reports/` and `config/`. No behaviour change.
- `tests/fixtures/mock_nuclei.jsonl`, previously present on disk but absent from version control.
- `LICENSE` — the MIT text the README had always declared but which was never committed.

Fixed

- **`.gitignore` no longer excludes the documentation.** A blanket `*.md` rule silently excluded every Markdown file; `README.md` predated the rule and stayed tracked, which masked the effect. Explicit negations now version `docs/` and the community files while leaving agent and scratch notes local.
- `README.md` stated 128 tests in three places. The suite contains **143**.
- `README.md` listed three deal-killer override rules; `analysis/triage.py` implements **four**. The missing rule is the manual analyst flag triggered by `override_reason`.
- `README.md`'s mock-fixture table omitted `tests/fixtures/mock_nuclei.jsonl`.

Removed

- `lib/` — 2.7 MB of vendored JavaScript from the superseded pyvis attack-graph implementation, with no remaining code references.

[1.0.0] — 2026-07-27 — Handover

The state of the project at handover: a complete M&A cybersecurity due-diligence platform with fourteen integrations, five analysis engines, a two-bucket cost model, and 143 passing tests.

Summary of capabilities

- **Collection** — Nmap, Shodan, Nuclei, OpenVAS, ZAP, Vulners NSE, DNS, TLS/crt.sh, LeakIX, Excel asset inventory
- **Enrichment** — CISA KEV, NVD CVSS, Vulners API, FIRST.org EPSS
- **Analysis** — weighted risk scoring, 7-domain maturity assessment, Day-1 Safe Harbor Blueprint, MITRE ATT&CK attacker-brain, networkx attack graph, self-improving knowledge base
- **Costing** — remediation and integration budgets, low/base/high scenarios, CapEx/OpEx split, variance-based 80% confidence interval, vendor-quote overrides
- **Output** — deterministic narrative, CSV, three PDF reports
- **Interface** — Reflex web application, nine routed pages

[0.9.0] — 2026-07-02 — Day-1 integration budget

Added

- **Day-1 integration budget** (`cost/day1_costing.py`, `config/day1_cost_catalog.yaml`) — prices the recommended connectivity model as a separate `integration` cost bucket, with sourced 2026 benchmark pricing citing a real source per line item.
- **Ladder costing** — `cost_all_models()` prices every rung of the connectivity ladder, so the UI can show what integrating faster would cost.
- **Estimate accuracy readout** — a variance-based 80% confidence interval (P10/P50/P90) in `cost/simulation.py`, replacing naive band-averaging. Triangular distributions widened by confidence, aggregated with partial correlation.
- **Vendor-quote overrides** — a firm quote replaces the benchmark for a line item, pins it to `HIGH` confidence, collapses its range to a single figure, and clears its high-variance flag.
- Headcount input driving per-user integration costs; an assumed headcount widens the accuracy band by 15%.

Changed

- `CostRollup` gained `remediation_total`, `integration_total`, `accuracy_band_pct`, `accuracy_pct` and the `ci_low`, `ci_p50`, `ci_high` interval.
- `CostLineItem` gained a `bucket` field (`"remediation"` | `"integration"`).
- `RemediationCategory` gained an `INTEGRATION` member.

Commits: `764d9b4`, `25ebe58`, `a0f982a`

[0.8.0] — 2026-06-30 — Nuclei, EPSS and graph analytics

Added

- **Nuclei integration** (`scanners/nuclei_scan.py`) — runs the local binary if present, or ingests uploaded JSONL. Correlation-merges into the Nmap layer. Degrades to `[]` when absent.
- **EPSS scoring** (`scanners/epss_scan.py`) — FIRST.org exploitation probability, batched 100 CVEs per request, no API key. A score of ≥ 0.50 promotes an unknown exploit status to `PUBLIC_EXPLOIT` (ADR-0007).
- **Attack-graph analytics** (`analysis/attack_graph.py`) — networkx model of the estate producing chokepoints ranked by removal impact, blast radius, and shortest paths to crown-jewel data.
- `epss_score` and `epss_percentile` fields on `Finding`.
- A fifth upload slot for Nuclei JSONL.

Changed

- `networkx` added to `requirements.txt`.
- README data-flow diagram updated to include Nuclei, EPSS and the graph.

Commits: `55b5150`, `2180e2c`

[0.7.0] — 2026-06-29 — Migration to Reflex

The largest structural change in the project's history.

Added

- **Reflex UI** (`redflag_ui/`) — nine routed pages, `RedFlagState` with view-model flattening, `assets/redflag.css` implementing the "Executive Editorial / emerald" design.
- **Sanitised brain seed** (`analysis/brain_seed/brain.json`) — a fresh clone bootstraps pre-loaded rather than empty. `export_seed()` strips the `targets` map so publishing it never reveals who was scanned.

Removed

- **Streamlit `app.py`** and all Streamlit dependencies (`streamlit`, `plotly`, `pyvis`).
- The Streamlit UI test suite — **not replaced**, leaving `redflag_ui/` without coverage.

Notes

- **The migration required zero changes to any engine.** `analysis/`, `cost/`, `narrative/`, `reports/` and `scanners/` were untouched — see ADR-0004.
- Introduced the constraint that runtime writes must stay **outside** the worktree: Nmap output to

`%TEMP%/redflag_scans`, the brain to `~/RedFlag-Brain`. A write inside the tree trips Reflex's file-watcher and resets backend state mid-scan.

Commits: `23774bc`, `bf0d606`, `ece88d9`, `64c31c7`

[0.6.0] — 2026-06-24 — Day-1 Safe Harbor Blueprint

Added

- **Day-1 engine** (`analysis/day1.py`, `config/day1_blueprint.yaml`):
  - A four-rung connectivity ladder — Isolate → Broker → Federate → Integrate — with cited industry sources per model.
  - Tier entry gates with `no_finding` and `maturity_min` criterion types. The engine recommends the **highest tier whose gate passes**.
  - Three review pillars (Identity Sources, Network Boundaries, Remote Access Pathways) with RAG status and status-keyed recommendations.
  - A P0–P3 fix-first roadmap driven by ordered, first-match phase rules.
  - Remote-access pathway detection across 16 service names and 15 ports.
- `build_day1_narrative()` in the narrative engine.
- A Day-1 PDF report section.

Commit: `f6329fd`

[0.5.0] — 2026-06-08 to 2026-06-15 — Attack path and UI iteration

Added

- **Attacker-brain** (`analysis/attack_brain.py`) — an offline MITRE ATT&CK expert system mapping findings to techniques, chaining them into a kill-chain, and rendering a radial mind-map as SVG. No LLM, no network (ADR-0001).
- **Brain memory** (`analysis/brain_memory.py`) — a persistent knowledge base that recalls before it learns, and writes an Obsidian-compatible vault (ADR-0005).
- Attack Path tab.

Changed

- Design System v5 (ULTRAVIOLET) theme revamp.
- Fixed a raw-HTML rendering bug on the Findings view; removed deprecated API usage.
- Corrected the Python version floor; dropped unused dependencies.

Commits: `35206a3`, `6fb3b11`, `22add93`, `b8222bb`, `a53bf12`

[0.4.0] — 2026-06-05 — Phase 1 scanners

Added

- **DNS/email security scanner** — SPF, DMARC, DKIM (12 common selectors) and DNSSEC checks, each gap becoming a scored finding.
- **TLS certificate health scanner** — expiry, weak TLS versions, and crt.sh certificate- transparency subdomain discovery.
- **Breach exposure scanner** — LeakIX domain and host lookup, classifying credential leaks and exposed databases as actively exploited.
- **What-If simulator** for the cost model.

Commit: `774e49d`

[0.3.0] — 2026-05-29 — Maturity, cost and narrative engines

Added

- **Maturity assessment** (`analysis/maturity.py`) — 23 questions across 7 domains, weighted scoring, with only answered questions counting toward a domain score.
- **Standards comparison** (`analysis/standards_compare.py`) — gap report against a configurable corporate acquisition standard.
- **Cost engine** (`cost/`) — catalogue → estimator → deduplicator → scenario engine → rollup, with low/base/high triples, CapEx/OpEx split and a human-review gate.
- **Narrative engine** (`narrative/`) — deterministic, YAML-backed template blocks (ADR-0002).
- Config YAMLs: `maturity_questions`, `corporate_standard`, `pricing_benchmarks`, `remediation_catalog`, `narrative_blocks`.

Commit: `51cf3cd`

[0.2.0] — 2026-05-25 to 2026-05-28 — Scanner pipeline

Added

- **Vulners NSE** parsing from Nmap XML, with optional API-key support.
- **OpenVAS/GVM XML** and **OWASP ZAP XML** parsing with correlation merge into the Nmap layer (ADR-0008).
- **PDF report** generation via fpdf2.
- **Configuration centralisation** into `config/`.

- Comprehensive README.

Fixed

- `FPDFUnicodeEncodingException` — non-ASCII characters are now stripped before PDF output.
- Vulners NSE pipeline status now distinguishes "installed" from "not installed".

Changed

- `CLAUDE.md` removed from tracking.

Commits: `eb4cecc`, `f4052fa`, `4bdbe07`, `dbd2607`, `4307768`, `a8a844f`

[0.1.0] — 2026-05-25 — Initial commit

Added

- The `Finding` Pydantic model and all enums (`analysis/schema.py`).
- Nmap XML parsing (`analysis/parser.py`).
- **Weighted risk scoring** with an evidence-strength multiplier and deal-killer overrides (`analysis/triage.py`) — ADR-0006.
- Shodan integration with CISA KEV cross-reference and NVD CVSS lookup.
- CSV export.

Commit: `72bb001`

Related documents

- docs/handover/PROJECT_REPORT.md §9 — the timeline in narrative form
- docs/process/ROADMAP.md — what comes next
- docs/process/adr/ — why the big changes were made
- docs/handover/KNOWN_ISSUES_AND_BACKLOG.md — outstanding work

Roadmap

Where RedFlag could go if the project continues.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

> NOTE
>
> **Scope.** This document records **direction** — the capabilities the product could acquire and the rationale for each. Concrete tasks, including defects and technical debt with file paths, are recorded separately in KNOWN_ISSUES_AND_BACKLOG.md.

Completed

Everything below shipped and is in the handover build.

CAPABILITY	LANDED
Multi-scanner fusion with correlation merge (Nmap, Shodan, OpenVAS, ZAP, Nuclei, Vulners)	0.2.0 – 0.8.0
Weighted risk scoring with evidence-strength multiplier and deal-killer overrides	0.1.0
Maturity assessment: 23 questions, 7 domains, corporate-standard comparison	0.3.0
Cost engine: catalogue, dedup, low/base/high scenarios, CapEx/OpEx, review gate	0.3.0
Deterministic YAML narrative engine	0.3.0
DNS, TLS/crt.sh and breach scanners	0.4.0
Attacker-brain: MITRE ATT&CK kill-chain and radial mind-map	0.5.0
Self-improving offline knowledge base with an Obsidian vault	0.5.0
Day-1 Safe Harbor Blueprint: ladder, gates, pillars, P0–P3 roadmap	0.6.0
Reflex UI migration with zero engine changes	0.7.0
Sanitised brain seed so fresh clones start pre-loaded	0.7.0
EPSS exploitation probability with status promotion	0.8.0
networkx attack-graph: chokepoints, blast radius, crown-jewel paths	0.8.0
Day-1 integration budget with variance-based confidence interval and vendor quotes	0.9.0
Complete documentation set	1.0.0

Near term — finish what is started

Small, well-defined, and each closes a visible gap. Roughly a week in total.

1. Repository hygiene (hours)

Add the missing `LICENSE`; commit `tests/fixtures/mock_nuclei.jsonl`; correct the README's test count from 128 to 143; delete the dead `analysis/graph_builder.py` and root `config.py`, porting the scoring-weight rationale comments into `config/__init__.py` first.

2. Integration budget in the cost PDF (days)

`reports/pdf_report.generate_cost_section()` prices remediation only. The `CostRollup` already carries `integration_total`, `accuracy_pct` and the P10/P50/P90 interval — this is a reporting gap, not a modelling one, and it is the one place the UI is ahead of the exports.

3. Log swallowed scanner exceptions (hours)

`run_scan` wraps each scanner in `except Exception: pass`. The resilience is right; the silence is not. Log the exception and surface a "some sources were unavailable" hint in the notice bar. This also mitigates the biggest interpretive risk in the product — a clean report produced by dead feeds looking identical to a clean report produced by live ones.

4. An NVD API key (hours)

Free to obtain and raises the rate limit substantially above the unauthenticated 5 requests / 30 s that currently makes CVSS enrichment the pipeline's slowest step. **The highest value-per-hour item on this list**, and it stays inside ADR-0003.

5. Tests for buildview() and exportseed() (days)

`build_view()` is already a pure module-level function; a handful of tests would cover a large surface. `export_seed()` is the six-line boundary between local memory and published data, and it is currently unguarded — assert that `targets` is stripped and that nothing else names a host.

6. Continuous integration (hours)

A GitHub Actions workflow running `pytest tests/ -v` on push. The suite needs no network, no secrets and no binaries, so it works in a stock runner with only `pip install -r requirements.txt`.

Medium term — the M&A product

These are the features that would most change what RedFlag is worth to a deal team.

7. Compliance gap mapping — highest business value

Map findings and maturity domains onto ISO 27001, NIST CSF, SOC 2, GDPR and PCI-DSS control families.

Why it matters: deal teams and their counsel think in frameworks. "Seven findings map to ISO 27001 A.9 Access Control, three of which are deal-blocking" is a sentence that goes straight into a diligence report. It also turns the maturity questionnaire's seven domains from an internal construct into something an auditor recognises.

Why it fits: the maturity domains already align closely with control families. This is largely a new YAML plus a view — the same pattern as every other config-driven feature.

8. Multi-target comparison

Assess several targets and compare risk, maturity and cost side by side.

Why it matters: an acquirer usually evaluates a shortlist. Comparative risk is a different and often more useful question than absolute risk.

Why it is hard: **the only item on this roadmap that forces an architectural change.** RedFlag holds one assessment in memory per session and persists nothing. This needs a real store, an identity for each assessment, and a decision about how long results are retained — which immediately engages the retention concerns in SECURITY_AND_PRIVACY.md §6. Do this last.

9. Secret scanning

Integrate trufflehog to scan the target's public repositories for leaked credentials.

Why it matters: leaked credentials in a public repository are among the highest-severity, most directly exploitable findings possible — and squarely a deal-killer class.

Why it fits: it slots into the existing scanner contract exactly — produce `Finding` objects, never raise. A `SECRETS` member on `ScannerSource` and a new module.

Caution: scanning a target's public repositories is OSINT, but it should still be inside the engagement's scope. See AUTHORIZED_USE.md §4.

10. Multi-host Shodan

`lookup_host()` handles one IP. Shodan's search API supports `net:` queries for a whole range.

Caution: credit cost scales per IP, which pushes against ADR-0003. Make it opt-in with a clear credit estimate before it runs.

11. UI test coverage

`redflag_ui/` has no tests at all — the largest single gap in the suite. Item 5 covers `build_view()`; this is the rest: upload handling, the `run_scan` orchestration, and page rendering.

Longer term — direction

12. Optional LLM narrative layer

The one roadmap item that touches a settled decision. If built, it must be **additive and clearly labelled** — never replacing or altering a deterministic output. Free-tier or local only, per ADR-0003.

Read ADR-0001 before starting. The reasons for excluding an LLM — reproducibility, auditability and the fact that these documents inform a commercial decision — have not changed. A generative layer over the brain's retrieval is the defensible version; regenerating the executive summary is not.

13. Docker Compose

One-command startup. Complicated by two things: Nmap's binary requirement (and the Nmap Public Source License's restrictions on redistribution — see LICENSES_AND_ATTRIBUTION.md §4), and Reflex's Node frontend build. Worth doing for reproducibility, but neither trivial nor purely technical.

14. Continuous monitoring

RedFlag is point-in-time. A deal takes months, and the target's posture moves. Scheduled re-scans with change detection — "*three new internet-facing services since the last assessment*" — would be genuinely valuable during a live deal, and would give the brain a much richer corpus.

15. Remediation verification

Re-scan after remediation and prove which findings are actually closed. Turns RedFlag from a diligence tool into a post-close integration tool, and gives the P0–P3 roadmap a feedback loop.

16. Recency weighting in the brain

Today the brain never forgets and never ages knowledge — a CVE seen a year ago carries the same weight as one seen last week. Whether that is right is an open design question flagged in ADR-0005. Worth answering before the corpus gets large enough that it matters.

Suggested priority

PRIORITY	ITEMS	RATIONALE
Do first	1, 3, 4, 6	Hours each; each removes a real defect or risk
Do next	2, 5	Close the two gaps where the product is ahead of its exports and its tests
Highest value	7 — compliance mapping	Changes what the tool is worth to its actual audience
High value, high cost	8 — multi-target	Forces persistence; plan it properly
Opportunistic	9, 10, 11	Well-scoped, independent, each fits an existing pattern
Only with intent	12 — LLM layer	Re-read ADR-0001 first
Someday	13, 14, 15, 16	Direction rather than plan

Related documents

- KNOWN_ISSUES_AND_BACKLOG.md — the concrete task list
- adr/ — the decisions several of these items would touch
- CHANGELOG.md — what has already shipped
- LIMITATIONS.md — the gaps several of these would close
- TEST_PLAN.md §7 — the coverage gaps behind items 5 and 11

Architecture Decision Records

The deliberate design choices behind RedFlag, and the reasoning that produced them.

Last updated: 2026-07-27 · Owner: Adi · Status: Handover

What an ADR is

An Architecture Decision Record captures a single significant decision: the situation that forced it, the choice made, what that choice costs as well as what it buys, and what else was considered.

The value is not the decision — that is visible in the code. The value is the **reasoning**, which is not. Six months later, when someone asks "why doesn't this use an LLM?", the answer should be a document rather than a guess.

An ADR is **immutable once accepted**. If a decision is later reversed, write a new ADR that supersedes it rather than editing the original. The history of a system's thinking is part of its documentation.

Each record uses the standard template: **Title · Status · Context · Decision · Consequences · Alternatives considered**.

Index

#	DECISION	STATUS	ONE-LINE RATIONALE
0001	No LLM in the core	Accepted	Deal documents must be reproducible and auditable; a stochastic generator is neither
0002	Deterministic YAML narrative engine	Accepted	Analysts can read and edit the exact sentences the tool will print
0003	Entirely free — no paid API	Accepted	A per-scan cost changes how a tool is used, not just what it costs
0004	Reflex as the UI framework	Accepted	A routed multi-page app in pure Python; Streamlit's rerun model could not carry it
0005	The brain accumulates, never trains	Accepted	Retrieval over a growing corpus compounds with no GPU, no drift, and no cost
0006	Evidence-strength multiplier	Accepted	A confirmed finding should outrank a banner-only inference at equal severity
0007	EPSS can promote exploit status	Accepted	Makes the risk model forward-looking rather than purely retrospective
0008	Uploads correlate into the Nmap layer	Accepted	Preserves port-level ground truth while raising confidence — one list, not five

The through-line

Read together, these eight records describe one system of belief:

Determinism over sophistication. ADRs 0001 and 0002 give up generative fluency in exchange for output that is identical every time and defensible in a deal room. That is the right trade when the reader is a lawyer.

Free as a design constraint, not a compromise. ADR 0003 rules out paid APIs — which forced the discovery that CISA KEV, EPSS, NVD, crt.sh and LeakIX are all free and all good. The constraint improved the product.

Evidence quality is first-class. ADRs 0006, 0007 and 0008 all concern how confident the tool should be about what it thinks it knows: corroboration raises a score, prediction can promote exploitability, and a second opinion strengthens an existing finding rather than duplicating it.

Value should compound. ADR 0005 asks how a free, offline tool can get better over time, and answers with accumulation rather than training.

The UI is replaceable. ADR 0004 records a framework migration completed without touching a single engine — the strongest available evidence that the layering in ARCHITECTURE.md is real.

Adding an ADR

1. Copy the structure of an existing record.
2. Number it sequentially: `0009-short-kebab-title.md`.
3. Fill in **Context** honestly — the constraints and pressures as they actually were, including the ones that were uncomfortable.
4. State the **Decision** in one sentence.
5. In **Consequences**, write the costs before the benefits. An ADR that lists only upsides is a press release.
6. In **Alternatives considered**, say why each was rejected. "We didn't think of it" is a valid and useful entry.
7. Add a row to the index above.

Related documents

- PROJECT_REPORT.md §5 — these decisions in narrative form
- ARCHITECTURE.md — the structure they produced
- ROADMAP.md — decisions still to be made
- KNOWLEDGE_TRANSFER.md — the tacit reasoning around them

ADR-0001 — No LLM in the core

Status: Accepted

Context

RedFlag produces documents that inform a commercial decision: whether to buy a company, at what price, and with what warranties. Those documents may be attached to a diligence pack, cited in negotiation, and read years later by people reconstructing why a decision was made.

Three properties follow from that audience:

1. **Reproducibility.** The same scan data must produce the same report. If two analysts run the same assessment and get materially different narratives, neither can be relied upon.
2. **Auditability.** A reader must be able to trace any statement back to the finding and the rule that produced it. "The model said so" is not a defensible provenance.
3. **Zero marginal cost.** A hard constraint from ADR-0003.

A large language model was the obvious way to generate fluent narrative from structured findings, and the temptation was real — the alternative is more work and less elegant prose. It fails all three properties. Temperature-zero sampling reduces variance but does not eliminate it, and it does nothing for auditability or cost. Local models remove the cost but add a GPU requirement, a multi-gigabyte download, and a new class of failure.

There is also a specific risk in this domain: an LLM asked to summarise security findings will occasionally produce a confident, well-written statement that is not supported by the data. In a deal document, a plausible-sounding hallucination is worse than no narrative at all.

Decision

RedFlag's core contains no LLM, and no dependency on any model — local or remote.

Every analytical output is produced by deterministic code:

- Risk scoring: a weighted formula plus lookup tables (`analysis/triage.py`)
- Attack-path reasoning: a rule-based MITRE ATT&CK expert system (`analysis/attack_brain.py`)
- Graph analytics: networkx (`analysis/attack_graph.py`)
- Narrative: condition-matched templates (ADR-0002)
- The knowledge base: accumulation and retrieval, never training (ADR-0005)

The README's roadmap lists an *optional* free-tier LLM layer over the brain. If built, it must be **additive and clearly labelled** — it must never replace or alter a deterministic output.

Consequences

Costs

- The narrative is less fluent than generated prose, and more repetitive across reports.
- Every sentence must be written by hand into `config/narrative_blocks.yaml`. Adding a nuance means adding a template and a condition.
- The attacker-brain reasons only about patterns explicitly encoded in it. It cannot recognise a novel technique or an unusual chain.
- No natural-language querying of findings.
- Some reviewers will read "no AI" as "unsophisticated". The sophistication is in the scoring and the graph analytics, but it is less visible than a chat box.

Benefits

- **The same input always produces the same output.** Two analysts, two machines, same report.
- Every statement traces to a finding, a rule and a template — fully auditable.
- **Zero marginal cost per scan.** No API bill, no rate limit, no vendor dependency.
- Fully offline. Works on an air-gapped machine, in a data room with no internet, on a plane.
- No hallucination risk. The tool cannot invent a finding or overstate one.
- No target data is ever sent to a model provider — a genuine concern when the data describes a third party's security weaknesses under an NDA.
- The engines are trivially testable: 143 tests, nine seconds, no network, no mocking.

Alternatives considered

A hosted LLM API (OpenAI, Anthropic, or similar). The best narrative quality by a distance. Rejected on all three counts: it costs money per scan, output varies between runs, and it would transmit a third party's security findings to a model provider — a hard conversation to have with a target that agreed to be scanned but not to have its weaknesses uploaded.

A local open-weights model (Llama, Mistral). Removes the cost and the egress problem. Rejected because it adds a GPU expectation and a multi-gigabyte download to a tool otherwise installable with `pip install -r requirements.txt`, and because it does not solve reproducibility or auditability — the two properties that actually mattered.

LLM for narrative only, deterministic scoring. The most tempting middle ground, and the closest call. Rejected because the narrative is what most readers actually read. A scored table nobody reads plus a paragraph that varies between runs is not meaningfully more auditable than a fully generated report.

Retrieval-augmented generation over the brain. Attractive in principle. Deferred rather than rejected — it remains on the roadmap as an optional layer, precisely because the brain's retrieval half already works deterministically without it.

Related

- ADR-0002 — what replaced generated prose

- ADR-0003 — the cost constraint that reinforced this
- ADR-0005 — improvement without training
- BRAIN_KNOWLEDGE_BASE.md

ADR-0002 — Deterministic YAML narrative engine

Status: Accepted

Context

ADR-0001 ruled out an LLM, but the requirement it was meant to serve did not go away: a report needs prose. A table of 47 scored findings does not tell a deal team what to think. Someone still has to write "*This assessment identified 2 deal-killer findings that present unacceptable risk to the proposed acquisition.*"

The question became how to generate that prose deterministically, and at what level of sophistication. Several tiers were available, from simple f-strings up to a full rules engine.

The design was settled as **"Option B, Tier 3"**: a condition-matched template block system backed by YAML — more capable than string formatting, less than a general inference engine.

Two audiences drove the requirements. **Analysts** need to control exactly what a client-facing document says, without a developer in the loop. **Reviewers** need to verify that the wording matches the data.

Decision

Narrative text is generated by selecting the first matching template block from a YAML knowledge base and substituting variables into it.

The mechanism, in `narrative/blocks.py`:

1. Six sections in `config/narrative_blocks.yaml`: `executive_summary`, `maturity_summary`, `cost_summary`, `day1_summary`, `finding_detail`, `remediation_priority`.
2. Each block declares a `priority`, a `when` condition dict, and a `text` template.
3. Blocks are sorted by `priority` ascending; **the first block whose conditions all match wins**.
4. `{variable}` placeholders are substituted from a context dict built by `narrative/engine.py`. Unknown placeholders are left in place unchanged.

```
executive_summary:
  - id: ex_deal_killer_present
    priority: 1
    when: {has_deal_killers: true}
    text: >
      This assessment identified {deal_killer_count} deal-killer finding(s) that
      present unacceptable risk to the proposed acquisition of {target}.
```

The general catch-all block carries the highest priority number, so it fires only when nothing more specific matched.

Consequences

Costs

- Every sentence must be authored by hand. Nuance requires a new block and a new condition.
- Reports repeat across assessments. Two similar targets produce near-identical prose, which reads as templated — because it is.
- Combinatorial pressure: a condition on five variables would need many blocks to cover meaningfully. In practice the sections stay small (3–6 blocks each), which is a discipline as much as a limit.
- A typo in a placeholder name renders as a literal `{brace}` in the output rather than raising. Visible, but only if someone reads the report.
- No cross-sentence coherence. Blocks are selected independently, so adjacent paragraphs can repeat a fact.

Benefits

- **Identical input produces identical output.** Verifiable, and verified by 15 tests.
- **An analyst can change the wording without touching code.** For a client-facing document this is the single most valuable property in the design.
- Fully auditable: any sentence traces to a block `id`, and the condition that selected it is written down.
- Trivially fast — no model, no network, no latency.
- Localisation is a translated YAML file, not a re-architecture.
- Testable in isolation: block selection, priority ordering and substitution are each covered directly.

Alternatives considered

Plain f-strings in Python. Simplest possible approach. Rejected because it puts client-facing wording inside code, requiring a developer and a release for every phrasing change — the exact coupling this decision exists to break.

A general rules engine (`durable_rules`, `pyknow`, or similar). More expressive conditions, forward chaining, the full apparatus. Rejected as over-engineered: the actual conditions in use are flat equality checks on 3–5 boolean and string variables. A dependency and a concept the next maintainer must learn, bought for nothing.

Jinja2 templates. Genuinely tempting, and arguably a better fit for complex substitutions. Rejected because Jinja's power is its weakness here — loops and conditionals inside templates would let logic migrate out of Python and into the content layer, which is precisely the coupling being avoided. The `{variable}` substitution in `blocks.py` is deliberately too weak to express logic.

Markdown fragments assembled by rules. Similar effect with more file-system machinery. YAML keeps conditions and text adjacent, which makes reviewing the whole narrative a matter of reading one file.

Related

- ADR-0001 — the decision this implements
- CONFIGURATION.md §7 — the block format and how to edit it

- MODULE_REFERENCE.md — the `narrative/` API
- TEST_PLAN.md — the 15 narrative tests

ADR-0003 — Entirely free, no paid API

Status: Accepted

Context

Commercial security-assessment platforms carry per-scan or per-asset pricing. That model changes behaviour in a way that is easy to miss: **when a scan costs money, people scan less.** In M&A diligence specifically, this means a marginal target does not get assessed, or a re-scan after remediation does not happen, or the tool is reserved for the largest deals.

RedFlag was built as an internship project without a budget, so the constraint arrived from outside. But examining it showed the constraint was worth adopting on the merits.

The question was whether a useful assessment could actually be built from free sources. The investigation found that the security ecosystem's most authoritative data is already public:

- **CISA KEV** — the definitive list of exploited-in-the-wild vulnerabilities. Free.
- **EPSS (FIRST.org)** — exploitation probability. Free, no key.
- **NVD (NIST)** — CVSS scores. Free, rate-limited without a key.
- **crt.sh** — certificate transparency. Free.
- **LeakIX** — breach and exposure indexing. Free for basic queries.
- **Nmap, Nuclei, OpenVAS, OWASP ZAP** — all free and open source.
- **MITRE ATT&CK** — free, with attribution.

The commercial data would have added: Shodan at scale, richer breach intelligence, vendor-specific vulnerability feeds. Valuable, but not load-bearing.

Decision

RedFlag must produce a complete assessment with zero paid services and zero API keys.

Concretely:

- **Ten of the fourteen integrations require no key at all.**
- The two keyed integrations — Shodan and Vulners — are **optional**, use free tiers, and gate no feature.
- Every keyed path has a free alternative: Shodan has an upload slot that takes priority over the live call; Vulners' exploit signal is largely covered by KEV and EPSS.
- No LLM API (ADR-0001), no GPU, no training cost (ADR-0005).

Consequences

Costs

- **Shodan's free tier is limited.** 1 credit per IP means a heavy user runs out. The upload slot is the mitigation, but it requires the target to supply an export.
- **NVD's unauthenticated rate limit is 5 requests per 30 seconds.** `fetch_cvss_batch()` caps its thread pool at 5 to respect it, which makes CVSS enrichment slow on a large finding set.
- No commercial vulnerability feed, so coverage depends on what CVE data is public.
- LeakIX is the only breach source. No HIBP, no dark-web monitoring, no paid intelligence.
- **Free services can disappear or change terms.** crt.sh and LeakIX have no SLA and no contractual commitment to anyone.
- Free feeds are less reliable, which is why every scanner degrades silently — and that degradation is itself a real risk. See LIMITATIONS.md §4.

Benefits

- **Scan without hesitation.** No approval, no budget line, no per-target arithmetic. The behavioural effect is the point.
- Anyone can clone and run it — no procurement, no trial account, no sales call.
- No vendor lock-in and no renewal exposure.
- Fully offline-capable, which matters in a data room with no internet.
- **The constraint improved the product.** Being unable to buy commercial exploit data forced a proper look at KEV and EPSS — and EPSS's predictive signal (ADR-0007) turned out to be genuinely better for M&A than a retrospective commercial feed. The attacker-brain and the brain memory (ADR-0005) exist because "buy an AI service" was not available.

Alternatives considered

A paid Shodan plan. Would remove the credit limit and enable subnet scanning. Rejected as a *requirement*; the upload slot covers the common case. If someone has a plan, the tool uses it — the decision is that it must not *need* one.

An NVD API key. Free to obtain and raises the rate limit substantially. Not rejected on principle — it is simply not implemented. **This is arguably the single highest-value free improvement available**, since NVD's limit is the pipeline's slowest step, and the key costs nothing. Worth adding.

Commercial vulnerability intelligence (Recorded Future, Flashpoint, Mandiant). Better data, genuinely. Rejected on cost, and because a tool that requires a five-figure subscription is not a tool a small deal team will use.

A freemium model — free core, paid enrichment. Rejected because it makes the free path the degraded path, and the degraded path is where most users live. Better to make the free path the real product.

Requiring at least one API key. Would simplify the code by removing the "no key" branches. Rejected: the ability to clone and run immediately, with nothing, is what makes the tool adoptable.

Related

- ADR-0001 — the same constraint applied to narrative

- ADR-0005 — improvement without a training budget
- ADR-0007 — the free feed that improved the model
- INTEGRATIONS.md §16 — running with no keys
- LIMITATIONS.md §4 — the cost of free feeds failing silently

ADR-0004 — Reflex as the UI framework

Status: Accepted (superseded Streamlit, 2026-06-29)

Context

RedFlag's first UI was **Streamlit**, chosen for the reason everyone chooses it: an analytical Python tool gets a working interface in an afternoon with no frontend code.

It carried the project from the initial commit (2026-05-25) through the Maturity, Cost and Narrative engines, the DNS/TLS/breach scanners, and the Attack Path tab. Roughly a month of development.

By June the product had outgrown it. Three specific pressures:

1. **Seven tabs, each with real state.** Streamlit re-runs the entire script on every widget interaction. Keeping findings, a maturity assessment, a gap report, staged uploads and a cost rollup alive across that model meant increasingly elaborate `st.session_state` management, and the risk of losing a scan to an unrelated interaction.
2. **The interface had become the product's face.** RedFlag produces documents that go to deal teams. Streamlit's default aesthetic is unmistakably Streamlit, and fighting it — an "Executive Editorial / emerald" design direction was already defined — meant increasing amounts of injected CSS working against the framework.
3. **No real routing.** Streamlit's tab and multipage models do not give addressable URLs with independent lifecycles. `/day1` should be a link someone can send.

The scan pipeline in particular was uncomfortable: a long-running operation with staged file uploads, in a framework whose central abstraction is "re-run everything on every interaction".

Rewriting the UI in React with a Python API was the conventional answer, and was rejected: it doubles the language surface for a solo project and puts the presentation layer out of reach of the person who understands the domain.

Decision

Migrate to [Reflex](https://reflex.dev): pure-Python components that compile to a Next.js/React frontend with a WebSocket-backed Python state server.

The migration completed on 2026-06-29 `bf0d606`, "Migrate to Reflex UI; retire Streamlit app.py").

Key constraints adopted with it:

- `redflag_ui/` is **presentation only.** All state lives in one `RedFlagState` class, which calls the engines and flattens their output into flat dataclass view-models.
- Raw engine objects live in **backend-only vars** (leading underscore) so they are not serialised to the browser.
- Tailwind's preflight reset is deliberately omitted in `rxconfig.py` — `assets/redflag.css` owns the design and a utility reset would fight it.

The migration required zero changes to any engine. `analysis/`, `cost/`, `narrative/`, `reports/` and

`scanners/` were untouched. That is the strongest available evidence that the layering described in ARCHITECTURE.md is real rather than aspirational.

Consequences

Costs

- **Node.js is now a dependency.** The first launch compiles a Next.js frontend — roughly a minute, and Reflex offers to install Node if absent. A genuine regression against Streamlit's pure-Python install.
- **The build is fragile in synced folders.** OneDrive and Dropbox fight the Vite build, producing `EBUSY` errors and a blank page. The working checkout had to be moved out of OneDrive entirely.
- **The dev file-watcher resets backend state on any write inside the worktree.** This is why Nmap output goes to `%TEMP%/redflag_scans` and the brain to `~/RedFlag-Brain` — a constraint that must be understood before touching either path.
- **A compile error crashes the dev server.** Risky component edits need a `.render()` check in the venv first.
- Reflex-specific traps: `rx.upload` cannot take a `Var` `class_name`; dynamic widths and gradients must be precomputed string Vars, not f-strings over a Var.
- Reflex is a younger framework with a smaller community than Streamlit.
- **The Streamlit UI tests were removed and never replaced.** `redflag_ui/` currently has **no test coverage** — the largest gap in the suite.

Benefits

- **Nine real routes** with addressable URLs: `/`, `/findings`, `/attack`, `/maturity`, `/day1`, `/cost`, `/export`, `/privacy`, `/contact`.
- **Persistent server-side state.** No re-run model, so a long scan is not at risk from an unrelated widget interaction.
- **Full design control.** `assets/redflag.css` implements the intended aesthetic without fighting a framework. The risk donut is a CSS conic-gradient and the attack mind-map is precomputed SVG — no chart library, no JS graph library.
- Still pure Python. One language, one mental model, one debugger.
- The migration itself proved the architecture — and that proof is worth more than the framework.

Alternatives considered

Stay on Streamlit and work around it. Cheapest option. Rejected because the workarounds were already accumulating and the state-management pressure would grow with every new tab. The design ceiling was the deciding factor.

A React/Next.js frontend with a FastAPI backend. The conventional and most powerful answer. Rejected for a solo project: it doubles the language surface, splits the codebase, and requires maintaining an API contract between two halves. Worth revisiting only if a team forms.

Dash / Plotly. Mature, Python-native, good routing. Rejected on aesthetics — Dash's idiom is dashboards and charts, while RedFlag's output is closer to an editorial document. The "Executive Editorial" direction would have fought it much as it fought Streamlit.

Gradio. Optimised for ML demos and single-input/single-output flows. Wrong shape for a seven-tab stateful application.

A static report generator — no web UI at all. Genuinely attractive, and the closest thing to a real alternative: the PDF and CSV exports are arguably the product. Rejected because the maturity questionnaire, the What-If cost controls and the vendor-quote overrides are all interactive by nature.

Related

- ARCHITECTURE.md — the layering this migration validated
- KNOWLEDGE_TRANSFER.md §2 — the Reflex traps in full
- TROUBLESHOOTING.md §2.7 — the OneDrive build failure
- TEST_PLAN.md §7 — the UI coverage gap this created
- CHANGELOG.md — the migration in context

ADR-0005 — The brain accumulates, it never trains

Status: Accepted

Context

The attacker-brain (`analysis/attack_brain.py`) is a rule-based expert system. It maps findings to MITRE ATT&CK techniques and chains them into a kill-chain — and it produces exactly the same output on the hundredth scan as on the first. It does not improve.

That is a real limitation. An assessment tool used across many targets accumulates something valuable: which CVEs actually recur, which services are usually internet-facing, which attack chains keep appearing. A tool that discards that after every run is throwing away its own best asset.

The obvious framing is "add machine learning". That collided with two prior decisions — ADR-0001 (no model in the core) and ADR-0003 (no paid services, no GPU) — and with a third problem specific to the domain: **there is no training signal.** Supervised learning needs labels, and the label RedFlag would want is "did this finding lead to a breach?", which nobody has. Any trained model would be learning to predict its own prior outputs.

Restating the goal without the word "learning" clarified it: *what should scan 100 know that scan 1 did not?* The answer is not a better model. It is **more remembered context**.

Decision

The brain improves by accumulating and retrieving, never by training.

`analysis/brain_memory.py` maintains a persistent knowledge base at `~/RedFlag-Brain` (overridable with `REDFLAG_BRAIN_DIR`):

- `brain.json` — an index of prevalence counts: techniques, CVEs, services, ports, kill-chain signatures, deal tiers, and the targets seen.
- `vault/` — a real Obsidian vault of Markdown notes wired together with `[[wikilinks]]`.

Two operations, in a strict order:

1. **`recall(findings, plan)`** runs *first*, reading the prior corpus and returning prevalence-weighted insights: "*CVE-2021-44228 — seen in 3 prior scans · KEV*", "*this exact kill-chain has been seen before*".
2. **`learn_from_scan(findings, plan, target)`** runs *second*, folding this scan in.

The order is not incidental. Learning first would make every CVE report "seen in 1 prior scan" — tautological rather than informative.

A **sanitised seed** ships in the repository (`analysis/brain_seed/brain.json`) so a fresh clone starts pre-loaded rather than empty. `export_seed()` strips the `targets` map — the only target-identifying field — before writing it.

Consequences

Costs

- **The insights are prevalence, not prediction.** "Seen in 3 prior scans" tells you a CVE recurs across your estates; it says nothing about this target's risk. It is context, not intelligence.
- **Nothing decays.** A CVE seen once a year ago carries the same weight as one seen last week. Whether recency should matter is a genuinely open question.
- The brain does not feed back into scoring. It is presentational — a panel on the Attack path tab. It could inform triage; it does not.
- Value scales with corpus size, so a first-time user sees little beyond the shipped seed.
- **Privacy weight.** The local brain accumulates a durable, permanent record of every target assessed. Nothing expires. That is a retention obligation someone has to own.
- **`export_seed()` is a single unguarded boundary.** Six lines decide what is safe to publish, and they are currently untested.

Benefits

- **Genuinely free.** No GPU, no training run, no API, no model file. Pure standard library.
- **Fully deterministic and inspectable.** The brain is a JSON file. You can open it, read it, diff it, and see exactly why an insight appeared. No model has that property.
- No drift, no retraining schedule, no reproducibility problem.
- **The vault is a real artefact.** Open `~/RedFlag-Brain/vault` in Obsidian, use Graph View, and the accumulated knowledge is literally visible as a graph. That is a better explanation of "learning" to a non-technical stakeholder than any accuracy metric.
- Notes you write by hand become part of the brain — there is no training step to re-run.
- The shipped seed means a fresh clone is useful on day one.
- Works offline, on an air-gapped machine.

Alternatives considered

Train a model on accumulated findings. The obvious framing. Rejected on three counts: no training signal exists (no breach-outcome labels), it would violate ADR-0001 and ADR-0003, and a model trained on RedFlag's own outputs would learn to reproduce RedFlag's own biases with added opacity.

A vector database with embeddings over findings. Semantic similarity search. Rejected as disproportionate: the queries actually needed are exact lookups — "*have I seen this CVE ID before?*", "*have I seen this technique sequence?*" — which a dict answers in constant time. An embedding model plus a vector store, for exact-match lookups, is a dependency bought for nothing.

A SQLite knowledge base. More rigorous than JSON, with real queries. **The closest call.** Rejected because JSON is human-readable and diffable, which matters for the shipped seed — a reviewer can read a JSON seed before committing it and cannot read a `.db` file. If the brain grows to the point where the whole index no longer fits comfortably in memory, revisit this.

No memory at all. Simplest. Rejected because discarding the corpus wastes the one asset a repeatedly-used assessment tool naturally builds.

Feed prevalence into the risk score. Tempting — a CVE seen in 8 prior scans could nudge its score. **Deliberately not done**, because it would make the score depend on the operator's scan history: the same finding on the same host would score differently on two analysts' machines, destroying the reproducibility property ADR-0001 exists to protect.

Related

- ADR-0001 — the no-model constraint
- ADR-0003 — the no-cost constraint
- BRAIN_KNOWLEDGE_BASE.md — the full technical reference
- SECURITY_AND_PRIVACY.md §5 — committed vs. local data
- TEST_PLAN.md §7 — the untested sanitisation boundary

ADR-0006 — Evidence-strength multiplier

Status: Accepted

Context

RedFlag fuses evidence from up to twelve sources, and those sources are not equally trustworthy.

Consider two findings that are identical on every scored dimension — same CVSS, same exposure, same data sensitivity, same exploit status:

- **A:** OpenVAS ran an authenticated check against the host and confirmed the vulnerability is present and unpatched.
- **B:** Nmap read a version banner, and Shodan's database associates a CVE with that version string.

Under a purely additive weighted model these score identically. That is wrong. A is a verified fact; B is an inference from a banner that may be stale, spoofed, or belong to a backported build that was patched without changing its version string.

Getting this wrong has an asymmetric cost. If banner-derived findings outrank verified ones, an analyst chases false positives, loses confidence in the tool, and eventually stops trusting the ranking at all — at which point the ranking has no value.

Two shapes of solution were available: change the scored dimensions to encode confidence, or apply a separate confidence adjustment after scoring.

Decision

Every `Finding` carries an `evidence_strength`, and the weighted base score is multiplied by a factor derived from it.

```
risk_score = round(base_score * EVIDENCE_MULTIPLIERS[evidence_strength], 2)
```

STRENGTH	MULTIPLIER	MEANING	TYPICAL SOURCE
`CONFIRMED`	**1.00**	Directly verified by a tool that tested it	Nmap (the port *is* open); OpenVAS/ZAP/Nuclei; LeakIX (observed)
`CORRELATED`	**0.95**	Two independent sources agree	Nmap + Shodan port match
`UNKNOWN`	**0.90**	Not established	Default
`INFERRED`	**0.85**	Deduced rather than observed	"No DKIM found across 12 common selectors"
`EXTERNAL`	**0.80**	Third-party intelligence, unverified	Shodan's own CVE associations

Two supporting rules make it work:

- **Correlation upgrades, never downgrades.** When an OpenVAS, ZAP or Nuclei result matches an existing Nmap finding, the finding's evidence strength rises (ADR-0008). It never falls.
- **The range is deliberately narrow — 0.80 to 1.00.** A maximum 20% discount.

Consequences

Costs

- **Every scanner author must set `evidence_strength` honestly.** It is a judgement call embedded in each module, and an over-generous `CONFIRMED` silently inflates scores. This is the weakest point in the design: the discipline is convention, not enforcement.
- The multiplier is applied uniformly across all four dimensions, which is a simplification. Uncertainty about *exposure* is arguably different from uncertainty about *whether the vulnerability exists*.
- **The five values are judgement, not measurement.** Why 0.85 for `INFERRED` rather than 0.75? Because it felt proportionate. No empirical validation exists.
- `UNKNOWN` at 0.90 sits above `INFERRED` at 0.85, which is initially counter-intuitive — "we don't know" scoring higher than "we deduced it". It is deliberate precaution: an unset default should not be penalised more than an explicit weak inference.

Benefits

- **A verified finding outranks a banner-only inference at equal severity.** The ranking becomes trustworthy, which is the entire point of a ranking.
- **It creates an incentive that shapes behaviour correctly.** Uploading OpenVAS or ZAP output visibly moves findings up the list, so users supply better evidence — which makes the assessment better. The product rewards the behaviour it wants.
- Provenance is explicit and visible in the UI and the CSV export, so a reader can see *why* a finding ranks where it does.
- **The narrow 0.80–1.00 range is a deliberate safety property.** Evidence quality adjusts the ranking; it cannot suppress a finding. A `CVSS 10` / internet-facing / actively-exploited finding with the weakest possible evidence still scores about 80 and lands in the Critical tier. Weak evidence must never hide a severe problem.
- One multiplicative step, trivially auditable and directly tested.

Alternatives considered

Encode confidence into the CVSS input instead — discount the CVSS for weak evidence. Rejected because it corrupts a standard, externally-defined value. A reader seeing "CVSS 6.2" should be able to look that CVE up and see 6.2.

A fifth weighted dimension. Add evidence as a fifth term in the weighted sum. Rejected because evidence quality is *not* a risk dimension — it is a statement about how much to trust the other four. Multiplication expresses that; addition does not. Additively, a finding with perfect evidence and no risk would score points for being well-verified.

A wider multiplier range (0.50–1.00). Sharper separation between verified and inferred. Rejected as dangerous:

at 0.50, a genuinely severe finding with weak evidence could drop from Critical to Manageable and be deprioritised. **Under-ranking a real vulnerability is a worse failure than over-ranking a false positive** — the first hides a problem, the second wastes an hour.

Filter out low-evidence findings entirely. Rejected outright. Suppressing a finding because it is unverified is exactly how a diligence process misses something. Rank it lower and label it clearly; never hide it.

Bayesian confidence intervals per finding. More rigorous in principle. Rejected as unimplementable — it needs prior probabilities that nobody has, and it would make the score uninterpretable to the deal team who has to read it.

Related

- ADR-0008 — how correlation raises evidence strength
- DATA_MODEL.md §2 — the `EvidenceStrength` enum
- RESULTS_INTERPRETATION.md §2 — how to read it
- LIMITATIONS.md §2 — false positives and how this mitigates them

ADR-0007 — EPSS can promote exploit status

Status: Accepted

Context

Exploit status carries the heaviest weight in RedFlag's scoring model — 0.30, more than CVSS. The top value, `ACTIVE_EXPLOITATION`, also triggers a deal-killer override that forces the score to 100.

Its authoritative source is **CISA KEV**, which is excellent data with one structural property: it is **retrospective**. A CVE enters the KEV catalogue *after* exploitation has been observed and verified. The catalogue is a record of what has already happened.

That creates a window. A vulnerability can be trivially exploitable, have public proof-of-concept code circulating, and be days from mass exploitation — and RedFlag would score its exploit status as `UNKNOWN` (30 points) because nothing had entered the paperwork yet.

For M&A diligence the window matters more than usual. A deal closes on a date. The question is not "*has this been exploited?*" but "*will this be exploited before or shortly after we own it?*" Retrospective data answers the wrong question.

EPSS — the Exploit Prediction Scoring System, from the FIRST.org EPSS Special Interest Group — answers the right one. It publishes, per CVE, the probability of exploitation **in the next 30 days**, derived from a model over real-world exploitation telemetry. It is free, needs no API key, and updates daily.

The design question was whether EPSS should be purely informational, or whether it should change the score.

Decision

EPSS is fetched for every CVE, displayed on every finding, and a high probability promotes an otherwise-unknown exploit status.

```
EPSS_PROMOTE_THRESHOLD = 0.50

if epss >= 0.50 and exploit_status in ("unknown", "no_exploit"):
    exploit_status = ExploitStatus.PUBLIC_EXPLOIT
    raw_data["epss_promoted"] = True
```

Three constraints on the rule:

1. **Promotion only.** It never downgrades. A finding already at `ACTIVE_EXPLOITATION` is untouched, and one already at `PUBLIC_EXPLOIT` is unaffected.
2. **It promotes to `PUBLIC_EXPLOIT` (65), never to `ACTIVE_EXPLOITATION` (100).** A prediction — however strong — must not trigger the deal-killer override. That verdict is reserved for confirmed observation.

3. **It is marked.** `raw_data["epss_promoted"] = True` records that the status was inferred rather than sourced.

Score effect: **+10.5 points** from `UNKNOWN`, **+16.5** from `NO_EXPLOIT`.

EPSS runs **last in the pipeline before `triage_all()`**, because promotion changes the score.

Consequences

Costs

- **The 0.50 threshold is a round number, not a calibrated one.** No sensitivity analysis was run. It is defensible — a coin-flip chance of exploitation within a month is clearly material — but it is a judgement.
- A finding can be promoted on a prediction that does not come true. EPSS is probabilistic; a

0.60 CVE is 40% likely not to be exploited.

- The dependency is invisible when it fails. If the FIRST.org API is unreachable, no promotion occurs and the report simply looks cleaner — with no indication that a signal is missing.
- Adds a network round-trip per scan (batched 100 CVEs per request, so the cost is small).
- EPSS scores change daily, so re-running the same scan a week later can produce different tiers for the same findings. **Correct behaviour, but it means a RedFlag score is only reproducible against a fixed EPSS snapshot** — a caveat on the reproducibility property that ADR-0001 otherwise guarantees.

Benefits

- **The risk model becomes forward-looking.** It estimates what is about to happen rather than cataloguing what has.
- Closes the KEV window: a CVE with 92% exploitation probability is ranked like one with a known public exploit, days or weeks before it lands in the catalogue.
- **Free, keyless, and authoritative** — squarely within ADR-0003.
- EPSS percentages are displayed on every finding regardless of promotion, so an analyst sees the likelihood even when it did not change the tier.
- Partial cover for a Vulners key being absent — EPSS supplies an exploit signal that would otherwise require a paid-tier API.
- The promotion ceiling at `PUBLIC_EXPLOIT` keeps the deal-killer verdict reserved for confirmed facts. A prediction can raise urgency; it cannot, on its own, stop a deal.

Alternatives considered

Display EPSS but never let it change the score. The conservative option. Rejected because exploit status is 30% of the weighting, and leaving a strong exploitation signal out of the weighted dimension it directly speaks to means the score ignores the best available evidence. Analysts would have had to mentally re-rank the list — which is the tool's job.

Make EPSS a fifth weighted dimension. Score `epss × 100` as its own term. Rejected as double-counting: EPSS and exploit status measure the same underlying property — exploitability. Two correlated terms would let a single signal dominate the model.

A higher threshold (0.70 or 0.90). Fewer promotions, higher precision. Rejected because it would miss most of the window this decision exists to close — a CVE at 0.60 is genuinely urgent, and by the time it reaches 0.90 it is often already in KEV.

A lower threshold (0.20 or 0.30). More sensitive. Rejected because EPSS distributions are heavily skewed toward zero; a 0.20 threshold would promote a large share of all CVEs and destroy the signal's discriminating power.

Promote all the way to `ACTIVE_EXPLOITATION`. Rejected firmly. That value forces a deal-killer verdict with a score of 100. **A prediction must never, on its own, stop a deal** — that conclusion has to rest on confirmed observation.

Use the EPSS percentile instead of the raw probability. Percentile is relative to all CVEs, so a 99th-percentile score can still be a low absolute probability. The absolute probability is what the decision actually needs.

Related

- ADR-0003 — the constraint that surfaced EPSS
- ADR-0006 — the other confidence-handling decision
- RESULTS_INTERPRETATION.md §5 — reading EPSS in the UI
- INTEGRATIONS.md §10 — the EPSS API integration
- TEST_PLAN.md — the six EPSS tests

ADR-0008 — Uploads correlate into the Nmap layer

Status: Accepted

Context

RedFlag accepts scanner output from four external tools — OpenVAS/GVM, OWASP ZAP, Nuclei and Shodan — alongside its own Nmap scan. These overlap heavily: all five will report something about `10.0.0.5:443`.

The naive approaches both fail.

Concatenate everything. Upload an OpenVAS report and a ZAP report on a live scan, and the same HTTPS service appears three or four times as separate findings. The list length becomes a function of how many scanners were run rather than how much risk exists. Deal tier counts inflate. The Overview donut misleads. Cost estimates double-count. And an analyst reading the list cannot tell whether three entries mean three problems or one problem seen three times.

Let uploads replace the Nmap layer. OpenVAS data is richer, so use it instead. This loses the one thing Nmap uniquely establishes — the **complete port inventory that RedFlag itself observed**. It also throws away the Shodan correlation already applied to those findings, and it means the finding set changes shape depending on upload order.

The project's stated philosophy — "*every scanner contributes evidence to one unified risk picture, not disconnected scanner outputs*" — pointed at a third option.

Decision

Uploaded scanner results are correlated into the existing Nmap findings by `host:port`. A match upgrades the existing finding in place; only unmatched results become new findings.

Implemented identically in three modules:

```
merge_openvas_with_nmap(nmap_findings, openvas_findings)
merge_zap_with_nmap(nmap_findings, zap_findings)
merge_nuclei_with_nmap(nmap_findings, nuclei_findings)
```

On a `(host, port)` match, the existing finding is upgraded:

FIELD	RULE
`evidence_strength`	Raised toward `CONFIRMED` — a second tool verified it
`cvss_score`	`max()` — take the higher severity
`exploit_status`	`_higher_exploit()` — never downgrade
`cve_id, description, remediation`	Enriched with the richer detail

Upgrade only, in every direction. A correlation can strengthen a finding; it can never weaken one. The `_higher_exploit()` helpers and the `max()` on CVSS exist specifically to enforce that.

Shodan follows the same principle through a different function: `enrich_findings_with_shodan()` upgrades matched findings to `INTERNET_FACING`, `CORRELATED`, and `create_shodan_findings()` separately creates standalone findings for CVEs and risky ports that have no Nmap counterpart.

The merge order in `run_scan` is Nmap → Vulners → Shodan → OpenVAS → ZAP → Nuclei → DNS/TLS/breach → asset sensitivity → EPSS → triage.

Consequences

Costs

- **Correlation is `host:port` only.** A ZAP finding reported against a URL path rather than a port, or an OpenVAS result whose host is a hostname where Nmap recorded an IP, will not match — and will be added as a separate finding. Some duplication survives.
- Upgrading in place **loses per-scanner attribution**. The merged finding carries one `scanner_source`, so "which tool found this?" is no longer cleanly answerable for correlated findings.
- The merge functions mutate their input, which is efficient but makes them harder to reason about than pure transformations.
- Three near-identical merge implementations exist — a duplication noted as tech debt in KNOWN_ISSUES_AND_BACKLOG.md §3.
- Upload-only mode (no target, therefore no Nmap layer) takes a different path: uploads are simply appended, since there is nothing to correlate into.

Benefits

- **One finding per real problem.** Tier counts, the risk donut, and the cost model all describe risk rather than scanner count.
- **Nmap's port inventory is preserved** as the structural backbone. RedFlag always knows what it itself observed.
- **Evidence strength rises exactly when it should** — when two independent tools agree, which is precisely what ADR-0006 is designed to reward.
- **No double-counting in the cost model.** The deduplicator handles catalogue-level duplication; this handles finding-level duplication. Both are needed.
- **Uploading more data always improves the assessment and never inflates it.** That property is what makes the upload slots safe to use liberally — an analyst can add every artefact the target supplies without worrying about distorting the numbers.
- Uniform contract across OpenVAS, ZAP and Nuclei, so adding a fourth correlating scanner is a known shape of work.

Alternatives considered

Concatenate and deduplicate afterwards. Add everything, then collapse duplicates in a separate pass. Rejected because deduplication after the fact must decide which duplicate is canonical and how to combine their fields — the same merge logic, done later, on a larger set, with the intermediate inflated state visible if anything fails

partway.

Let uploads replace the Nmap layer. Rejected: loses RedFlag's own port inventory, discards Shodan enrichment already applied, and makes the result depend on upload order.

Keep every scanner's findings separate, in per-source sections. How most multi-scanner tools present results. Rejected because it is exactly the problem RedFlag exists to solve — a deal team needs one ranked list, not five lists to reconcile themselves.

Correlate on a fingerprint rather than `host:port` — service name, product, version, CVE. Would catch more matches, including the port-less cases. Rejected for this version as over-engineering with a real risk of *false* merges, which would be worse than the duplicates it prevents: two genuinely distinct problems collapsed into one is a finding lost. **The best candidate for a future improvement**, if paired with careful tests.

Correlate on CVE ID. Would match across hosts. Rejected because the same CVE on two different hosts is genuinely two findings with different exposure and different remediation work.

Related

- ADR-0006 — what correlation earns a finding
- DATA_MODEL.md §4 — the finding lifecycle
- ARCHITECTURE.md §4 — merge order in the pipeline
- MODULE_REFERENCE.md — the merge function signatures
- USER_GUIDE.md §1 — how this appears to a user

PART VIII

Contributing

Contribution, security-reporting and conduct policies

Contributing

Thanks for your interest. This is the short version — for depth, see docs/operations/DEVELOPERONBOARDING.md.

Before you start

RedFlag is a security scanner. Contributions must not add capability for unauthorised scanning, exploitation, or evasion of detection. RedFlag observes and reasons; it never attacks. Read docs/legal/AUTHORIZED_USE.md.

Never commit a secret. API keys live in `.env`, which is git-ignored. If you accidentally commit one, rotate it first and clean the history second.

Setup

```
git clone https://github.com/adityaa206/redflag.git
cd redflag
python -m venv venv

# Windows
.\venv\Scripts\Activate.ps1
# macOS / Linux
source venv/bin/activate

pip install -r requirements.txt
pytest tests/ -v            # expect: 143 passed
```

Two environment constraints:

- **Do not put the checkout in OneDrive, Dropbox or any synced folder.** The sync engine fights Reflex's Node build and produces `EBUSY` errors and a blank page.
- **Nmap is required for live scanning** and is discovered by absolute Windows path, not `PATH`. See docs/operations/INSTALLATION.md.

Branch and commit

```
git checkout -b short-descriptive-name
```

Never commit directly to `main`.

Commit messages: a short imperative subject line describing the effect, matching the existing history.

```
Add EPSS exploitation-probability enrichment
Fix Vulners NSE pipeline status: distinguish installed vs not-installed
docs: sync README data-flow diagram with Nuclei / EPSS / graph
```

Test

```
pytest tests/ -v            # everything
pytest tests/test_triage.py -v
pytest tests/ -k "day1"
```

All 143 tests must pass before you open a PR.

New behaviour requires a test. The suite observes three rules, which contributions should follow:

1. **No network access, ever.** Inject external data instead: EPSS takes a `scores` dict, Nuclei takes inline JSONL, OpenVAS and ZAP take fixture files. A test that hits the internet fails on a plane and lies in CI.
2. **No mocking framework.** The engines are pure functions over plain objects; construct real `Finding` objects and assert on real output. Needing heavy mocks is a signal the module has the wrong dependencies.
3. **Test the contract, not the implementation.** Assert scores, tiers, phases and totals — the things a user sees. This is what let the Streamlit → Reflex migration happen without touching a single test.

Code conventions

Match the surrounding code rather than importing your own style.

- `from __future__ import annotations`; modern generics `list[Finding]`, `str | None`).
- Dataclasses for engine outputs; Pydantic where validation is needed.
- Section banner comments mark logical blocks: `# ── Phase assignment + roadmap ──────────────────────────`
- A leading underscore means private to the module.
- **Normalise enums with `str(getattr(x, "value", x))`, never `str(x)`.** On an enum member the latter yields `"DealTier.CRITICAL"` and silently breaks every comparison.
- **Scanners never raise.** Wrap network I/O and return `[]` or an unchanged input. The deliberate exceptions are `run_nmap_scan()` (missing binary) and the upload parsers (malformed input), which raise because the user needs to know.
- **Upgrade, never downgrade.** Merge and enrichment helpers use `_higher_exploit()` and `max()` on CVSS, so a correlation can only strengthen a finding.
- Read config through `config/loader.py` — never open a YAML directly, and never from `redflag_ui/`.
- Backend-only Reflex vars carry a leading underscore so raw engine objects are not serialised to the browser.

The one architectural rule

Business logic does not go in `redflag_ui/`.

If a change would alter a number in a report, it belongs in `analysis/`, `cost/`, `scanners/` or `config/`. `redflag_ui/` calls engines and flattens their output into view-models. That boundary is why the interface framework was replaced without modifying a single engine, and it should be maintained.

Runtime writes stay outside the worktree

Nmap output goes to `%TEMP%/redflag_scans`; the brain to `~/RedFlag-Brain`. Writing inside the repository trips Reflex's dev file-watcher, which hot-reloads the backend **mid-scan** and loses the findings. Do not change these paths.

Adding things

WHAT	WHERE	CODE NEEDED
A cost line item	`config/remediation_catalog.yaml`	**None**
A Day-1 integration cost	`config/day1_cost_catalog.yaml`	**None**
Report wording	`config/narrative_blocks.yaml`	**None**
A maturity question	`config/maturity_questions.yaml`	**None**
A new scanner	`scanners/` + a `ScannerSource` member	Yes — see below
A new page	`redflag_ui/pages/` + `_PAGES` in `redflag_ui.py`	Yes

A new scanner must: produce `Finding` objects; expose either `run_my_scan(target)` or a `parse_*` + `merge_*_with_nmap` pair; set `evidence_strength` **honestly** (it multiplies the risk score — `CONFIRMED` means *verified*, not *observed*); wrap all I/O so failure returns `[]`; and be called from `RedFlagState.run_scan` before `triage_all()`. Add a fixture under `tests/fixtures/` and tests against it.

Note that YAML edits are cached per process — **restart the app** to see them.

Documentation

Code and docs ship together. If your change alters behaviour, update the relevant document:

CHANGE	UPDATE
Scoring, weights, thresholds	CONFIGURATION.md, RESULTS_INTERPRETATION.md
A new integration	INTEGRATIONS.md, SECURITY_AND_PRIVACY.md egress table
A public function signature	MODULE_REFERENCE.md
A schema or enum	DATA_MODEL.md
A user-visible feature	USER_GUIDE.md

CHANGE	UPDATE
Anything notable	CHANGELOG.md under `[Unreleased]`

A significant design decision needs an ADR — see docs/process/adr/README.md. Write the costs before the benefits; an ADR listing only upsides is a press release.

Opening a pull request

1. Rebase or merge the latest `main`.
2. `pytest tests/ -v` — 143 passing.
3. Push and open a PR against `main`.
4. In the description: what changed, why, and how you verified it. Screenshots for UI changes.
5. Link any issue or the backlog item it addresses.

Good first contributions are listed in docs/handover/KNOWN_ISSUES_AND_BACKLOG.md §5 — items 1 to 3 are deliberately sized as a first change.

Reporting

- **A bug in RedFlag** — open a GitHub issue with steps to reproduce, expected versus actual behaviour, your OS and Python version, and whether Nmap and Nuclei are installed. Never paste an API key or real target data.
- **A security vulnerability in RedFlag itself** — do **not** open a public issue. See SECURITY.md.

Code of conduct

By participating you agree to the Code of Conduct.

Related documents

- docs/operations/DEVELOPER_ONBOARDING.md — the full guide
- docs/technical/ARCHITECTURE.md — structure and extension points
- docs/testing/TEST_PLAN.md — the suite and its gaps
- docs/handover/KNOWLEDGE_TRANSFER.md — the gotchas

Security Policy

How to report a vulnerability in RedFlag itself.

Scope

This policy covers security defects in the RedFlag codebase — for example:

- Code injection through an uploaded scanner file (XML, JSONL, JSON or XLSX)
- Command injection via the target input or a subprocess call
- Path traversal in file handling or export
- Leakage of API keys through logs, process listings, error messages or exports
- A flaw in `BrainMemory.export_seed()` that lets target-identifying data reach the committed seed
- Any way the local web interface could be reached or driven by a remote party

Out of scope:

- Vulnerabilities in the systems RedFlag scans. Report those to the system's owner.
- Vulnerabilities in third-party tools RedFlag integrates with (Nmap, Nuclei, OpenVAS, ZAP, Shodan, LeakIX and others). Report those to the respective projects.
- The absence of authentication on the local web interface. This is a **documented design decision** — RedFlag is a single-user local tool bound to `localhost` and must never be exposed to a network. See docs/legal/SECURITY_AND_PRIVACY.md §7.
- Reports that RedFlag "can be used to scan systems". That is what it is for; lawful use is the operator's responsibility under docs/legal/AUTHORIZED_USE.md.

Reporting a vulnerability

Please do not open a public GitHub issue for a security vulnerability.

Report it privately through **GitHub private vulnerability reporting** — the repository's **Security** tab → *Report a vulnerability*. This is the only reporting channel, and it is the right one: the disclosure, the discussion and the fix stay in one place, visible only to the maintainers until a fix is published.

Please include:

- A description of the issue and its impact
- Steps to reproduce, or a proof of concept
- The affected file and, if you have it, a suggested fix
- Your commit or version, OS, and Python version

Do not include real API keys, real target data, or anything belonging to a third party.

What to expect

RedFlag is a personal project maintained on a best-effort basis. There is no service-level commitment, and no response time is guaranteed — a promise that cannot be kept is worse than none at all.

What a reporter can expect in practice: an acknowledgement once the report has been read, an assessment of whether the issue is in scope under the section above, and notification when a fix or a documented mitigation lands. Credit in the commit or release notes is given unless the reporter prefers otherwise.

Please give a reasonable opportunity to address the issue before disclosing it publicly.

Supported versions

VERSION	SUPPORTED
`main` **(latest)**	Yes
Anything earlier	No

The repository carries no git tags and has no released versions. **Only the current `main` branch is supported.** Always update to latest before reporting.

Security posture

RedFlag is a **local, single-user desktop application**. It has no server deployment, no database, no authentication layer, and no multi-user support. It binds to `localhost` only.

The most important operational security control is one the user applies: **do not expose the backend port to a network.** There is no authentication, so anyone who can reach it can launch an active Nmap scan against an arbitrary target from your machine and your IP address.

Full detail — data handling, the egress table, secrets management, and what is committed versus local — is in docs/legal/SECURITY_AND_PRIVACY.md.

Secrets

API keys live only in `.env`, which is git-ignored. `.env.example` is the committed template and contains placeholders only.

As of 2026-07-27, **no API key, token or credential appears in any tracked file** in this repository.

If you find a real secret committed anywhere — including in the git history — please report it privately using the process above and **do not** post the value in an issue.

Rotation procedures: docs/handover/ACCESS_AND_CREDENTIALS.md §4.

Related documents

- docs/legal/SECURITY_AND_PRIVACY.md — data handling and egress
- docs/legal/AUTHORIZED_USE.md — lawful operation of the scanner
- docs/handover/ACCESS_AND_CREDENTIALS.md — key inventory and rotation
- CONTRIBUTING.md — reporting non-security bugs

Code of Conduct

Our Pledge

We as members, contributors, and leaders pledge to make participation in our community a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socio-economic status, nationality, personal appearance, race, caste, color, religion, or sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open, welcoming, diverse, inclusive, and healthy community.

Our Standards

Examples of behavior that contributes to a positive environment for our community include:

- Demonstrating empathy and kindness toward other people
- Being respectful of differing opinions, viewpoints, and experiences
- Giving and gracefully accepting constructive feedback
- Accepting responsibility and apologizing to those affected by our mistakes, and learning from the experience
- Focusing on what is best not just for us as individuals, but for the overall community

Examples of unacceptable behavior include:

- The use of sexualized language or imagery, and sexual attention or advances of any kind
- Trolling, insulting or derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information, such as a physical or email address, without their explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

Project-specific standards

RedFlag is a security tool. In addition to the above, participation in this community requires that contributors:

- Do not use the project, its issue tracker, or its discussions to solicit, coordinate, or assist unauthorised scanning or access of systems.
- Do not post real target data, real scan results belonging to a third party, or any API key or credential in issues, pull requests, or discussions.
- Report security vulnerabilities in RedFlag privately, following SECURITY.md, rather than in a public issue.

Enforcement Responsibilities

Community leaders are responsible for clarifying and enforcing our standards of acceptable behavior and will take appropriate and fair corrective action in response to any behavior that they deem inappropriate, threatening, offensive, or harmful.

Community leaders have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, and will communicate reasons for moderation decisions when appropriate.

Scope

This Code of Conduct applies within all community spaces, and also applies when an individual is officially representing the community in public spaces. Examples of representing our community include using an official email address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported privately to the maintainers through the repository's **Security** tab → *Report a vulnerability*, which opens a private channel visible only to the maintainers. Conduct reports are welcome there even though the form is labelled for security issues; it is the repository's only private reporting route.

All complaints will be reviewed and investigated promptly and fairly.

All community leaders are obligated to respect the privacy and security of the reporter of any incident.

Enforcement Guidelines

Community leaders will follow these Community Impact Guidelines in determining the consequences for any action they deem in violation of this Code of Conduct:

1. Correction

Community Impact: Use of inappropriate language or other behavior deemed unprofessional or unwelcome in the community.

Consequence: A private, written warning from community leaders, providing clarity around the nature of the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.

2. Warning

Community Impact: A violation through a single incident or series of actions.

Consequence: A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, for a specified period of time. This includes avoiding interactions in community spaces as well as external channels like social media. Violating these

terms may lead to a temporary or permanent ban.

3. Temporary Ban

Community Impact: A serious violation of community standards, including sustained inappropriate behavior.

Consequence: A temporary ban from any sort of interaction or public communication with the community for a specified period of time. No public or private interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, is allowed during this period. Violating these terms may lead to a permanent ban.

4. Permanent Ban

Community Impact: Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior, harassment of an individual, or aggression toward or disparagement of classes of individuals.

Consequence: A permanent ban from any sort of public interaction within the community.

Attribution

This Code of Conduct is adapted from the [Contributor Covenant][homepage], version 2.1, available at [https://www.contributor-covenant.org/version/2/1/code_of_conduct.html][v2.1].

Community Impact Guidelines were inspired by [Mozilla's code of conduct enforcement ladder][Mozilla CoC].

For answers to common questions about this code of conduct, see the FAQ at [https://www.contributor-covenant.org/faq][FAQ]. Translations are available at [https://www.contributor-covenant.org/translations][translations].

[homepage]: https://www.contributor-covenant.org [v2.1]: https://www.contributor-covenant.org/version/2/1/code_of_conduct.html [Mozilla CoC]: https://github.com/mozilla/diversity [FAQ]: https://www.contributor-covenant.org/faq [translations]: https://www.contributor-covenant.org/translations